

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ VÀ XÂY DỰNG MÔ HÌNH HỆ
THỐNG NHẬN DẠNG CHUYỂN ĐỘNG TAY
ỨNG DỤNG HỌC MÁY TRONG ĐIỀU KHIỂN
ĐỐI TƯỢNG TRÒ CHƠI**

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

Sinh viên: **Nguyễn Đỗ Đức Hoan**

MSSV: 22139022

Võ Văn Nam

MSSV: 22139043

Hướng dẫn: ThS. **Trịnh Quốc Thanh**

TP. HỒ CHÍ MINH, 06/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ - THÔNG TIN



HCM-UTE

ĐỒ ÁN TỐT NGHIỆP

**THIẾT KẾ VÀ XÂY DỰNG MÔ HÌNH HỆ
THỐNG NHẬN DẠNG CHUYỂN ĐỘNG TAY
ỨNG DỤNG HỌC MÁY TRONG ĐIỀU KHIỂN
ĐỐI TƯỢNG TRÒ CHƠI**

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

Sinh viên: **Nguyễn Đỗ Đức Hoan**

MSSV: 22139022

Võ Văn Nam

MSSV: 22139043

Hướng dẫn: ThS. **Trịnh Quốc Thanh**

TP. HỒ CHÍ MINH, 06/2026

THÔNG TIN ĐỒ ÁN TỐT NGHIỆP

1. Thông tin sinh viên

- Họ và tên sinh viên: Nguyễn Đỗ Đức Hoan
- Mã số sinh viên: 22139022
- Họ và tên sinh viên (nếu có): Võ Văn Nam
- Mã số sinh viên (nếu có): 22139043
- Lớp: 22139C
- Ngành: Hệ thống nhúng và IoT
- Khoa: Điện – Điện tử
- Email:
- Số điện thoại:

2. Thông tin đề tài

- Tên đề tài (Tiếng Việt): Thiết kế và xây dựng hệ thống nhận dạng chuyển động tay ứng dụng học máy trong điều khiển đối tượng trò chơi.
- Tên đề tài (Tiếng Anh): Design and implementation of a hand gesture recognition system using machine learning for controlling game objects.
- Loại hình:
 - ☐ Nghiên cứu
 - ☒ Thiết kế
 - ☐ Chế tạo
 - ☐ Mô phỏng
 - ☐ Khác:
- Thời gian thực hiện:
 - Ngày giao đề tài: ...03../...03../...2026.....
 - Ngày hoàn thành:/...../.....

3. Giảng viên hướng dẫn

- Họ và tên: Trịnh Quốc Thanh
- Học hàm/Học vị: Thạc sĩ
- Đơn vị công tác:
- Email: tqthanh@hcmute.edu.vn

5. Tóm tắt nội dung đồ án

Đề tài tập trung nghiên cứu và xây dựng hệ thống nhận dạng chuyển động tay nhằm điều khiển đối tượng trong môi trường trò chơi. Hệ thống thực hiện quá trình thu nhận, xử lý và nhận dạng dữ liệu chuyển động để chuyển đổi thành các lệnh điều khiển tương ứng theo thời gian thực. Nội dung nghiên cứu góp phần ứng dụng học máy và tương tác người – máy vào lĩnh vực điều khiển và giải trí thông minh.

7. Kết quả đạt được

- ☒ Báo cáo đồ án tốt nghiệp.
- ☒ Mô hình / Thiết bị / Phần mềm hoàn chỉnh.
- ☒ Mã nguồn chương trình.
- ☒ Kết quả mô phỏng và thực nghiệm.
- ☐ Bài báo khoa học (nếu có).
- ☐ Sản phẩm khác:

1. 8. Kiểm tra đạo văn

- Phần mềm kiểm tra:
- Ngày kiểm tra:/...../.....
- Tỷ lệ tương đồng (Similarity Index): %
- Kết luận:
 - ☐ Đạt yêu cầu theo quy định của Nhà trường.
 - ☐ Không đạt yêu cầu theo quy định của Nhà trường.

9. Cam kết của sinh viên

Tôi cam kết rằng đồ án tốt nghiệp này là kết quả nghiên cứu và thực hiện của riêng tôi dưới sự hướng dẫn của giảng viên hướng dẫn. Các tài liệu tham khảo, số liệu, hình ảnh và kết quả nghiên cứu của người khác đều được trích dẫn đầy đủ theo đúng quy định. Tôi hoàn toàn chịu trách nhiệm trước Nhà trường về tính trung thực và tính hợp pháp của nội dung đồ án.

Sinh viên thực hiện

(Ký và ghi rõ họ tên)


Nguyễn Đỗ Đức Hoàn


Võ Văn Tâm

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP
(Dành cho giảng viên hướng dẫn)

Đề tài: THIẾT KẾ VÀ XÂY DỰNG MÔ HÌNH HỆ THỐNG NHẬN DẠNG CHUYÊN ĐỘNG TAY ỨNG DỤNG HỌC MÁY TRONG ĐIỀU KHIỂN ĐỐI TƯỢNG TRÒ CHƠI

Sinh viên: Nguyễn Đỗ Đức Hoan **MSSV:** 22139022
Võ Văn Nam **MSSV:** 22139043

Hướng dẫn: ThS. Trịnh Quốc Thanh

Nhận xét bao gồm các nội dung sau đây:

- 1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:** Đề tài có mục tiêu rõ ràng, phù hợp thực tiễn và có khả năng ứng dụng thực tế.
- 2. Phương pháp thực hiện/ phân tích/ thiết kế:** Phương pháp nghiên cứu phù hợp, thiết kế hệ thống hợp lý và có cơ sở khoa học, phân tích và đánh giá đầy đủ.
- 3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:** Kết quả đạt được bám sát mục tiêu đề ra và được đánh giá bằng thực nghiệm.
- 4. Kết luận và đề xuất:** Kết luận phù hợp với kết quả nghiên cứu, có đề xuất hướng phát triển trong tương lai.
- 5. Hình thức trình bày và bố cục báo cáo:** Trình bày chuẩn cấu trúc báo cáo theo yêu cầu của bộ môn.
- 6. Kỹ năng chuyên nghiệp và tính sáng tạo:** Có nỗ lực tự tìm hiểu kỹ năng lập trình và thi công phần cứng. Giao tiếp lịch sự, có kỹ năng làm việc nhóm.
- 7. Tài liệu trích dẫn:** Có sử dụng tài liệu tham khảo phù hợp với đề tài. Việc trích dẫn tài liệu đã tuân thủ định dạng theo quy định.
- 8. Đánh giá về sự trùng lặp của đề tài:** Đề tài là sự nỗ lực tự tích hợp và thực hiện của sinh viên, không có dấu hiệu sao chép toàn bộ. Kết quả kiểm tra đạo văn ở mức 29%.
- 9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa:** Thực nghiệm phần cứng chưa đạt nhiều chức năng, hệ thống hoạt động còn dao động nhiều, chưa có sự cân bằng ổn định lâu dài. Cần bổ sung thêm để tăng tính ổn định của hệ thống.
- 10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên:** Có sự kiên trì, thái độ cầu thị, biết lắng nghe và hợp tác tốt với giáo viên hướng dẫn.

Đề nghị của giảng viên hướng dẫn

Báo cáo đạt yêu cầu của một khóa luận tốt nghiệp. Được phép bảo vệ khóa luận tốt nghiệp.

Tp. HCM, ngày 26 tháng 6 năm 2026

Người nhận xét

(Ký và ghi rõ họ tên)



ThS. Trịnh Quốc Thanh

LỜI CẢM ƠN

Lời đầu tiên, em xin bày tỏ lòng biết ơn sâu sắc đến quý Thầy, Cô giáo khoa Điện - Điện tử, trường Đại học Công nghệ Kỹ thuật TP.HCM, những người đã tận tình truyền đạt kiến thức và tạo điều kiện thuận lợi cho em trong suốt quá trình học tập tại trường.

Đặc biệt, em xin gửi lời cảm ơn chân thành và sâu sắc nhất tới ThS. Trịnh Quốc Thanh. Trong suốt thời gian thực hiện đề án tốt nghiệp ngành Hệ thống nhúng và IoT, Thầy đã luôn dành thời gian hướng dẫn, đưa ra những định hướng chuyên môn quý báu và động viên em vượt qua những khó khăn để hoàn thành tốt đề tài. Những kinh nghiệm và phong cách làm việc khoa học của Thầy là hành trang quý giá cho em trong sự nghiệp sau này.

Cuối cùng, em cũng xin cảm ơn các bạn sinh viên lớp 22139 đã cùng chia sẻ kiến thức và hỗ trợ em trong những giai đoạn thực nghiệm khó khăn nhất.

Dù đã cố gắng hoàn thiện đề án với tất cả tâm huyết, nhưng do kiến thức còn hạn chế, chắc chắn bản báo cáo không tránh khỏi những thiếu sót. Em rất mong nhận được sự chỉ bảo và đóng góp ý kiến từ quý Thầy, Cô.

Em xin chân thành cảm ơn!

Nhóm thực hiện đề án tốt nghiệp
(Ký và ghi rõ họ tên)



Nguyễn Đỗ Đức Hoan Võ Văn Nam

LỜI CAM ĐOAN

Chúng em xin cam đoan đề tài "Thiết kế và xây dựng mô hình hệ thống nhận dạng chuyển động tay ứng dụng học máy trong điều khiển đối tượng trò chơi" là công trình nghiên cứu do nhóm trực tiếp thực hiện dưới sự hướng dẫn của ThS. Trịnh Quốc Thanh.

Các nội dung, kết quả thực nghiệm và số liệu được trình bày trong đồ án là hoàn toàn trung thực và không sao chép từ bất kỳ công trình, đồ án hay luận văn của tác giả khác. Tất cả các nguồn tài liệu tham khảo, số liệu trích dẫn đều đã được ghi chú và liệt kê đầy đủ trong danh mục tài liệu tham khảo.

Chúng em xin hoàn toàn chịu trách nhiệm trước Hội đồng và Nhà trường về tính xác thực của lời cam đoan này.

Nhóm thực hiện đồ án tốt nghiệp
(Ký và ghi rõ họ tên)



Nguyễn Đỗ Đức Hoan Võ Văn Nam

TÓM TẮT NỘI DUNG

Đề tài tập trung nghiên cứu và xây dựng một hệ thống điều khiển đối tượng trò chơi bằng chuyển động tay của người dùng. Thay vì sử dụng các thiết bị điều khiển truyền thống, hệ thống hướng đến việc thu nhận chuyển động tự nhiên của tay, xử lý dữ liệu và chuyển đổi thành các lệnh điều khiển tương ứng trong môi trường trò chơi.

Nội dung chính của đề tài bao gồm thiết kế phần cứng thu nhận dữ liệu, xây dựng chương trình xử lý tín hiệu cảm biến, truyền dữ liệu giữa các khối trong hệ thống, xây dựng mô hình học máy để nhận dạng chuyển động tay và tích hợp kết quả nhận dạng vào môi trường trò chơi Unity. Dữ liệu sau khi được thu nhận sẽ được xử lý, trích xuất đặc trưng và đưa vào mô hình để phân loại các trạng thái chuyển động cơ bản. Kết quả nhận dạng sau đó được sử dụng để điều khiển đối tượng trong trò chơi theo thời gian thực.

Thông qua đề tài, người thực hiện có thể vận dụng các kiến thức về hệ thống nhúng, cảm biến, truyền thông dữ liệu, học máy trong trí tuệ nhân tạo vào một ứng dụng cụ thể. Hệ thống không chỉ có ý nghĩa trong phạm vi điều khiển trò chơi mà còn có thể được phát triển theo hướng các ứng dụng tương tác người – máy, thiết bị hỗ trợ điều khiển không tiếp xúc hoặc các mô hình mô phỏng sử dụng cử chỉ tay làm phương thức điều khiển.

Mục lục

1	GIỚI THIỆU	1
1.1	Tổng quan về nhận dạng chuyển động tay trong điều khiển tương tác . .	1
1.1.1	Tổng quan	1
1.2	Hiện trạng nghiên cứu và ứng dụng hệ thống nhận dạng chuyển động tay	2
1.2.1	Ngoài nước	2
1.2.2	Trong nước	3
1.3	Mục tiêu đề tài	5
1.4	Đối tượng và phạm vi nghiên cứu	5
1.4.1	Đối tượng nghiên cứu	5
1.4.2	Phạm vi nghiên cứu	5
1.5	Cách tiếp cận và phương pháp nghiên cứu	6
1.6	Nội dung nghiên cứu và tiến độ thực hiện	6
1.7	Sản phẩm	8
2	CƠ SỞ LÝ THUYẾT VÀ LỰA CHỌN PHƯƠNG ÁN	9
2.1	Tổng quan về hệ thống nhận dạng chuyển động tay điều khiển đối tượng trò chơi	9
2.2	Lựa chọn cảm biến MPU	9
2.2.1	Cảm biến MPU6050	10
2.2.2	Cảm biến MPU9250	10
2.2.3	Cảm biến Wit Sensor	11
2.3	Lựa chọn cảm biến lực FSR	12
2.3.1	Cảm biến FSR402	13
2.3.2	Cảm biến FSR408	13
2.3.3	Cảm biến SingleTact Force Sensor	13
2.3.4	Tổng hợp và lựa chọn cảm biến lực	14
2.4	Lựa chọn vi điều khiển đọc và xử lý dữ liệu cảm biến	14
2.4.1	STM32	15

2.4.2	MSP430	15
2.4.3	Arduino Uno	16
2.4.4	Tổng hợp và lựa chọn vi điều khiển	16
2.5	Lựa chọn phương án truyền dữ liệu và kết nối IoT	17
2.5.1	Giao tiếp UART giữa STM32 và ESP32	17
2.5.2	ESP32 và kết nối Wi-Fi	17
2.5.3	Giao thức UDP	18
2.6	Lựa chọn phương pháp nhận dạng chuyển động tay bằng trí tuệ nhân tạo	18
2.6.1	Thuật toán Random Forest	19
2.6.2	Thuật toán K-Nearest Neighbors	20
2.6.3	Thuật toán Gaussian Naive Bayes	21
2.6.4	Tổng hợp và lựa chọn phương pháp nhận dạng	22
2.7	Các thuật toán lọc tín hiệu cảm biến	22
2.7.1	Bộ lọc IIR cho dữ liệu gia tốc và con quay hồi chuyển	22
2.7.2	Tính toán góc Roll và Pitch từ gia tốc kế	23
2.7.3	Bộ lọc bổ sung Complementary Filter	24
2.7.4	Bộ lọc IIR cho tín hiệu FSR	25
2.7.5	Tổng hợp thuật toán lọc trong hệ thống	25
2.8	Phần mềm Unity trong điều khiển đối tượng trò chơi	26
2.8.1	Tổng quan về Unity	26
2.8.2	Nhận dữ liệu điều khiển từ UDP	26
2.8.3	Điều khiển nhân vật hoặc đối tượng trong trò chơi	27
3	THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG	29
3.1	Yêu cầu về chức năng thiết kế hệ thống	29
3.1.1	Yêu cầu chức năng	29
3.1.2	Yêu cầu về khả năng hoạt động theo thời gian thực	30
3.1.3	Yêu cầu về dữ liệu và kết quả điều khiển	31
3.2	Sơ đồ tổng quát của hệ thống	32
3.3	Thiết kế phần cứng	33
3.3.1	Thiết kế khối cảm biến chuyển động MPU6050	34

3.3.2	Thiết kế khối cảm biến lực FSR402	36
3.3.3	Thiết kế khối xử lý trung tâm STM32	37
3.3.4	Thiết kế khối truyền dữ liệu ESP32	38
3.3.5	Thiết kế giao tiếp giữa STM32 và ESP32	39
3.3.6	Thiết kế khối nguồn	40
3.3.7	Sơ đồ nguyên lý phần cứng hoàn chỉnh	43
3.4	Thiết kế phần mềm hệ thống	45
3.4.1	Thiết kế xử lý tín hiệu cảm biến MPU6050	46
3.4.2	Thiết kế xử lý tín hiệu cảm biến FSR402	49
3.4.3	Thiết kế giao thức truyền dữ liệu UART	51
3.4.4	Thiết kế chương trình ESP32 nhận và truyền dữ liệu bằng UDP .	54
3.4.5	Thiết kế luồng truyền dữ liệu UDP và xử lý nhận dạng	54
3.4.6	Thiết kế dữ liệu đầu vào và chương trình nhận dạng chuyển động cho mô hình AI	56
3.4.7	Thiết kế chương trình điều khiển đối tượng trong Unity	60
4	KẾT QUẢ THỰC NGHIỆM	63
4.1	Môi trường thực nghiệm và các tiêu chí đánh giá	63
4.1.1	Môi trường và thiết bị thử nghiệm	63
4.1.2	Các tiêu chí đánh giá	64
4.2	Kết quả thi công mô hình phần cứng	65
4.3	Kết quả việc truyền, xử lý dữ liệu	67
4.3.1	Kết quả truyền dữ liệu từ STM32 cho đến UNITY	67
4.3.2	Kết quả truyền dữ liệu bằng UDP	68
4.3.3	Đánh giá thời gian xử lý và độ trễ hệ thống	69
4.3.4	Kết quả điều khiển đối tượng trong Unity	70
4.3.5	Đánh giá quá trình huấn luyện và kiểm thử mô hình AI	71
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	81
5.1	Ưu điểm hạn chế và hướng phát triển	81
5.1.1	Ưu điểm	81

5.1.2	Hạn chế	82
5.1.3	Hướng phát triển	84

Danh sách bảng

1.1	Tổng hợp một số nghiên cứu ngoài nước liên quan đến nhận dạng chuyển động tay.	3
1.2	Tổng hợp một số nghiên cứu trong nước liên quan đến nhận dạng cử chỉ tay và điều khiển bằng chuyển động tay.	4
1.3	Tiền độ thực hiện đề tài.	8
1.4	Sản phẩm của đề tài.	8
2.1	So sánh các phương án cảm biến MPU.	12
2.2	So sánh các phương án cảm biến lực.	14
2.3	So sánh các phương án vi điều khiển đọc và xử lý dữ liệu cảm biến. . . .	16
2.4	So sánh các phương pháp nhận dạng chuyển động tay.	22
3.1	Kết nối chân giữa MPU6050 thứ nhất và STM32F1.	35
3.2	Kết nối chân giữa MPU6050 thứ hai và STM32F1.	35
3.3	Kết nối cảm biến lực FSR402 với STM32F1.	37
3.4	Tổng hợp các chân sử dụng trên STM32F103C8T6.	38
3.5	Kết nối chân của ESP32 trong hệ thống.	39
3.6	Kết nối UART giữa STM32F103C8T6 và ESP32.	40
3.7	Yêu cầu điện áp và dòng cấp cho các khối phần cứng.	41
3.8	Bảng kết nối các khối trong mạch nguồn.	43
3.9	Các thông số xử lý tín hiệu MPU6050.	46
3.10	Các thông số xử lý tín hiệu FSR402.	49
3.11	Cấu trúc gói dữ liệu UART từ STM32 sang ESP32.	52
3.12	Các tác vụ chính của chương trình ESP32 khi truyền dữ liệu bằng UDP. .	54
3.13	Luồng truyền dữ liệu UDP trong hệ thống.	55
3.14	Các đặc trưng đầu vào của mô hình AI.	56
3.15	Dữ liệu đầu ra của chương trình nhận dạng chuyển động.	57
3.16	Ảnh xạ kết quả nhận dạng sang hành động trong Unity.	60

4.1	Môi trường và thiết bị sử dụng trong quá trình thực nghiệm.	64
4.2	Các tiêu chí đánh giá hệ thống.	65
4.3	Kết quả kiểm tra các khối phần cứng.	66
4.4	Các giai đoạn truyền và xử lý dữ liệu trong hệ thống.	68
4.5	Kết quả đo thời gian xử lý và truyền dữ liệu trong hệ thống UDP.	69
4.6	Kết quả kiểm tra điều khiển đối tượng trong Unity.	71
4.7	Cấu trúc dữ liệu sử dụng trong quá trình huấn luyện mô hình AI.	73
4.8	Kết quả so sánh độ chính xác của các mô hình AI trên tập train và tập test.	74
4.9	Kết quả kiểm thử thực tế khi tách riêng hướng chuyển động và trạng thái FSR.	79

Danh sách hình vẽ

2.1	Giao diện tổng quan của phần mềm Unity trong thiết kế môi trường mô phỏng	26
2.2	Sơ đồ khối mô tả quá trình ánh xạ tín hiệu điều khiển vào đối tượng trò chơi	28
3.1	Sơ đồ tổng quát của hệ thống	32
3.2	sơ đồ kết nối chân giữa mpu và stm32	36
3.3	Sơ đồ kết nối chân giữa FSR và stm32	37
3.4	sơ đồ kết nối uart giữa stm và esp	40
3.5	Sơ đồ tạo nguồn chính của hệ thống	42
3.6	Sơ đồ hoàn chỉnh của hệ thống	44
3.7	Thiết kế PCB mặt trước	45
3.8	Thiết kế PCB mặt sau	45
3.9	Lưu đồ xử lý tín hiệu MPU	48
3.10	Sơ đồ xử lý tín hiệu FSR	50
3.11	Lưu đồ truyền, nhận dữ liệu uart	53
3.12	Lưu đồ chương trình nhận dạng bằng AI - phần 1	58
3.13	Lưu đồ chương trình nhận dạng bằng AI - phần 2	59
3.14	Cấu hình các tham số điều khiển và ánh xạ tín hiệu trong giao diện Unity Inspector	61
3.15	Lưu đồ thuật toán cơ chế chống dội tín hiệu (Debounce) trong vòng lặp cập nhật trạng thái	62
4.1	Mô hình phần cứng sau khi thi công.	66
4.2	Cấu trúc truyền dữ liệu của hệ thống.	67
4.3	Kết quả điều khiển đối tượng trong Unity.	70
4.4	Dữ liệu huấn luyện mô hình AI dưới dạng file CSV.	73
4.5	Biểu đồ so sánh Accuracy của các mô hình AI trên tập train và tập test. .	74
4.6	Ma trận nhầm lẫn của mô hình Random Forest trên tập test.	75

4.7	Độ quan trọng của các đặc trưng	76
4.8	Kết quả đánh giá chi tiết từng lớp của mô hình Random Forest.	76
4.9	Dữ liệu cảm biến gửi qua UDP trong ESP32.	78
4.10	Chương trình Python dự đoán hướng chuyển động trong quá trình kiểm thử thực tế.	78
4.11	Kết quả điều khiển đối tượng trong Unity khi kiểm thử thực tế.	80

Danh mục các từ viết tắt

Dưới đây là danh mục các từ viết tắt được sử dụng trong luận văn.

Các từ viết tắt	Định nghĩa
FSR	Force Sensitive Resistor (Cảm biến lực)
MPU	Motion Processing Unit (Khối xử lý chuyển động)
UDP	User Datagram Protocol (Giao thức gói dữ liệu người dùng)

Chương 1. GIỚI THIỆU

1.1 Tổng quan về nhận dạng chuyển động tay trong điều khiển tương tác

1.1.1 Tổng quan

Trong bối cảnh chuyển đổi số diễn ra mạnh mẽ, nhu cầu xây dựng các phương thức tương tác giữa con người và máy tính theo hướng trực quan và linh hoạt ngày càng trở nên cấp thiết. Các thiết bị điều khiển truyền thống như bàn phím, chuột hoặc tay cầm vẫn được sử dụng rộng rãi, tuy nhiên chúng còn tồn tại những hạn chế do phải thông qua thiết bị trung gian. Vì vậy, nhiều công nghệ tương tác hiện đại, bao gồm nhận dạng giọng nói, xử lý hình ảnh và nhận dạng chuyển động, đang thu hút sự quan tâm của các nhà nghiên cứu.

Trong số đó, nhận dạng chuyển động của bàn tay được xem là một hướng tiếp cận có nhiều tiềm năng nhờ khả năng ghi nhận trực tiếp các thao tác của người sử dụng. Những hành động như di chuyển, nâng, hạ hoặc thực hiện các cử chỉ có thể được hệ thống phân tích và chuyển đổi thành các lệnh điều khiển tương ứng. Công nghệ này đặc biệt phù hợp với các ứng dụng trong lĩnh vực trò chơi và mô phỏng, góp phần tạo nên trải nghiệm tương tác tự nhiên, trực quan và nâng cao mức độ nhập vai của người dùng.

Bên cạnh đó, sự phát triển nhanh chóng của trí tuệ nhân tạo và học máy đã tạo điều kiện nâng cao hiệu quả của các hệ thống nhận dạng chuyển động. Thông qua quá trình học từ dữ liệu thực tế, mô hình có thể thích nghi với sự khác biệt trong thao tác của từng người dùng, đồng thời cải thiện khả năng nhận diện và tăng độ chính xác khi thực hiện điều khiển.

Xuất phát từ những lợi ích trên, việc nghiên cứu và phát triển hệ thống nhận dạng chuyển động tay để điều khiển đối tượng trong trò chơi là một hướng nghiên cứu có tính thực tiễn cao. Hướng tiếp cận này kết hợp các lĩnh vực như hệ thống nhúng, xử lý dữ liệu và trí tuệ nhân tạo nhằm xây dựng phương thức điều khiển hiện đại, trực quan và gần với hành vi tự nhiên của con người hơn.

1.2 Hiện trạng nghiên cứu và ứng dụng hệ thống nhận dạng chuyển động tay

1.2.1 Ngoài nước

Trên phạm vi quốc tế, nhận dạng chuyển động và cử chỉ tay đã trở thành một trong những chủ đề nghiên cứu quan trọng trong lĩnh vực tương tác người – máy. Công nghệ này được ứng dụng rộng rãi trong điều khiển thiết bị thông minh, hệ thống thực tế ảo (VR), thực tế tăng cường (AR), hỗ trợ phục hồi chức năng, robot và các trò chơi tương tác. Phần lớn các nghiên cứu đều hướng đến việc thu thập dữ liệu chuyển động hoặc tín hiệu sinh học từ người sử dụng, sau đó áp dụng các kỹ thuật xử lý tín hiệu kết hợp với các thuật toán học máy để nhận diện và phân loại cử chỉ. Kết quả nhận dạng được sử dụng để điều khiển thiết bị, hỗ trợ người dùng trong nhiều tác vụ khác nhau cũng như tạo ra phương thức tương tác trực tiếp với môi trường số.

Song song với sự phát triển của các thiết bị thu thập dữ liệu, nhiều nghiên cứu tập trung vào việc xây dựng và tối ưu các mô hình nhận dạng nhằm nâng cao hiệu quả phân loại cử chỉ. Các thuật toán học máy truyền thống như Support Vector Machine (SVM), Random Forest (RF), K-Nearest Neighbors (KNN) và Artificial Neural Network (ANN) được sử dụng phổ biến nhờ khả năng xử lý hiệu quả các đặc trưng được trích xuất từ dữ liệu đầu vào. Trong những năm gần đây, sự phát triển của học sâu đã thúc đẩy việc ứng dụng các mô hình như Convolutional Neural Network (CNN) để nhận dạng các chuyển động có tính phức tạp cao, góp phần nâng cao độ chính xác, tăng khả năng tổng quát hóa và cải thiện khả năng thích nghi của hệ thống đối với nhiều điều kiện sử dụng khác nhau.

Không chỉ dừng lại ở các nghiên cứu mang tính lý thuyết, nhiều công trình đã triển khai thành công các hệ thống nhận dạng cử chỉ tay trong điều khiển trò chơi, môi trường thực tế ảo và các ứng dụng thực tế tăng cường. Những kết quả này cho thấy tiềm năng lớn của công nghệ trong việc xây dựng các hệ thống tương tác thời gian thực, mang lại trải nghiệm trực quan và tự nhiên hơn cho người dùng. Tuy nhiên, các hệ thống hiện có vẫn còn đối mặt với nhiều thách thức, bao gồm sự khác biệt về đặc điểm chuyển động giữa các cá nhân, ảnh hưởng từ vị trí lắp đặt cảm biến, sự tương đồng giữa một số cử chỉ và yêu cầu hiệu chuẩn trước khi sử dụng. Do đó, việc nghiên cứu các phương pháp nhận

dạng có độ chính xác cao, khả năng thích nghi tốt và hoạt động ổn định trong nhiều điều kiện khác nhau vẫn là một hướng nghiên cứu có ý nghĩa thực tiễn và tiếp tục được quan tâm.

Bảng dưới đây tổng hợp một số công trình nghiên cứu tiêu biểu trên thế giới về nhận dạng cử chỉ tay, bao gồm phương pháp thu thập dữ liệu, mô hình nhận dạng được áp dụng và những kết quả nổi bật mà các nghiên cứu đã đạt được.

Bảng 1.1: Tổng hợp một số nghiên cứu ngoài nước liên quan đến nhận dạng chuyển động tay.

Tài liệu	Phương pháp	Mô hình sử dụng	Kết quả
Enhanced Hand Gesture Recognition with Surface Electromyogram and Machine Learning. Link truy cập	Thu thập tín hiệu cơ tay, xử lý dữ liệu, trích xuất đặc trưng và phân loại 7 cử chỉ tay.	Random Forest, SVM, Logistic Regression, KNN.	Random Forest cho kết quả tốt nhất; accuracy, precision, recall và F1-score trung bình đều trên 95%, ROC-AUC trên 99%.
Quaternion-Based Gesture Recognition Using Wireless Wearable Motion Capture Sensors. Link truy cập	Sử dụng cảm biến chuyển động không dây, biểu diễn chuyển động bằng quaternion và phân loại 6 cử chỉ.	Support Vector Machine và Artificial Neural Network.	Độ chính xác sau xử lý có thể đạt xấp xỉ 99% hoặc cao hơn; kết quả có thể giảm khi thay đổi người dùng.
Machine Learning-Based Gesture Recognition Glove: Design and Implementation. Link truy cập	Thiết kế găng tay thông minh giá thấp để thu nhận cử chỉ động phục vụ điều khiển trò chơi.	Convolutional Neural Network.	Mô hình CNN đạt khoảng 90% độ chính xác trong phân loại cử chỉ động; có tiềm năng ứng dụng trong game, VR và AR.

1.2.2 Trong nước

Tại Việt Nam, lĩnh vực nghiên cứu nhận dạng cử chỉ tay và điều khiển thiết bị thông qua chuyển động của bàn tay đang ngày càng nhận được sự quan tâm của nhiều nhóm nghiên cứu. Các công trình được triển khai theo nhiều hướng tiếp cận khác nhau, trong đó nổi bật là phương pháp ứng dụng xử lý ảnh và thị giác máy tính để nhận diện hình dạng bàn tay, số lượng ngón tay hoặc các tư thế đặc trưng thông qua dữ liệu thu nhận từ camera. Bên cạnh đó, một số nghiên cứu lựa chọn sử dụng các cảm biến đeo trên tay nhằm ghi nhận trực tiếp dữ liệu chuyển động, sau đó xử lý và truyền tín hiệu đến bộ điều khiển để vận hành robot hoặc các hệ thống tương tác thông minh.

Ngoài các phương pháp dựa trên hình ảnh, găng tay thông minh tích hợp cảm biến cũng là một hướng nghiên cứu được quan tâm trong những năm gần đây. Hệ thống này sử dụng các cảm biến được bố trí trên găng tay để thu thập thông tin về chuyển động của các ngón tay và bàn tay, từ đó chuyển đổi thành tín hiệu điều khiển cho các thiết bị hoặc ứng dụng tương ứng. So với giải pháp sử dụng camera, phương pháp này có ưu điểm là thu nhận dữ liệu trực tiếp nên ít chịu tác động bởi điều kiện ánh sáng, góc quan sát hay các yếu tố môi trường bên ngoài, góp phần nâng cao độ ổn định của quá trình nhận dạng. Bảng 1.2: Tổng hợp một số nghiên cứu trong nước liên quan đến nhận dạng cử chỉ tay và điều khiển bằng chuyển động tay.

Tài liệu	Phương pháp	Mô hình sử dụng	Kết quả
Nhận dạng cử chỉ của bàn tay người theo thời gian thực. Link truy cập	Sử dụng cảm biến Kinect thu nhận thông tin ảnh chiều sâu và khung xương, phát hiện vùng bàn tay, trích xuất đặc trưng và nhận dạng cử chỉ theo thời gian thực.	HLAC kết hợp SVM đa lớp.	Hệ thống nhận dạng cử chỉ một tay với độ chính xác trên 93%; thời gian nhận dạng trung bình khoảng 85 ms cho mỗi cử chỉ, đáp ứng yêu cầu xử lý thời gian thực.
Nhận dạng cử chỉ bàn tay dùng mạng nơ-ron chập. Link truy cập	Thu thập ảnh bàn tay từ camera điện thoại, xây dựng tập dữ liệu sau đó huấn luyện và kiểm tra mô hình nhận dạng.	Mạng nơ-ron chập CNN.	Hệ thống nhận dạng 6 cử chỉ bàn tay với độ chính xác đạt 98,6% trong điều kiện ảnh chụp chính diện, độ sáng và khoảng cách ngón tay phù hợp.
Thiết kế và chế tạo bàn tay robot được điều khiển bằng găng tay. Link truy cập	Sử dụng găng tay có cảm biến flex để thu nhận chuyển động các ngón tay, truyền dữ liệu không dây đến mạch điều khiển và điều khiển các động cơ servo trên bàn tay robot.	Điều khiển theo giá trị cảm biến flex và ánh xạ sang chuyển động servo.	Bàn tay robot có thể mô phỏng chuyển động các ngón tay theo găng tay điều khiển; hệ thống có độ trễ khoảng 0,5 s và phạm vi hoạt động có thể đạt khoảng 10 m khi ít vật cản.

Phần lớn nghiên cứu còn tập trung riêng vào xử lý ảnh, điều khiển robot hoặc nhận dạng cử chỉ trong điều kiện thử nghiệm nhất định. Vì vậy, việc xây dựng một hệ thống kết hợp phần cứng nhúng, truyền dữ liệu thời gian thực, mô hình trí tuệ nhân tạo và ứng dụng điều khiển đối tượng trò chơi là hướng phát triển phù hợp, có tính thực tiễn và gần với xu hướng tương tác người – máy hiện nay.

1.3 Mục tiêu đề tài

Mục tiêu của đề tài là nghiên cứu và xây dựng hệ thống nhúng có khả năng nhận dạng chuyển động tay theo thời gian thực để điều khiển đối tượng trong trò chơi. Hệ thống tích hợp cảm biến, vi điều khiển, truyền dữ liệu, mô hình trí tuệ nhân tạo và môi trường trò chơi nhằm tạo ra phương thức điều khiển trực quan và tự nhiên hơn.

Để thực hiện mục tiêu này, đề tài tập trung vào các nội dung sau:

- Thiết kế phần cứng thu thập dữ liệu chuyển động tay bằng cảm biến.
- Xây dựng chương trình xử lý, lọc nhiễu và trích xuất đặc trưng dữ liệu trên vi điều khiển.
- Xây dựng bộ dữ liệu phục vụ huấn luyện mô hình nhận dạng.
- Huấn luyện và đánh giá mô hình AI bằng các chỉ số Accuracy, Precision, Recall, F1-score và ma trận nhầm lẫn.
- Xây dựng môi trường trò chơi và ánh xạ các cử chỉ thành lệnh điều khiển.
- Đánh giá toàn bộ hệ thống về độ chính xác, độ ổn định, độ trễ và khả năng hoạt động thời gian thực.

1.4 Đối tượng và phạm vi nghiên cứu

1.4.1 Đối tượng nghiên cứu

Đề tài tập trung nghiên cứu và xây dựng hệ thống nhận dạng chuyển động tay theo thời gian thực nhằm điều khiển đối tượng trong môi trường trò chơi. Đối tượng nghiên cứu chính bao gồm quá trình thu nhận dữ liệu chuyển động tay của người dùng, xử lý dữ liệu cảm biến, trích xuất bộ dữ liệu huấn luyện để huấn luyện mô hình trí tuệ nhân tạo và ứng dụng kết quả nhận dạng để điều khiển đối tượng trong trò chơi.

1.4.2 Phạm vi nghiên cứu

Đề tài được thực hiện trong phạm vi nghiên cứu và xây dựng mô hình thử nghiệm, chưa hướng đến phát triển thành sản phẩm thương mại. Hệ thống tập trung nhận dạng một số cử chỉ tay đã được định nghĩa trước để điều khiển đối tượng trong trò chơi, như di chuyển trái, phải, lên, xuống và thực hiện một số lệnh đặc biệt.

Về phần cứng, hệ thống sử dụng cảm biến kết hợp với vi điều khiển để thu thập và xử lý dữ liệu chuyển động tay. Về phần mềm, đề tài phát triển chương trình thu thập, truyền dữ liệu, huấn luyện mô hình trí tuệ nhân tạo và kết nối với trò chơi để thực hiện điều khiển. Mô hình AI được xây dựng từ dữ liệu thu thập trong quá trình thực nghiệm.

Phạm vi ứng dụng của đề tài giới hạn ở trò chơi thử nghiệm và chưa mở rộng sang các lĩnh vực như robot, y tế hay thực tế ảo. Tuy nhiên, kết quả nghiên cứu có thể làm cơ sở cho việc phát triển các hệ thống tương tác người – máy trong tương lai.

1.5 Cách tiếp cận và phương pháp nghiên cứu

Đề tài tiếp cận bài toán nhận dạng chuyển động tay bằng cách kết hợp hệ thống nhúng, truyền dữ liệu UDP và mô hình trí tuệ nhân tạo. Dữ liệu từ cảm biến được thu thập và xử lý trên vi điều khiển, sau đó truyền đến mô hình AI để nhận dạng cử chỉ. Kết quả nhận dạng được chuyển thành các lệnh điều khiển đối tượng trong trò chơi theo thời gian thực.

Quy trình thực hiện bao gồm thu nhận dữ liệu, lọc nhiễu, hiệu chỉnh, trích xuất đặc trưng và đưa vào mô hình AI để phân loại các chuyển động. Các cử chỉ được ánh xạ thành những thao tác như di chuyển lên, xuống, trái, phải hoặc thực hiện lệnh đặc biệt.

Đề tài sử dụng các phương pháp nghiên cứu chính sau:

- Nghiên cứu cơ sở lý thuyết về hệ thống nhúng, cảm biến, giao thức UDP và trí tuệ nhân tạo.
- Thiết kế phần cứng và xây dựng hệ thống thu thập dữ liệu chuyển động tay.
- Thu thập, xử lý và gán nhãn dữ liệu phục vụ huấn luyện mô hình.
- Huấn luyện và đánh giá mô hình AI bằng Accuracy, Precision, Recall, F1-score và ma trận nhầm lẫn.
- Tích hợp hệ thống với môi trường trò chơi thông qua truyền dữ liệu UDP.
- Thử nghiệm để đánh giá độ chính xác, độ ổn định, độ trễ và khả năng đáp ứng thời gian thực.

1.6 Nội dung nghiên cứu và tiến độ thực hiện

Nội dung nghiên cứu của đề tài hướng đến việc xây dựng một hệ thống nhúng có khả năng nhận dạng chuyển động tay theo thời gian thực và sử dụng kết quả nhận dạng để

điều khiển đối tượng trong môi trường trò chơi. Hệ thống được thiết kế gồm các thành phần chính như khối thu thập dữ liệu từ cảm biến, khối xử lý dữ liệu trên vi điều khiển, khối truyền dữ liệu bằng giao thức UDP, mô hình trí tuệ nhân tạo dùng để nhận dạng cử chỉ và khối điều khiển đối tượng trong trò chơi.

Các nội dung nghiên cứu chính của đề tài bao gồm:

- **Nghiên cứu cơ sở lý thuyết và đề xuất kiến trúc hệ thống:** Khảo sát các phương pháp nhận dạng chuyển động tay, các mô hình tương tác giữa người và máy, cũng như các kỹ thuật điều khiển trò chơi bằng cử chỉ. Trên cơ sở đó, lựa chọn và đề xuất kiến trúc hệ thống phù hợp với mục tiêu của đề tài.
- **Thiết kế hệ thống phần cứng:** Lựa chọn các loại cảm biến thích hợp, kết hợp với vi điều khiển để xây dựng thiết bị thu thập dữ liệu chuyển động tay. Hệ thống phần cứng được thiết kế nhằm bảo đảm hoạt động ổn định và đáp ứng yêu cầu của quá trình thực nghiệm.
- **Phát triển chương trình xử lý dữ liệu:** Xây dựng chương trình trên vi điều khiển để đọc dữ liệu từ cảm biến, thực hiện lọc nhiễu, hiệu chỉnh giá trị và trích xuất các đặc trưng cần thiết trước khi đưa vào mô hình nhận dạng.
- **Xây dựng cơ chế truyền dữ liệu:** Thiết kế chương trình truyền dữ liệu từ hệ thống nhúng đến máy tính thông qua giao thức UDP, đáp ứng yêu cầu truyền dữ liệu nhanh và ổn định trong các ứng dụng thời gian thực.
- **Thu thập và xây dựng bộ dữ liệu:** Tiến hành thu thập dữ liệu từ các chuyển động tay đã được định nghĩa, thực hiện gán nhãn và xây dựng bộ dữ liệu phục vụ cho quá trình huấn luyện mô hình trí tuệ nhân tạo.
- **Huấn luyện và đánh giá mô hình AI:** Xây dựng các mô hình nhận dạng chuyển động tay, đồng thời đánh giá hiệu quả bằng các chỉ số Accuracy, Precision, Recall, F1-score và ma trận nhầm lẫn để lựa chọn mô hình có hiệu suất tốt nhất.
- **Xây dựng môi trường trò chơi:** Thiết kế trò chơi thử nghiệm và thiết lập cơ chế ánh xạ giữa các cử chỉ tay với các thao tác điều khiển như di chuyển lên, xuống, sang trái, sang phải hoặc thực hiện các hành động đặc biệt.
- **Thử nghiệm và đánh giá:** Đánh giá toàn diện hệ thống thông qua các tiêu chí về độ chính xác, tính ổn định và khả năng đáp ứng thời gian thực.

Để đảm bảo hoàn thành đồ án đúng tiến độ, nội dung công việc được chia thành các giai đoạn cụ thể như sau:

Bảng 1.3: Tiến độ thực hiện đề tài.

STT	Nội dung công việc	Sản phẩm	Thời gian	Người thực hiện
1	Nghiên cứu tổng quan đề tài, khảo sát các công trình liên quan và đề xuất phương án hệ thống.	Cơ sở lý thuyết, sơ đồ khối tổng thể, phương án thiết kế hệ thống.	1 tháng	Nhóm thực hiện
2	Thiết kế và xây dựng phần cứng thu nhận dữ liệu chuyển động tay.	Mô hình phần cứng, mạch kết nối cảm biến, khối xử lý và khối truyền dữ liệu.	1 tháng	Nhóm thực hiện
3	Lập trình đọc cảm biến, xử lý dữ liệu, truyền dữ liệu lên nền tảng IoT và xây dựng bộ dữ liệu huấn luyện.	Chương trình nhúng, dữ liệu cảm biến, file dữ liệu huấn luyện.	1 tháng	Nhóm thực hiện
4	Xây dựng, huấn luyện và đánh giá mô hình AI nhận dạng chuyển động tay.	Mô hình AI, báo cáo đánh giá, ma trận nhầm lẫn, kết quả so sánh mô hình.	1 tháng	Nhóm thực hiện
5	Xây dựng môi trường trò chơi, tích hợp kết quả nhận dạng và điều khiển đối tượng trong trò chơi.	Chương trình trò chơi, cơ chế điều khiển đối tượng, kết quả tích hợp hệ thống.	1 tháng	Nhóm thực hiện
6	Thực nghiệm toàn hệ thống, đánh giá kết quả, hoàn thiện báo cáo và sản phẩm đồ án.	Mô hình hoàn chỉnh, kết quả thực nghiệm, báo cáo tổng kết, slide thuyết trình.	1 tháng	Nhóm thực hiện

1.7 Sản phẩm

Bảng 1.4: Sản phẩm của đề tài.

Tên sản phẩm	Đạt được
Báo cáo tổng kết đề tài	X
Bài báo khoa học	
Video giới thiệu đề tài	
Mô hình đồ án	X

Chương 2. CƠ SỞ LÝ THUYẾT VÀ LỰA CHỌN PHƯƠNG ÁN

2.1 Tổng quan về hệ thống nhận dạng chuyển động tay điều khiển đối tượng trò chơi

Hệ thống nhận dạng chuyển động tay là một hệ thống tương tác giữa người và máy. Tức là người dùng sẽ điều khiển, tương tác với các đối tượng trò chơi bằng các thao tác tự nhiên thay vì nhập liệu từ các thiết bị truyền thống như bàn phím, chuột hoặc tay cầm. Dữ liệu chuyển động sẽ được cảm biến thu thập và xử lý thông qua mô hình AI và cuối cùng chuyển đổi thành các lệnh tương ứng.

Đây là một hệ thống yêu cầu tốc độ phản hồi nhanh chóng và ổn định vì phải đáp ứng trải nghiệm điều khiển thời gian thực. Vậy nên các quá trình từ thu thập, xử lý và nhận dạng tín hiệu cần được tối ưu nhằm giảm độ trễ và tăng độ chính xác.

Nhìn chung, đây là một hệ thống kết hợp giữa phần cứng nhúng, cảm biến, xử lý dữ liệu và thuật toán trí tuệ nhân tạo nhằm tạo ra phương pháp điều khiển bằng chuyển động tay tự nhiên và hiệu quả.

2.2 Lựa chọn cảm biến MPU

Trong hệ thống nhận dạng chuyển động tay thời gian thực, cảm biến MPU có chức năng ghi nhận mọi cử động của tay. Thiết bị này không chỉ đo lường gia tốc và vận tốc góc mà còn xác định chính xác tư thế của tay trong không gian, giúp hệ thống hiểu được các thao tác nghiêng, xoay hay thay đổi hướng di chuyển một cách linh hoạt.

Thông thường, cảm biến MPU là sự kết hợp giữa gia tốc kế và con quay hồi chuyển, một số dòng cao cấp hơn còn có thêm từ kế hoặc bộ xử lý tích hợp. Những dữ liệu quan trọng mà cảm biến này cung cấp bao gồm:

- **Gia tốc theo trục X, Y, Z** ($AccX, AccY, AccZ$): biểu diễn gia tốc tuyến tính của cảm biến theo ba trục không gian.
- **Vận tốc góc theo trục X, Y, Z** ($GyroX, GyroY, GyroZ$): biểu diễn tốc độ quay của

cảm biến, hỗ trợ nhận biết các chuyển động như xoay, nghiêng hoặc thay đổi tư thế.

- **Từ trường theo trục X, Y, Z** ($MagX, MagY, MagZ$): hỗ trợ xác định hướng tuyệt đối nhưng dễ bị ảnh hưởng bởi nhiễu từ môi trường.
- **Nhiệt độ cảm biến**: dùng để theo dõi trạng thái hoạt động và hiệu chỉnh sai số.
- **Góc Roll, Pitch, Yaw**: biểu diễn tư thế của cảm biến trong không gian, lần lượt là góc nghiêng, góc ngả và góc xoay ngang.

2.2.1 Cảm biến MPU6050

MPU6050 là cảm biến quán tính 6 trục gồm gia tốc kế 3 trục và con quay hồi chuyển 3 trục, cho phép đo gia tốc và vận tốc góc theo các trục X, Y, Z.

Trong đề tài, MPU6050 được gắn trên tay để thu thập dữ liệu chuyển động phục vụ nhận dạng trạng thái điều khiển. Các dữ liệu thu được gồm:

- Gia tốc: $AccX, AccY, AccZ$.
- Vận tốc góc: $GyroX, GyroY, GyroZ$.
- Nhiệt độ cảm biến.
- Góc nghiêng Roll và Pitch.

MPU6050 giao tiếp qua chuẩn I2C, có kích thước nhỏ gọn, chi phí thấp và tương thích với nhiều nền tảng như Arduino, ESP32 và STM32. Khi kết hợp với các thuật toán lọc như Kalman hoặc bộ lọc bổ sung, dữ liệu cảm biến có thể ổn định hơn, đáp ứng tốt các chuyển động cơ bản như nghiêng, xoay và thay đổi tư thế.

2.2.2 Cảm biến MPU9250

MPU9250 là cảm biến quán tính 9 trục gồm gia tốc kế, con quay hồi chuyển và từ kế 3 trục. So với MPU6050, cảm biến này bổ sung khả năng đo từ trường giúp xác định hướng trong không gian chính xác hơn.

Các dữ liệu có thể thu thập gồm:

- Gia tốc: $AccX, AccY, AccZ$.
- Vận tốc góc: $GyroX, GyroY, GyroZ$.
- Từ trường: $MagX, MagY, MagZ$.
- Nhiệt độ cảm biến.

- Góc Roll, Pitch, Yaw.

Ưu điểm của MPU9250 là khả năng cung cấp nhiều thông tin tư thế hơn và hỗ trợ giao tiếp I2C hoặc SPI. Tuy nhiên, cảm biến có giá thành cao hơn, yêu cầu xử lý phức tạp hơn do ảnh hưởng của nhiễu từ trường. Đối với các chuyển động cơ bản trong điều khiển trò chơi, việc sử dụng từ kế thường không cần thiết.

2.2.3 Cảm biến Wit Sensor

Wit Sensor là dòng cảm biến quán tính tích hợp bộ xử lý tín hiệu bên trong, có khả năng xuất trực tiếp các giá trị như gia tốc, vận tốc góc và góc tư thế. Điều này giúp giảm tải xử lý cho vi điều khiển và đơn giản hóa quá trình lập trình.

Các dữ liệu thu được gồm:

- Gia tốc: *AccX*, *AccY*, *AccZ*.
- Vận tốc góc: *GyroX*, *GyroY*, *GyroZ*.
- Từ trường: *MagX*, *MagY*, *MagZ*.
- Góc Roll, Pitch, Yaw.
- Nhiệt độ cảm biến.

Wit Sensor có ưu điểm là dữ liệu ổn định, hỗ trợ nhiều chuẩn giao tiếp như UART, I2C và RS485. Nhờ thuật toán xử lý tích hợp, cảm biến phù hợp cho các ứng dụng đo góc và theo dõi tư thế.

Tuy nhiên, cảm biến có chi phí cao hơn MPU6050 và hạn chế khả năng tùy chỉnh thuật toán xử lý do dữ liệu đã được xử lý sẵn bên trong.

Bảng 2.1: So sánh các phương án cảm biến MPU.

Cảm biến	Ưu điểm	Nhược điểm	Đánh giá
MPU6050	Giá thành thấp, dễ tìm mua, kích thước nhỏ gọn, dễ gắn lên tay. Giao tiếp I2C đơn giản, có nhiều tài liệu và thư viện hỗ trợ.	Dữ liệu con quay hồi chuyển có thể bị trôi theo thời gian nếu không được hiệu chỉnh và lọc tín hiệu.	Phù hợp với đề tài, được lựa chọn.
MPU9250	Có đầy đủ 9 trục gồm gia tốc kế, con quay hồi chuyển và từ kế.	Tín hiệu từ kế dễ bị ảnh hưởng bởi nhiễu từ môi trường xung quanh.	Tốt nhưng dư tính năng so với yêu cầu đề tài.
Wit Sensor	Có tích hợp sẵn bộ xử lý dữ liệu bên trong, module có thể xuất trực tiếp gia tốc, vận tốc góc và góc nghiêng.	Giá thành cao hơn so với các module MPU thông dụng. Khả năng can thiệp sâu vào thuật toán xử lý bên trong bị hạn chế.	Ổn định nhưng chi phí cao, chưa tối ưu cho mô hình đồ án.

Sau quá trình so sánh, MPU6050 được lựa chọn làm cảm biến chính cho hệ thống nhận dạng chuyển động tay nhờ khả năng đo gia tốc và vận tốc góc đáp ứng yêu cầu nhận dạng chuyển động cơ bản theo thời gian thực. Cảm biến có ưu điểm về chi phí thấp, kích thước nhỏ gọn, dễ kết nối với vi điều khiển và có nhiều tài liệu hỗ trợ trong quá trình phát triển.

Trong hệ thống, hai cảm biến MPU6050 được gắn tại cánh tay và cẳng tay để thu thập dữ liệu chuyển động. Dựa trên dữ liệu từ hai cảm biến, hệ thống có thể xác định sự thay đổi tư thế, góc nghiêng và góc xoay giữa các phần của tay. Các dữ liệu này được kết hợp với tín hiệu từ FSR để tạo thành dữ liệu đầu vào cho mô hình nhận dạng, từ đó thực hiện điều khiển đối tượng trong trò chơi theo thời gian thực.

2.3 Lựa chọn cảm biến lực FSR

Cảm biến FSR hoạt động dựa trên nguyên lý thay đổi điện trở theo lực tác động. Khi chưa có lực nhấn, điện trở của cảm biến thường rất lớn. Khi người dùng tác động lực lên vùng nhạy của cảm biến, điện trở giảm xuống, làm điện áp đầu ra trong mạch phân áp thay đổi. Vi điều khiển có thể đọc sự thay đổi điện áp này thông qua chân ADC và chuyển đổi thành giá trị số để xử lý.

Các giá trị thường khai thác từ cảm biến lực gồm:

- **Giá trị ADC thô (FSR_{raw}):** là giá trị đọc trực tiếp từ chân ADC của vi điều khiển.

Với ADC 12 bit, giá trị thường nằm trong khoảng từ 0 đến 4095. Giá trị này thay đổi tùy theo lực nhấn lên cảm biến.

- **Giá trị FSR sau lọc** (*FSR_filtered*): là giá trị sau khi được xử lý bằng bộ lọc như lọc trung bình hoặc lọc IIR. Thông số này giúp giảm nhiễu và làm tín hiệu ổn định hơn.

Đối với đề tài này, cảm biến lực cần có kích thước nhỏ gọn, dễ bố trí trên ngón tay hoặc lòng bàn tay, dễ đọc tín hiệu bằng ADC, giá thành phù hợp và có khả năng phản ánh rõ sự thay đổi giữa trạng thái không nhấn và có nhấn. Dựa trên các tiêu chí đó, nhóm tiến hành khảo sát ba phương án gồm FSR402, FSR408 và SingleTact Force Sensor.

2.3.1 Cảm biến FSR402

FSR402 là cảm biến lực dạng điện trở, dùng để phát hiện lực tác động thông qua sự thay đổi điện trở khi bị nhấn. Cảm biến có kích thước nhỏ, giá thành thấp, dễ tích hợp với vi điều khiển thông qua mạch phân áp và đọc bằng ADC.

Trong đề tài, FSR402 được sử dụng để nhận biết thao tác nhấn hoặc bóp tay, tạo tín hiệu điều khiển bổ sung cho hệ thống trò chơi. Tuy nhiên, cảm biến cần được hiệu chuẩn và chủ yếu dùng để phát hiện sự thay đổi lực thay vì đo lực chính xác.

2.3.2 Cảm biến FSR408

FSR408 là cảm biến lực dạng dải dài, hoạt động dựa trên sự thay đổi điện trở theo lực tác động và có thể đọc tín hiệu analog thông qua ADC của vi điều khiển. Cảm biến có ưu điểm là vùng nhạy lực lớn, phù hợp với các ứng dụng cần phát hiện lực trên diện rộng.

Tuy nhiên, do kích thước dài và khó bố trí trên tay người dùng, FSR408 không phù hợp với hệ thống nhận dạng chuyển động tay yêu cầu tính nhỏ gọn và linh hoạt. Vì vậy, cảm biến này không được lựa chọn cho mô hình điều khiển trò chơi.

2.3.3 Cảm biến SingleTact Force Sensor

SingleTact Force Sensor là cảm biến lực có độ ổn định và độ chính xác cao hơn so với FSR thông thường, hỗ trợ kết nối analog hoặc I2C với các hệ thống nhúng. Cảm biến có thiết kế mỏng, độ nhạy tốt, phù hợp với các ứng dụng yêu cầu đo lực chính xác như

robot, y sinh và nghiên cứu.

Tuy nhiên, giá thành cao và yêu cầu mạch giao tiếp phù hợp khiến SingleTact chưa phù hợp với phạm vi đồ án thử nghiệm có giới hạn về chi phí.

2.3.4 Tổng hợp và lựa chọn cảm biến lực

Bảng 2.2: So sánh các phương án cảm biến lực.

Cảm biến	Ưu điểm	Nhược điểm	Đánh giá
FSR402	Giá thành thấp, kích thước nhỏ gọn, dễ tích hợp với vi điều khiển thông qua ADC.	Độ chính xác chưa cao và tín hiệu cần được lọc để tăng độ ổn định.	Phù hợp với đề tài và được lựa chọn.
FSR408	Vùng cảm nhận lực lớn, dễ tạo tín hiệu analog để đo lực.	Kích thước dài, khó bố trí trên tay và làm giảm tính linh hoạt khi sử dụng.	Có ưu điểm về vùng đo nhưng không phù hợp về kích thước.
SingleTact Force Sensor	Độ nhạy cao, tín hiệu ổn định và khả năng đo lực chính xác.	Giá thành cao và yêu cầu phần cứng giao tiếp phù hợp.	Hiệu năng tốt nhưng chi phí chưa phù hợp với đề tài.

Sau khi phân tích và so sánh, nhóm lựa chọn cảm biến FSR402 cho hệ thống nhận dạng chuyển động tay. Lý do lựa chọn là FSR402 có kích thước nhỏ gọn, dễ bố trí trên tay người dùng, giá thành phù hợp và có thể đọc tín hiệu đơn giản bằng ADC thông qua mạch phân áp.

2.4 Lựa chọn vi điều khiển đọc và xử lý dữ liệu cảm biến

Trong hệ thống nhận dạng chuyển động tay, vi điều khiển có nhiệm vụ thu thập dữ liệu từ cảm biến MPU và FSR, xử lý tín hiệu ban đầu và truyền dữ liệu đến các khối xử lý tiếp theo. MPU sử dụng giao tiếp I2C, trong khi FSR cung cấp tín hiệu analog cần đọc qua ADC.

Vi điều khiển được lựa chọn cần đáp ứng các yêu cầu như hỗ trợ I2C, ADC, UART, tốc độ xử lý đủ nhanh và khả năng hoạt động ổn định khi truyền dữ liệu liên tục. Dựa trên các yêu cầu này, nhóm tiến hành khảo sát ba phương án gồm STM32, MSP430 và Arduino Uno để lựa chọn nền tảng phù hợp cho hệ thống. .

2.4.1 STM32

STM32F103C8T6 là vi điều khiển 32 bit sử dụng lõi ARM Cortex, phù hợp với các hệ thống nhúng yêu cầu tốc độ xử lý và nhiều ngoại vi. Trong đề tài, STM32F103C8T6 được xem xét nhờ giá thành phù hợp, dễ tìm mua và đáp ứng yêu cầu kết nối cảm biến.

Vi điều khiển này có nhiều chân GPIO, kênh ADC và hỗ trợ các giao tiếp như I2C, UART, SPI, cho phép đọc tín hiệu analog từ FSR và giao tiếp với cảm biến MPU. Nhờ khả năng xử lý tốt, STM32 phù hợp với hệ thống cần thu thập và xử lý dữ liệu cảm biến liên tục.

Đối với hệ thống của đề tài, STM32 có thể thực hiện các nhiệm vụ chính sau:

- Đọc tín hiệu analog từ cảm biến FSR thông qua ADC.
- Giao tiếp với cảm biến MPU6050 thông qua chuẩn I2C.
- Xử lý sơ bộ dữ liệu cảm biến như lọc nhiễu, tính góc nghiêng, tính đặc trưng và chuẩn hóa dữ liệu.
- Truyền dữ liệu đã xử lý sang ESP32 hoặc máy tính thông qua UART.

STM32 có ưu điểm về tốc độ xử lý, số lượng ngoại vi đa dạng và khả năng đọc nhiều cảm biến cùng lúc. Công cụ STM32CubeIDE hỗ trợ cấu hình các giao tiếp như ADC, I2C, UART và Timer thuận tiện hơn. Với khả năng xử lý dữ liệu theo thời gian thực, STM32 phù hợp cho việc thu thập và truyền dữ liệu cảm biến liên tục trong hệ thống.

2.4.2 MSP430

MSP430 là vi điều khiển 16 bit của Texas Instruments, nổi bật với khả năng tiêu thụ điện năng thấp và phù hợp cho các thiết bị sử dụng pin. Vi điều khiển này hỗ trợ các giao tiếp phổ biến như UART, SPI, I2C và tích hợp ADC để kết nối với cảm biến.

Tuy nhiên, so với STM32, MSP430 có khả năng xử lý và tài nguyên bộ nhớ thấp hơn, gây hạn chế khi cần xử lý nhiều dữ liệu cảm biến, lọc tín hiệu và truyền dữ liệu liên tục. Vì vậy, MSP430 chưa phù hợp với hệ thống nhận dạng chuyển động tay yêu cầu xử lý thời gian thực.

2.4.3 Arduino Uno

Arduino Uno sử dụng vi điều khiển ATmega328P, là nền tảng phổ biến với ưu điểm dễ lập trình, nhiều thư viện hỗ trợ và phù hợp cho việc thử nghiệm hệ thống nhúng. Board hỗ trợ đọc tín hiệu analog từ FSR, giao tiếp I2C với MPU và UART để truyền dữ liệu.

Tuy nhiên, Arduino Uno có hạn chế về tốc độ xử lý, bộ nhớ và khả năng mở rộng khi cần xử lý nhiều dữ liệu cảm biến cùng lúc. Vì vậy, Arduino Uno phù hợp cho thử nghiệm cơ bản nhưng chưa tối ưu cho hệ thống nhận dạng chuyển động tay theo thời gian thực.

2.4.4 Tổng hợp và lựa chọn vi điều khiển

Bảng 2.3: So sánh các phương án vi điều khiển đọc và xử lý dữ liệu cảm biến.

Vi điều khiển	Ưu điểm	Nhược điểm	Đánh giá
STM32	Tốc độ xử lý cao, nhiều kênh ADC và hỗ trợ I2C, UART. Phù hợp để đọc nhiều cảm biến và xử lý dữ liệu thời gian thực.	Không tích hợp Wi-Fi và cần kết hợp thêm module truyền thông. Lập trình phức tạp hơn Arduino Uno.	Phù hợp với đề tài, được lựa chọn.
MSP430	Tiêu thụ điện năng thấp, hỗ trợ ADC và các giao tiếp cơ bản. Phù hợp với các hệ thống tiết kiệm năng lượng.	Hiệu năng và bộ nhớ hạn chế, không tích hợp Wi-Fi. Khả năng xử lý nhiều cảm biến chưa cao.	Tiết kiệm năng lượng nhưng chưa phù hợp với đề tài.
Arduino Uno	Dễ lập trình, nhiều thư viện hỗ trợ và thuận tiện cho việc thử nghiệm cảm biến.	Tốc độ xử lý và bộ nhớ hạn chế, không tích hợp Wi-Fi. Chưa phù hợp với hệ thống thời gian thực.	Dễ sử dụng nhưng chưa đáp ứng tốt yêu cầu đề tài.

Sau khi phân tích và so sánh, nhóm lựa chọn STM32 làm vi điều khiển chính để đọc và xử lý dữ liệu cảm biến. Lý do lựa chọn là STM32 có tốc độ xử lý tốt, hỗ trợ nhiều kênh ADC để đọc tín hiệu analog FSR, đồng thời có giao tiếp I2C để đọc cảm biến MPU và UART để truyền dữ liệu sang khối xử lý khác.

Trong hệ thống của đề tài, STM32 thực hiện nhiệm vụ thu thập dữ liệu từ các cảm biến, lọc tín hiệu, tính toán các đặc trưng cần thiết và gửi dữ liệu đã xử lý sang ESP32 hoặc máy tính. Việc lựa chọn STM32 giúp hệ thống đáp ứng tốt hơn yêu cầu đọc nhiều cảm biến và truyền dữ liệu liên tục trong bài toán nhận dạng chuyển động tay theo thời gian thực.

2.5 Lựa chọn phương án truyền dữ liệu và kết nối IoT

Trong hệ thống, dữ liệu từ cảm biến MPU và FSR được STM32 thu thập, xử lý sơ bộ rồi truyền sang ESP32 để kết nối Wi-Fi và gửi dữ liệu đến máy tính qua UDP. Vì yêu cầu truyền dữ liệu ổn định và độ trễ thấp, việc lựa chọn phương thức giao tiếp giữa STM32 và ESP32 là yếu tố quan trọng.

Ba phương án giao tiếp được xem xét gồm UART, SPI và I2C. Mỗi chuẩn có ưu điểm khác nhau về tốc độ truyền, số lượng dây kết nối và độ phức tạp khi triển khai. Nhóm tiến hành phân tích các phương án này để lựa chọn giao tiếp phù hợp cho hệ thống.

2.5.1 Giao tiếp UART giữa STM32 và ESP32

UART là chuẩn giao tiếp nối tiếp không đồng bộ, sử dụng hai đường truyền TX và RX để trao đổi dữ liệu giữa STM32 và ESP32. Hai thiết bị cần cấu hình cùng thông số truyền để đảm bảo dữ liệu chính xác.

Trong hệ thống, dữ liệu được đóng gói thành chuỗi và truyền với tốc độ phù hợp nhằm hạn chế mất mẫu. ESP32 kiểm tra định dạng dữ liệu trước khi xử lý để đảm bảo tính ổn định.

UART có ưu điểm là đơn giản, dễ lập trình, hỗ trợ tốt trên STM32 và ESP32. Với lượng dữ liệu cảm biến trong đề tài, UART đáp ứng đủ yêu cầu truyền dữ liệu thời gian thực.

2.5.2 ESP32 và kết nối Wi-Fi

ESP32 được sử dụng làm bộ truyền dữ liệu không dây trong hệ thống nhờ tích hợp Wi-Fi và hỗ trợ nhiều giao tiếp như UART, I2C, SPI. ESP32 nhận dữ liệu cảm biến đã xử lý từ STM32 qua UART, kiểm tra dữ liệu và gửi đến máy tính bằng giao thức UDP.

Các nhiệm vụ chính của ESP32 trong hệ thống gồm:

- Nhận dữ liệu cảm biến đã được xử lý từ STM32 thông qua giao tiếp UART.
- Kiểm tra định dạng gói dữ liệu và loại bỏ các gói sai định dạng.
- Kết nối Wi-Fi trong cùng mạng với máy tính xử lý.
- Gửi dữ liệu cảm biến đến chương trình Python bằng giao thức UDP.

Ưu điểm của ESP32 là nhỏ gọn, giá thành phù hợp, dễ lập trình và phù hợp với yêu

cầu truyền dữ liệu thời gian thực. Tuy nhiên, hệ thống cần cấu hình đúng mạng Wi-Fi, địa chỉ IP và cổng UDP để hạn chế mất gói hoặc độ trễ khi truyền dữ liệu liên tục.

2.5.3 Giao thức UDP

UDP là giao thức truyền dữ liệu có tốc độ cao, độ trễ thấp và không yêu cầu thiết lập kết nối trước. Do không có cơ chế xác nhận gói tin, UDP phù hợp với các hệ thống thời gian thực cần truyền dữ liệu liên tục.

Trong đề tài, UDP được sử dụng để truyền dữ liệu cảm biến và kết quả nhận dạng từ ESP32 đến máy tính, giúp đối tượng trong trò chơi phản hồi nhanh theo chuyển động tay. Dữ liệu được đóng gói theo chu kỳ gồm thông tin chuyển động, giá trị FSR và kết quả nhận dạng để chương trình xử lý và cập nhật trạng thái điều khiển.

Các dữ liệu chính trong hệ thống gồm:

- **Dữ liệu cảm biến:** bao gồm các đặc trưng chuyển động từ MPU và giá trị lực từ FSR/
- **Dữ liệu nhận dạng:** là kết quả dự đoán của mô hình AI.
- **Dữ liệu điều khiển:** là lệnh được gửi đến Unity để điều khiển đối tượng trong trò chơi.

2.6 Lựa chọn phương pháp nhận dạng chuyển động tay bằng trí tuệ nhân tạo

Dữ liệu đầu vào của mô hình được thu thập từ cảm biến MPU và FSR, bao gồm các thông số về góc nghiêng, vận tốc góc và lực tác động của tay. Sau khi xử lý sơ bộ, dữ liệu được đưa vào mô hình trí tuệ nhân tạo để phân loại chuyển động và tạo lệnh điều khiển trong trò chơi.

Bài toán nhận dạng chuyển động tay trong đề tài có thể được xem là bài toán phân loại đa lớp. Mỗi mẫu dữ liệu đầu vào được biểu diễn bởi một vector đặc trưng $X = [x_1, x_2, x_3, \dots, x_n]$, trong đó $x_1, x_2, \dots, x_n$ là các đặc trưng được trích xuất từ cảm biến MPU và FSR. Kết quả đầu ra của mô hình là một nhãn chuyển động $Y \in \{C_1, C_2, C_3, \dots, C_k\}$.

Trong đó, $C_1, C_2, \dots, C_k$ là các lớp chuyển động cần nhận dạng, ví dụ như tay nghiêng trái, nghiêng phải, đưa lên, đưa xuống, bắn hoặc trạng thái nghỉ. Để lựa chọn mô hình phù hợp, nhóm khảo sát ba thuật toán phân loại phổ biến gồm Random Forest, K-Nearest

Neighbors và Gaussian Naive Bayes.

2.6.1 Thuật toán Random Forest

Random Forest là thuật toán học máy có giám sát, hoạt động dựa trên việc kết hợp nhiều cây quyết định để đưa ra kết quả phân loại. Việc sử dụng nhiều cây giúp mô hình giảm hiện tượng quá khớp, tăng độ ổn định và cải thiện khả năng nhận dạng so với một cây quyết định đơn lẻ.

Giả sử tập dữ liệu huấn luyện ban đầu được biểu diễn như sau:

$$D = \{(X_1, y_1), (X_2, y_2), \dots, (X_m, y_m)\}.$$

Trong đó, X_i là vector đặc trưng của mẫu thứ i , y_i là nhãn tương ứng và m là số lượng mẫu huấn luyện. Random Forest sẽ tạo ra nhiều tập dữ liệu con từ tập dữ liệu ban đầu bằng phương pháp lấy mẫu có hoàn lại, còn gọi là bootstrap sampling. Mỗi cây quyết định được huấn luyện trên một tập dữ liệu con khác nhau.

Với mỗi cây quyết định, tại từng nút của cây, thuật toán sẽ chọn một đặc trưng và một ngưỡng chia dữ liệu sao cho các mẫu sau khi chia trở nên thuần nhất hơn. Một chỉ số thường được sử dụng để đánh giá độ thuần nhất là chỉ số Gini. Với một nút dữ liệu t , chỉ số Gini được tính theo công thức:

$$Gini(t) = 1 - \sum_{j=1}^k p_j^2 \quad (2.1)$$

Trong đó, p_j là tỷ lệ mẫu thuộc lớp j tại nút t , còn k là số lớp cần phân loại. Nếu dữ liệu tại một nút chủ yếu thuộc về một lớp, giá trị Gini sẽ nhỏ. Ngược lại, nếu dữ liệu bị trộn lẫn nhiều lớp, giá trị Gini sẽ lớn.

Khi xét một cách chia dữ liệu tại một nút, độ giảm Gini được tính như sau:

$$\Delta Gini = Gini(t) - \frac{n_L}{n} Gini(t_L) - \frac{n_R}{n} Gini(t_R) \quad (2.2)$$

Trong đó, t là nút cha, t_L là nút con bên trái, t_R là nút con bên phải, n là số mẫu tại nút cha, n_L và n_R lần lượt là số mẫu tại nút trái và nút phải. Thuật toán sẽ chọn cách chia làm cho $\Delta Gini$ lớn nhất, tức là giúp dữ liệu sau khi chia trở nên rõ ràng hơn giữa các lớp.

Sau khi huấn luyện nhiều cây quyết định, Random Forest dự đoán nhãn đầu ra bằng cách cho từng cây đưa ra một kết quả riêng. Kết quả cuối cùng được quyết định theo

nguyên tắc bỏ phiếu đa số:

$$\hat{y} = \text{mode}(h_1(X), h_2(X), \dots, h_T(X)) \quad (2.3)$$

Trong đó, $h_i(X)$ là kết quả dự đoán của cây thứ i , T là tổng số cây trong rừng và $\hat{y}$ là nhãn dự đoán cuối cùng. Lớp nào được nhiều cây dự đoán nhất sẽ được chọn làm kết quả của mô hình.

Random Forest có hạn chế là mô hình cần nhiều bộ nhớ hơn khi số lượng cây tăng và thời gian huấn luyện lâu hơn một số thuật toán đơn giản. Tuy nhiên, trong đề tài, quá trình huấn luyện thực hiện trên máy tính, còn hệ thống chỉ cần dự đoán nên Random Forest vẫn đáp ứng yêu cầu triển khai.

2.6.2 Thuật toán K-Nearest Neighbors

K-Nearest Neighbors, viết tắt là KNN, là thuật toán phân loại dựa trên khoảng cách giữa các mẫu dữ liệu. Ý tưởng chính của thuật toán là một mẫu mới sẽ được phân loại dựa trên nhãn của K mẫu gần nó nhất trong tập dữ liệu huấn luyện. Đây là thuật toán đơn giản, dễ hiểu và dễ triển khai trong các bài toán phân loại dữ liệu cảm biến.

Giả sử có một mẫu dữ liệu cần phân loại là $X = [x_1, x_2, \dots, x_n]$. Với mỗi mẫu dữ liệu trong tập huấn luyện, ta có $X_i = [x_{i1}, x_{i2}, \dots, x_{in}]$.

Thuật toán sẽ tính khoảng cách giữa X và X_i . Khoảng cách thường được sử dụng là khoảng cách Euclid:

$$d(X, X_i) = \sqrt{\sum_{j=1}^n (x_j - x_{ij})^2} \quad (2.4)$$

Trong đó, n là số lượng đặc trưng, x_j là đặc trưng thứ j của mẫu cần phân loại và x_{ij} là đặc trưng thứ j của mẫu huấn luyện thứ i . Sau khi tính khoảng cách đến tất cả các mẫu huấn luyện, thuật toán chọn ra K mẫu có khoảng cách nhỏ nhất. Nhãn của mẫu mới được quyết định bằng cách lấy nhãn xuất hiện nhiều nhất trong K mẫu gần nhất:

$$\hat{y} = \arg \max_c \sum_{i \in N_K(X)} I(y_i = c) \quad (2.5)$$

Trong đó, $N_K(X)$ là tập K mẫu gần nhất với X , $I(y_i = c)$ là hàm chỉ thị có giá trị bằng 1 nếu mẫu thứ i thuộc lớp c , ngược lại bằng 0. Lớp nào có số phiếu nhiều nhất sẽ được chọn làm kết quả dự đoán.

KNN có ưu điểm là đơn giản, không cần quá trình huấn luyện phức tạp và phù hợp cho việc thử nghiệm dữ liệu cảm biến ban đầu. Thuật toán phân loại bằng cách tính khoảng cách giữa mẫu mới và dữ liệu đã có.

Tuy nhiên, KNN phụ thuộc vào việc chuẩn hóa dữ liệu và thời gian dự đoán tăng khi số lượng mẫu lớn. Điều này có thể ảnh hưởng đến khả năng xử lý thời gian thực của hệ thống.

2.6.3 Thuật toán Gaussian Naive Bayes

Gaussian Naive Bayes là thuật toán phân loại dựa trên định lý Bayes và giả định rằng các đặc trưng đầu vào độc lập với nhau khi biết lớp cần phân loại. Thuật toán này phù hợp với các bài toán có dữ liệu dạng số liên tục và có tốc độ huấn luyện, dự đoán nhanh.

Theo định lý Bayes, xác suất một mẫu dữ liệu X thuộc lớp C_k được tính như sau:

$$P(C_k|X) = \frac{P(X|C_k)P(C_k)}{P(X)} \quad (2.6)$$

Trong đó, $P(C_k|X)$ là xác suất mẫu X thuộc lớp C_k , $P(X|C_k)$ là xác suất xuất hiện dữ liệu X khi biết lớp C_k , $P(C_k)$ là xác suất tiên nghiệm của lớp C_k , còn $P(X)$ là xác suất xuất hiện dữ liệu X .

Với một vector đặc trưng $X = [x_1, x_2, \dots, x_n]$.

Naive Bayes giả định rằng các đặc trưng $x_1, x_2, \dots, x_n$ độc lập có điều kiện với nhau khi biết lớp C_k . Khi đó:

$$P(X|C_k) = \prod_{j=1}^n P(x_j|C_k) \quad (2.7)$$

Do $P(X)$ giống nhau đối với tất cả các lớp khi so sánh, thuật toán chỉ cần tìm lớp có giá trị xác suất lớn nhất:

$$\hat{y} = \arg \max_{C_k} P(C_k) \prod_{j=1}^n P(x_j|C_k) \quad (2.8)$$

Gaussian Naive Bayes có ưu điểm là tốc độ xử lý nhanh, mô hình đơn giản và yêu cầu ít tài nguyên, phù hợp với các hệ thống cần dự đoán nhanh.

Tuy nhiên, thuật toán giả định các đặc trưng độc lập với nhau, trong khi dữ liệu chuyển động tay từ MPU và FSR có sự liên quan giữa các thông số. Điều này có thể làm giảm độ chính xác khi nhận dạng các trạng thái chuyển động phức tạp.

2.6.4 Tổng hợp và lựa chọn phương pháp nhận dạng

Bảng 2.4: So sánh các phương pháp nhận dạng chuyển động tay.

Thuật toán	Ưu điểm	Nhược điểm	Đánh giá
Random Forest	Phù hợp với dữ liệu nhiều đặc trưng, có khả năng giảm nhiễu và đánh giá mức độ quan trọng của từng đặc trưng.	Thời gian huấn luyện tăng khi số lượng cây lớn và mô hình có thể chiếm nhiều bộ nhớ.	Phù hợp với đề tài, được lựa chọn.
K-Nearest Neighbors	Thuật toán đơn giản, dễ triển khai và phù hợp để đánh giá ban đầu dữ liệu cảm biến.	Thời gian dự đoán tăng khi dữ liệu lớn và phụ thuộc vào việc lựa chọn giá trị K .	Dễ triển khai nhưng chưa tối ưu cho dữ liệu lớn.
Gaussian Naive Bayes	Tốc độ huấn luyện và dự đoán nhanh, yêu cầu ít tài nguyên tính toán.	Giả định các đặc trưng độc lập nên độ chính xác có thể giảm với dữ liệu cảm biến.	Tốc độ cao nhưng còn hạn chế về giả định dữ liệu.

Sau khi phân tích và so sánh, nhóm lựa chọn Random Forest làm thuật toán chính cho hệ thống nhận dạng chuyển động tay. Lý do lựa chọn là Random Forest phù hợp với dữ liệu cảm biến có nhiều đặc trưng, có khả năng phân loại tốt trong điều kiện dữ liệu có nhiễu và không yêu cầu giả định phân phối dữ liệu như Gaussian Naive Bayes. Bên cạnh đó, thuật toán còn cho phép đánh giá mức độ quan trọng của từng đặc trưng, giúp nhóm phân tích được vai trò của các cảm biến MPU và FSR trong quá trình nhận dạng.

2.7 Các thuật toán lọc tín hiệu cảm biến

Dữ liệu cảm biến trong hệ thống có thể bị ảnh hưởng bởi nhiễu từ chuyển động tay, nhiễu điện hoặc sai số đo, làm giảm độ chính xác nhận dạng. Vì vậy, dữ liệu MPU và FSR cần được xử lý trước khi đưa vào mô hình.

Dữ liệu MPU được lọc để giảm dao động và kết hợp bằng bộ lọc bổ sung nhằm tính góc ổn định hơn. Tín hiệu FSR cũng được làm mượt để giảm nhiễu trước khi sử dụng cho quá trình nhận dạng.

2.7.1 Bộ lọc IIR cho dữ liệu gia tốc và con quay hồi chuyển

Sau khi dữ liệu thô từ MPU6050 được đọc ra và quy đổi sang đơn vị thực, hệ thống tiếp tục sử dụng bộ lọc IIR để làm mượt dữ liệu gia tốc và vận tốc góc. IIR là viết tắt của

Infinite Impulse Response, là dạng bộ lọc hồi quy có đầu ra hiện tại phụ thuộc vào giá trị đo hiện tại và giá trị đầu ra trước đó.

Công thức tổng quát của bộ lọc IIR bậc một được sử dụng như sau:

$$y[n] = y[n-1] + \alpha (x[n] - y[n-1]) \quad (2.9)$$

Trong đó:

- $x[n]$ là giá trị cảm biến đo được tại thời điểm hiện tại.
- $y[n]$ là giá trị sau lọc tại thời điểm hiện tại.
- $y[n-1]$ là giá trị sau lọc ở thời điểm trước đó.
- α là hệ số lọc, có giá trị trong khoảng từ 0 đến 1.

Nếu α nhỏ, tín hiệu sau lọc sẽ mượt hơn nhưng phản hồi chậm hơn. Nếu α lớn, tín hiệu phản hồi nhanh hơn nhưng khả năng giảm nhiễu kém hơn. Vì vậy, hệ số α cần được lựa chọn để cân bằng giữa độ mượt của tín hiệu và tốc độ đáp ứng của hệ thống.

Đối với dữ liệu MPU, bộ lọc IIR được áp dụng riêng cho gia tốc và con quay hồi chuyển:

$$Acc_f[n] = Acc_f[n-1] + \alpha_{acc} (Acc[n] - Acc_f[n-1]) \quad (2.10)$$

$$Gyro_f[n] = Gyro_f[n-1] + \alpha_{gyro} (Gyro[n] - Gyro_f[n-1]) \quad (2.11)$$

Trong đó, $Acc[n]$ và $Gyro[n]$ lần lượt là giá trị gia tốc và vận tốc góc đọc được từ cảm biến; $Acc_f[n]$ và $Gyro_f[n]$ là giá trị sau lọc. Dữ liệu sau lọc được sử dụng để tính góc nghiêng và làm đặc trưng đầu vào cho mô hình nhận dạng chuyển động tay.

2.7.2 Tính toán góc Roll và Pitch từ gia tốc kế

Gia tốc kế trong MPU6050 có thể được sử dụng để ước lượng góc nghiêng của cảm biến so với phương trọng lực. Khi tay đứng yên hoặc chuyển động chậm, dữ liệu gia tốc có thể phản ánh hướng của trọng lực, từ đó tính được góc Roll và Pitch.

Góc Roll được tính theo công thức:

$$Roll_{acc} = \arctan\left(\frac{AccY}{AccZ}\right) \times \frac{180}{\pi} \quad (2.12)$$

Góc Pitch được tính theo công thức:

$$Pitch_{acc} = \arctan \left(\frac{-AccX}{\sqrt{AccY^2 + AccZ^2}} \right) \times \frac{180}{\pi} \quad (2.13)$$

Trong đó:

- $AccX, AccY, AccZ$ là gia tốc theo ba trục X, Y, Z sau khi lọc.
- $Roll_{acc}$ là góc nghiêng trái hoặc phải tính từ gia tốc kế.
- $Pitch_{acc}$ là góc ngẩng lên hoặc cúi xuống tính từ gia tốc kế.

Ưu điểm của cách tính góc từ gia tốc kế là không bị trôi theo thời gian. Tuy nhiên, khi tay chuyển động nhanh, gia tốc kế không chỉ đo trọng lực mà còn đo cả gia tốc do chuyển động tạo ra. Điều này có thể làm góc tính từ gia tốc bị nhiễu. Vì vậy, hệ thống cần kết hợp thêm dữ liệu từ con quay hồi chuyển để tăng độ ổn định.

2.7.3 Bộ lọc bổ sung Complementary Filter

Con quay hồi chuyển có khả năng đo vận tốc góc, từ đó có thể tính góc bằng cách tích phân theo thời gian. Nếu vận tốc góc tại thời điểm hiện tại là $Gyro$, góc ước lượng từ con quay hồi chuyển được tính như sau:

$$Angle_{gyro}[n] = Angle[n-1] + Gyro_f[n]\Delta t \quad (2.14)$$

Trong đó, Δt là khoảng thời gian giữa hai lần cập nhật dữ liệu. Dữ liệu con quay hồi chuyển có ưu điểm là phản hồi nhanh với chuyển động, phù hợp để nhận biết sự thay đổi góc trong thời gian ngắn. Tuy nhiên, nếu chỉ sử dụng con quay hồi chuyển, sai số nhỏ sẽ tích lũy theo thời gian và gây ra hiện tượng trôi góc.

Để khắc phục nhược điểm của cả gia tốc kế và con quay hồi chuyển, hệ thống sử dụng bộ lọc bổ sung Complementary Filter. Bộ lọc này kết hợp góc tính từ con quay hồi chuyển và góc tính từ gia tốc kế theo công thức:

$$Angle[n] = \alpha (Angle[n-1] + Gyro_f[n]\Delta t) + (1 - \alpha)Angle_{acc}[n] \quad (2.15)$$

Trong đó:

- $Angle[n]$ là góc sau lọc tại thời điểm hiện tại.
- $Angle[n-1]$ là góc ở thời điểm trước đó.
- $Gyro_f[n]$ là vận tốc góc sau lọc.

- Δt là chu kỳ lấy mẫu.
- $Angle_{acc}[n]$ là góc tính từ gia tốc kế.
- α là hệ số kết hợp giữa gyro và accel.

Trong công thức trên, thành phần con quay hồi chuyển giúp hệ thống phản hồi nhanh khi tay thay đổi tư thế, còn thành phần gia tốc kế giúp hiệu chỉnh sai số trôi theo thời gian. Khi chọn α gần 1, hệ thống tin tưởng nhiều hơn vào gyro để giữ phản hồi nhanh; phần nhỏ còn lại từ gia tốc kế giúp kéo góc về giá trị ổn định lâu dài.

2.7.4 Bộ lọc IIR cho tín hiệu FSR

Cảm biến FSR được sử dụng để phát hiện lực tác động từ tay người dùng. Tín hiệu FSR được đọc thông qua ADC và có thể bị dao động do nhiễu điện, tiếp xúc cơ học hoặc sự thay đổi lực nhấn trong quá trình thao tác. Vì vậy, tín hiệu FSR cần được lọc trước khi đưa vào hệ thống nhận dạng.

Bộ lọc IIR cho FSR được biểu diễn như sau:

$$FSR_f[n] = FSR_f[n-1] + \alpha_{fsr} (FSR_{raw}[n] - FSR_f[n-1]) \quad (2.16)$$

Trong đó:

- $FSR_{raw}[n]$ là giá trị FSR đọc trực tiếp từ ADC.
- $FSR_f[n]$ là giá trị FSR sau lọc.
- α_{fsr} là hệ số lọc FSR.

Sau khi lọc, hệ thống có thể lấy giá trị trung bình của FSR trong một cửa sổ thời gian để làm đặc trưng đầu vào. Giá trị FSR sau lọc giúp giảm nhiễu và phản ánh ổn định hơn trạng thái có lực hoặc không có lực tác động. Dữ liệu này đặc biệt hữu ích trong việc nhận biết các thao tác nhấn, bóp hoặc kích hoạt lệnh điều khiển trong trò chơi.

2.7.5 Tổng hợp thuật toán lọc trong hệ thống

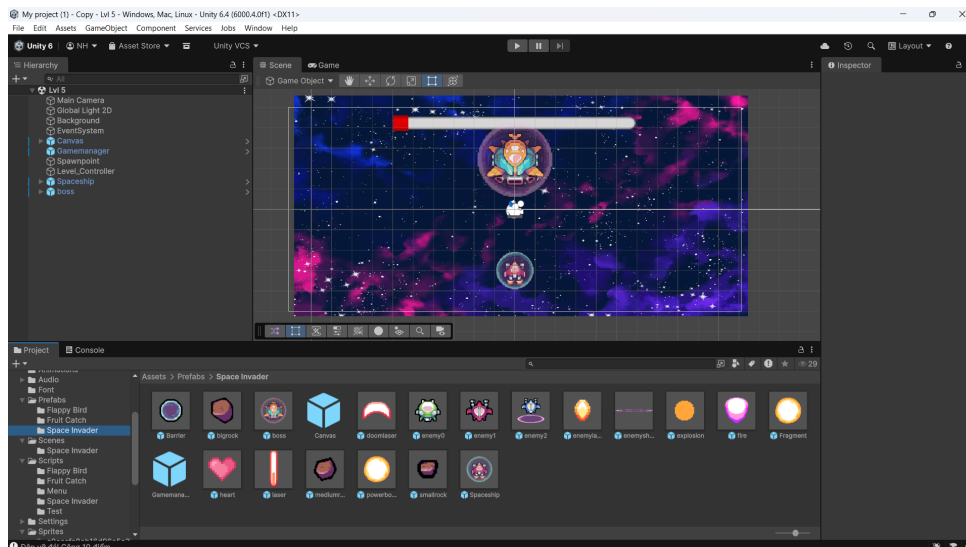
Trong hệ thống nhận dạng chuyển động tay, mỗi loại cảm biến có đặc điểm tín hiệu khác nhau nên cần áp dụng phương pháp lọc phù hợp. Đối với MPU6050, hệ thống sử dụng bộ lọc IIR và Complementary Filter để thu được góc nghiêng ổn định. Đối với FSR, hệ thống sử dụng IIR để làm mượt tín hiệu lực và xác định trạng thái tác động ổn định hơn.

Các thuật toán lọc này giúp dữ liệu đầu vào của mô hình AI ổn định, giảm sai số do nhiễu và tăng khả năng phân biệt giữa các trạng thái chuyển động tay. Nhờ đó, hệ thống có thể nhận dạng chuyển động theo thời gian thực chính xác hơn và điều khiển đối tượng trong trò chơi ổn định hơn.

2.8 Phần mềm Unity trong điều khiển đối tượng trò chơi

2.8.1 Tổng quan về Unity

Unity là game engine đa nền tảng, hỗ trợ xây dựng môi trường mô phỏng và tương tác thời gian thực. Với hệ thống Component linh hoạt và ngôn ngữ C#, Unity có khả năng kết nối dữ liệu ngoại vi, phù hợp làm giao diện hiển thị và điều khiển cho hệ thống nhận dạng cử chỉ tay.



Hình 2.1: Giao diện tổng quan của phần mềm Unity trong thiết kế môi trường mô phỏng

2.8.2 Nhận dữ liệu điều khiển từ UDP

Để thiết lập luồng giao tiếp giữa phần cứng và trò chơi, Unity đóng vai trò là trạm thu nhận tín hiệu cục bộ thông qua giao thức mạng UDP. Thay vì phải giao tiếp với máy chủ đám mây có độ trễ cao, Unity mở một cổng mạng (Port) để lắng nghe và đón trực tiếp các gói tin do chương trình Python gửi sang. Cụ thể, kết quả nhận dạng cử chỉ từ thiết bị (như nhãn hành động phân loại) sẽ được tải về và giải mã (Parse) thành các đối tượng dữ liệu trong C#.

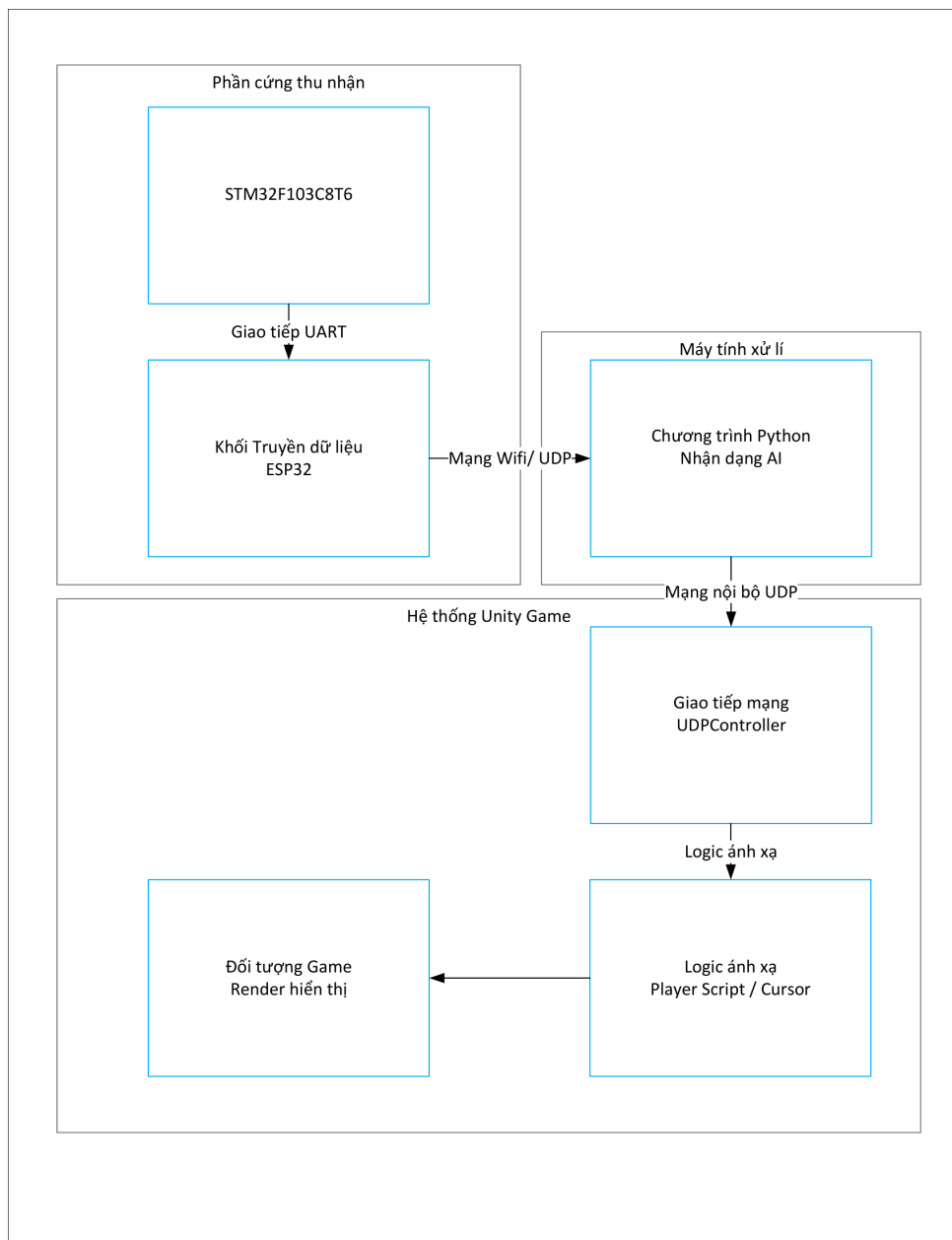
Sau khi giải mã, hệ thống tiến hành ánh xạ (mapping) các kết quả dự đoán thành

các tập lệnh điều khiển cụ thể. Ví dụ, các chuỗi ký tự trạng thái như "UP", "DOWN", "LEFT", "RIGHT" được chuyển đổi thành các vector chỉ hướng, trong khi các giá trị số nguyên đặc tả hành động nắm tay được chuyển đổi thành trạng thái logic (boolean) để kích hoạt sự kiện tương tác trong không gian ảo.

2.8.3 Điều khiển nhân vật hoặc đối tượng trong trò chơi

Quá trình điều khiển đối tượng trong Unity được thực hiện dựa trên các dữ liệu đã ánh xạ từ UDP, bao gồm các cơ chế chính sau:

- **Điều khiển hướng di chuyển:** Các lệnh định hướng được can thiệp trực tiếp vào thành phần Transform hoặc Rigidbody2D của đối tượng. Tùy thuộc vào logic của từng trò chơi (như điều khiển phi thuyền không gian hay di chuyển đối tượng hứng vật phẩm), vị trí của nhân vật sẽ được cập nhật liên tục trên các trục tọa độ dựa trên giá trị vận tốc quy định và thời gian khung hình thực tế.
- **Thực hiện thao tác đặc biệt:** Đối với các hành động như bắn tia laser, nhảy lên (tác dụng lực đẩy), hoặc tương tác điều hướng menu hệ thống, tín hiệu kích hoạt được xử lý thông qua cơ chế phát hiện thay đổi trạng thái (chống dội tín hiệu). Điều này đảm bảo mỗi cử chỉ tay thực tế chỉ tạo ra một hành động logic duy nhất trong môi trường mô phỏng.



Hình 2.2: Sơ đồ khối mô tả quá trình ánh xạ tín hiệu điều khiển vào đối tượng trò chơi

Chương 3. THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

Trong hệ thống được đề xuất, các cảm biến được gắn trên tay người dùng để thu thập thông tin về chuyển động và lực tác động trong quá trình thao tác. Cảm biến MPU đảm nhiệm việc đo các thông số như góc nghiêng, vận tốc góc và sự thay đổi tư thế của bàn tay. Trong khi đó, cảm biến FSR được sử dụng để phát hiện lực nhấn, từ đó bổ sung thêm các lệnh điều khiển trong trò chơi. Sau khi dữ liệu được thu thập, vi điều khiển sẽ thực hiện xử lý ban đầu trước khi gửi đến chương trình nhận dạng để xác định cử chỉ và tạo lệnh điều khiển tương ứng.

Chương này trình bày các yêu cầu thiết kế của hệ thống, kiến trúc tổng thể, phương án xây dựng phần cứng, phần mềm cũng như cơ chế truyền dữ liệu giữa các thành phần. Đây là cơ sở để xây dựng một hệ thống hoạt động ổn định, có khả năng nhận dạng chuyển động tay theo thời gian thực và điều khiển đối tượng trong môi trường trò chơi.

3.1 Yêu cầu về chức năng thiết kế hệ thống

Để đáp ứng mục tiêu của đề tài, hệ thống cần được thiết kế dựa trên các yêu cầu về chức năng, tốc độ xử lý và khả năng truyền dữ liệu theo thời gian thực. Đồng thời, việc xác định rõ dữ liệu đầu vào và kết quả đầu ra là cơ sở cho quá trình lựa chọn phần cứng, xây dựng thuật toán và phát triển chương trình điều khiển.

3.1.1 Yêu cầu chức năng

Hệ thống phải thu nhận được dữ liệu chuyển động tay thông qua các cảm biến gắn trên người dùng. Các cảm biến cần theo dõi được sự thay đổi tư thế của bàn tay khi thực hiện các thao tác như nghiêng sang trái, nghiêng sang phải, đưa lên, đưa xuống hoặc giữ nguyên vị trí. Ngoài ra, cảm biến FSR có nhiệm vụ phát hiện thao tác nhấn để tạo thêm tín hiệu điều khiển phục vụ trò chơi.

Các chức năng chính của hệ thống bao gồm:

- Thu thập dữ liệu từ cảm biến MPU và FSR.

- Xử lý dữ liệu ban đầu, bao gồm lọc nhiễu, hiệu chỉnh và tính toán các thông số cần thiết.
- Truyền dữ liệu từ STM32 đến ESP32 thông qua giao tiếp UART.
- Gửi dữ liệu từ ESP32 đến máy tính bằng giao thức UDP.
- Đưa dữ liệu vào mô hình trí tuệ nhân tạo để nhận dạng chuyển động tay.
- Truyền kết quả nhận dạng đến Unity để điều khiển đối tượng trong trò chơi.
- Cập nhật lệnh điều khiển liên tục nhằm bảo đảm phản hồi nhanh theo thao tác của người dùng.

Ngoài các yêu cầu trên, hệ thống cần duy trì hoạt động ổn định trong suốt quá trình sử dụng. Dữ liệu cảm biến phải có chất lượng đủ tốt để mô hình AI phân biệt chính xác các trạng thái chuyển động khác nhau. Đồng thời, vị trí lắp đặt cảm biến cần được bố trí hợp lý, tạo cảm giác thoải mái và hạn chế ảnh hưởng đến các thao tác tự nhiên của người dùng.

3.1.2 Yêu cầu về khả năng hoạt động theo thời gian thực

Do hệ thống được xây dựng nhằm điều khiển đối tượng trong trò chơi bằng chuyển động tay, một trong những yêu cầu quan trọng là khả năng hoạt động theo thời gian thực. Toàn bộ quá trình từ thu thập dữ liệu cảm biến, xử lý thông tin, nhận dạng cử chỉ đến gửi lệnh điều khiển cần được thực hiện với độ trễ thấp để bảo đảm phản hồi nhanh. Nếu thời gian xử lý quá dài, thao tác của người dùng sẽ không được cập nhật kịp thời, làm giảm độ chính xác và trải nghiệm khi điều khiển trò chơi.

Để đáp ứng yêu cầu này, hệ thống cần thỏa mãn các điều kiện sau:

- Tốc độ lấy mẫu của cảm biến phải đủ cao để ghi nhận chính xác các thay đổi trong quá trình chuyển động tay.
- Thuật toán xử lý dữ liệu cần được tối ưu, bảo đảm phù hợp với năng lực tính toán của vi điều khiển.
- Dữ liệu truyền giữa các khối phải được đóng gói ngắn gọn, chỉ bao gồm những thông tin cần thiết nhằm giảm thời gian truyền.
- Việc truyền dữ liệu từ STM32 đến ESP32 thông qua giao tiếp UART phải ổn định, hạn chế lỗi và mất dữ liệu.

- Giao thức UDP được sử dụng để gửi dữ liệu từ ESP32 đến máy tính cần duy trì độ trễ thấp và tần suất cập nhật phù hợp.
- Mô hình trí tuệ nhân tạo phải có thời gian suy luận nhanh để đáp ứng yêu cầu nhận dạng theo thời gian thực.
- Kết quả nhận dạng cần được truyền ngay đến Unity để đối tượng trong trò chơi phản hồi gần như tức thời.

Bên cạnh đó, dữ liệu từ cảm biến sẽ được xử lý và trích xuất các đặc trưng cần thiết trước khi truyền đến máy tính thay vì gửi toàn bộ tín hiệu thô. Cách tiếp cận này giúp giảm kích thước gói tin, tăng tốc độ truyền dữ liệu và hạn chế nguy cơ quá tải khi hệ thống hoạt động liên tục.

3.1.3 Yêu cầu về dữ liệu và kết quả điều khiển

Dữ liệu đầu vào của hệ thống được thu thập từ cảm biến MPU và cảm biến FSR. Trong đó, cảm biến MPU cung cấp các thông tin liên quan đến trạng thái chuyển động của bàn tay như góc nghiêng, vận tốc góc và sự thay đổi tư thế trong quá trình thao tác. Cảm biến FSR có nhiệm vụ đo lực nhấn của người dùng, giúp nhận biết các thao tác kích hoạt hoặc thực hiện lệnh điều khiển bổ sung.

Các loại dữ liệu chính được sử dụng trong hệ thống gồm:

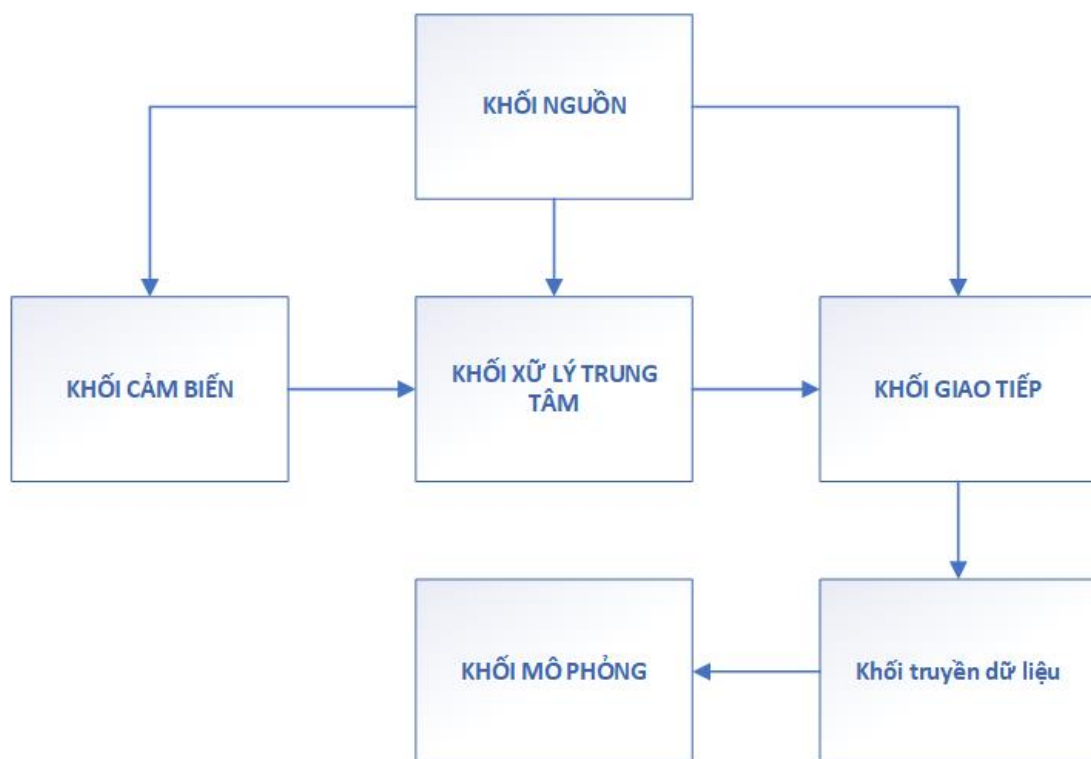
- Dữ liệu góc nghiêng của bàn tay, bao gồm các giá trị Roll và Pitch sau khi đã được xử lý.
- Dữ liệu góc tương đối giữa các vị trí gắn cảm biến nhằm mô tả sự thay đổi tư thế của tay.
- Dữ liệu vận tốc góc từ cảm biến MPU để phản ánh tốc độ thay đổi của chuyển động.
- Dữ liệu lực từ cảm biến FSR dùng để xác định trạng thái nhấn hoặc không nhấn.
- Dữ liệu trạng thái và mã gói tin phục vụ việc kiểm tra tính hợp lệ trong quá trình truyền nhận dữ liệu.

Trước khi đưa vào mô hình trí tuệ nhân tạo, dữ liệu cảm biến được xử lý nhằm giảm nhiễu và tăng độ ổn định. Đối với cảm biến MPU, dữ liệu được lọc và tính toán để xác định chính xác góc nghiêng của bàn tay. Đối với cảm biến FSR, tín hiệu được xử lý nhằm hạn chế dao động khi người dùng thực hiện thao tác nhấn hoặc nhả. Sau khi hoàn thành

bước tiền xử lý, các đặc trưng thu được sẽ được sử dụng làm dữ liệu đầu vào cho mô hình nhận dạng.

Đầu ra của hệ thống là kết quả phân loại cử chỉ tay cùng với lệnh điều khiển tương ứng trong trò chơi. Dựa trên kết quả nhận dạng, hệ thống sẽ tạo các lệnh như di chuyển sang trái, sang phải, đi lên, đi xuống, giữ nguyên trạng thái hoặc kích hoạt các thao tác đặc biệt. Các lệnh này được gửi đến phần mềm trò chơi để điều khiển đối tượng theo chuyển động của người dùng theo thời gian thực.

3.2 Sơ đồ tổng quát của hệ thống



Hình 3.1: Sơ đồ tổng quát của hệ thống

Trong hệ thống đề xuất, **khối cảm biến** có nhiệm vụ thu thập thông tin về chuyển động của bàn tay và lực tác động từ người dùng. Các cảm biến được bố trí tại những vị trí thích hợp trên tay để ghi nhận đầy đủ sự thay đổi tư thế trong quá trình thao tác. Cảm biến MPU được sử dụng để đo các thông số chuyển động như góc nghiêng và sự thay đổi hướng của bàn tay, trong khi cảm biến FSR dùng để phát hiện lực nhấn, phục vụ việc kích hoạt các lệnh điều khiển bổ sung. Sau khi được thu thập, dữ liệu sẽ được chuyển đến bộ xử lý trung tâm để tiếp tục xử lý.

Khối xử lý trung tâm sử dụng vi điều khiển STM32 để quản lý và xử lý dữ liệu cảm

biến. STM32 giao tiếp với MPU thông qua chuẩn I2C và đọc tín hiệu từ FSR bằng bộ chuyển đổi ADC. Sau khi nhận dữ liệu, vi điều khiển tiến hành lọc nhiễu, tính toán các thông số cần thiết và tạo gói dữ liệu trước khi gửi sang khối truyền thông.

Khối nguồn đảm bảo cung cấp điện áp ổn định cho toàn bộ hệ thống, bao gồm các cảm biến, vi điều khiển và module ESP32. Nguồn điện ổn định giúp hạn chế nhiễu, tăng độ tin cậy của dữ liệu và duy trì hoạt động liên tục của hệ thống.

Khối truyền thông sử dụng ESP32 để nhận dữ liệu từ STM32 qua giao tiếp UART. Sau khi kiểm tra tính hợp lệ của gói tin, ESP32 truyền dữ liệu đến máy tính thông qua mạng Wi-Fi bằng giao thức UDP. Việc lựa chọn UDP giúp giảm thời gian truyền và đáp ứng tốt yêu cầu điều khiển theo thời gian thực.

Trên máy tính, **khối nhận dạng** tiếp nhận dữ liệu từ ESP32 và đưa vào mô hình trí tuệ nhân tạo để xác định cử chỉ tay. Kết quả phân loại sau đó được chuyển thành các lệnh điều khiển và gửi đến chương trình mô phỏng.

Khối mô phỏng được xây dựng bằng Unity để hiển thị kết quả điều khiển của hệ thống. Dựa trên cử chỉ đã nhận dạng, nhân vật hoặc đối tượng trong trò chơi sẽ thực hiện các hành động tương ứng như di chuyển theo các hướng hoặc kích hoạt các thao tác đặc biệt. Khối này đồng thời được sử dụng để đánh giá khả năng phản hồi và hiệu quả hoạt động của hệ thống.

Tổng thể, hệ thống vận hành theo trình tự: cảm biến thu thập dữ liệu từ chuyển động của tay, STM32 xử lý và đóng gói dữ liệu, ESP32 truyền dữ liệu đến máy tính qua UDP, mô hình AI thực hiện nhận dạng cử chỉ và Unity nhận kết quả để điều khiển đối tượng trong trò chơi. Việc phân chia thành các khối chức năng độc lập giúp quá trình thiết kế, tích hợp và kiểm thử hệ thống được thực hiện thuận lợi và dễ dàng hơn.

3.3 Thiết kế phần cứng

Hệ thống phần cứng được xây dựng với mục tiêu thu thập dữ liệu về chuyển động của bàn tay và lực tác động của người dùng, từ đó cung cấp dữ liệu cho quá trình nhận dạng cử chỉ theo thời gian thực. Để thực hiện nhiệm vụ này, hệ thống sử dụng hai loại cảm biến là MPU6050 và FSR402. Trong đó, MPU6050 đảm nhận việc đo gia tốc và vận tốc góc, đồng thời hỗ trợ xác định góc nghiêng của bàn tay. Cảm biến FSR402 được sử dụng

để phát hiện lực nhấn, giúp nhận biết các thao tác kích hoạt trong quá trình điều khiển.

Vi điều khiển STM32F103C8T6 đóng vai trò là bộ xử lý chính, chịu trách nhiệm giao tiếp với các cảm biến, thu nhận tín hiệu và thực hiện các bước xử lý ban đầu như lọc dữ liệu, tính toán các thông số cần thiết và tạo gói dữ liệu. Sau khi hoàn thành quá trình xử lý, dữ liệu được truyền đến module ESP32 thông qua giao tiếp UART.

ESP32 đảm nhiệm chức năng truyền thông của hệ thống. Module này nhận dữ liệu từ STM32, thiết lập kết nối Wi-Fi và gửi thông tin đến nền tảng lưu trữ hoặc chương trình xử lý thông qua mạng không dây. Việc sử dụng riêng STM32 cho xử lý cảm biến và ESP32 cho truyền dữ liệu giúp phân tách nhiệm vụ giữa các phần cứng, giảm tải cho từng vi điều khiển, đồng thời nâng cao tính ổn định và hiệu quả hoạt động của toàn bộ hệ thống.

3.3.1 Thiết kế khối cảm biến chuyển động MPU6050

Cảm biến MPU6050 là cảm biến quán tính tích hợp 6 bậc tự do, bao gồm gia tốc kế ba trục và con quay hồi chuyển ba trục. Trong hệ thống, cảm biến này được sử dụng để thu thập các thông số chuyển động của cánh tay như gia tốc và vận tốc góc theo ba trục không gian. Dựa trên dữ liệu thu nhận được, hệ thống tính toán các góc Roll và Pitch, từ đó xác định tư thế và hướng chuyển động của tay trong quá trình điều khiển.

Để tăng khả năng mô tả chuyển động, hệ thống sử dụng hai cảm biến MPU6050. Một cảm biến được lắp tại cánh tay trên và cảm biến còn lại được đặt ở cẳng tay. Cách bố trí này cho phép theo dõi chuyển động của từng đoạn tay riêng biệt, đồng thời xác định góc tương đối giữa hai vị trí. Nhờ đó, dữ liệu đầu vào phản ánh chính xác hơn sự thay đổi tư thế của tay, giúp cải thiện hiệu quả của mô hình nhận dạng.

Trong quá trình lắp đặt, các cảm biến phải được cố định chắc chắn để tránh dịch chuyển hoặc xoay lệch khi người dùng thực hiện thao tác. Nếu cảm biến thay đổi vị trí so với tay, các giá trị đo sẽ không còn phản ánh đúng chuyển động thực tế và có thể làm giảm độ chính xác của hệ thống. Có thể sử dụng dây đeo, đai cố định hoặc băng dính hai mặt để gắn cảm biến. Đồng thời, cần thống nhất hướng lắp đặt giữa các lần thử nghiệm nhằm bảo đảm các giá trị Roll và Pitch luôn được tính theo cùng một hệ quy chiếu.

Ở trạng thái nghỉ, góc tương đối giữa hai cảm biến thường có giá trị gần bằng không. Khi người dùng nghiêng tay sang trái hoặc sang phải, góc Roll sẽ thay đổi đáng kể; ngược

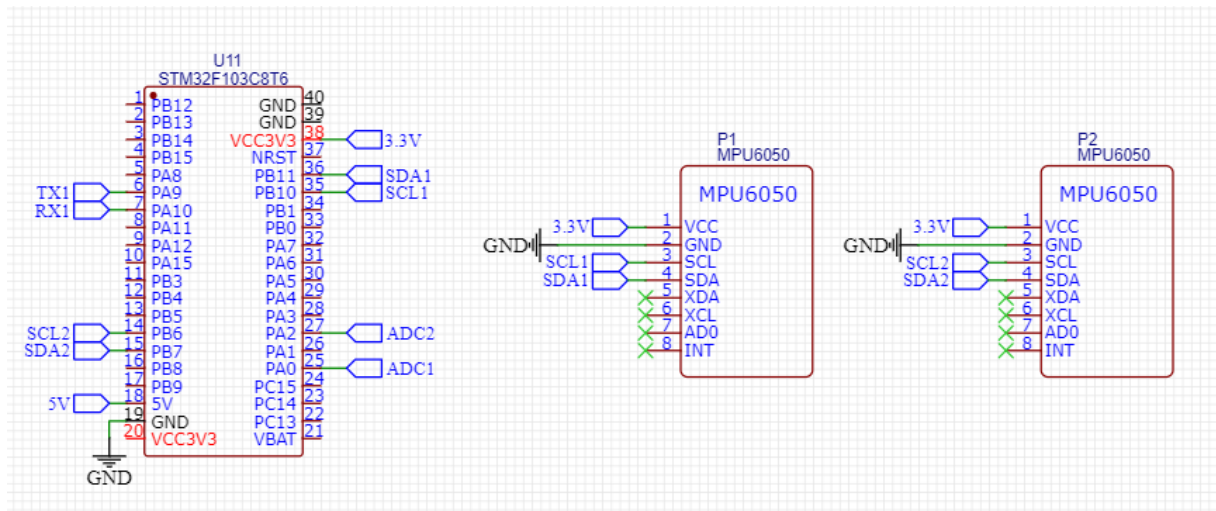
lại, khi thực hiện động tác nâng hoặc hạ tay, sự thay đổi chủ yếu thể hiện ở góc Pitch. Do chiều tăng hoặc giảm của các góc phụ thuộc vào hướng gắn cảm biến, quá trình hiệu chuẩn ban đầu cần được thực hiện để xác định chính xác mối liên hệ giữa từng chuyển động và lệnh điều khiển tương ứng trong hệ thống.

Bảng 3.1: Kết nối chân giữa MPU6050 thứ nhất và STM32F1.

Chân MPU6050 thứ nhất	Chân STM32F103C8T6	Chức năng	Ghi chú
VCC	3.3V	Cấp nguồn cho cảm biến	Sử dụng mức nguồn 3.3V
GND	GND	Mass của cảm biến	Nối chung GND toàn hệ thống
SCL	PB6	Xung clock của I2C1	Kết nối với chân I2C1_SCL
SDA	PB7	Đường dữ liệu của I2C1	Kết nối với chân I2C1_SDA

Bảng 3.2: Kết nối chân giữa MPU6050 thứ hai và STM32F1.

Chân MPU6050 thứ hai	Chân STM32F1	Chức năng	Ghi chú
VCC	3.3V	Cấp nguồn cho cảm biến	Sử dụng mức nguồn 3.3V
GND	GND	Mass của cảm biến	Nối chung GND toàn hệ thống
SCL	PB10	Xung clock của I2C2	Kết nối với chân I2C2_SCL
SDA	PB11	Đường dữ liệu của I2C2	Kết nối với chân I2C2_SDA



Hình 3.2: sơ đồ kết nối chân giữa mpu và stm32

3.3.2 Thiết kế khối cảm biến lực FSR402

Cảm biến FSR402 là cảm biến lực hoạt động theo nguyên lý thay đổi điện trở theo áp lực tác động lên bề mặt cảm biến. Khi không có lực tác động, điện trở của cảm biến có giá trị rất lớn. Ngược lại, khi người dùng nhấn hoặc tác động lực lên vùng cảm biến, điện trở sẽ giảm theo mức lực tác động. Để vi điều khiển có thể đọc được tín hiệu này, FSR402 được mắc trong mạch phân áp nhằm chuyển đổi sự thay đổi điện trở thành điện áp đầu ra, sau đó điện áp được đưa vào chân ADC của STM32 để chuyển đổi sang giá trị số.

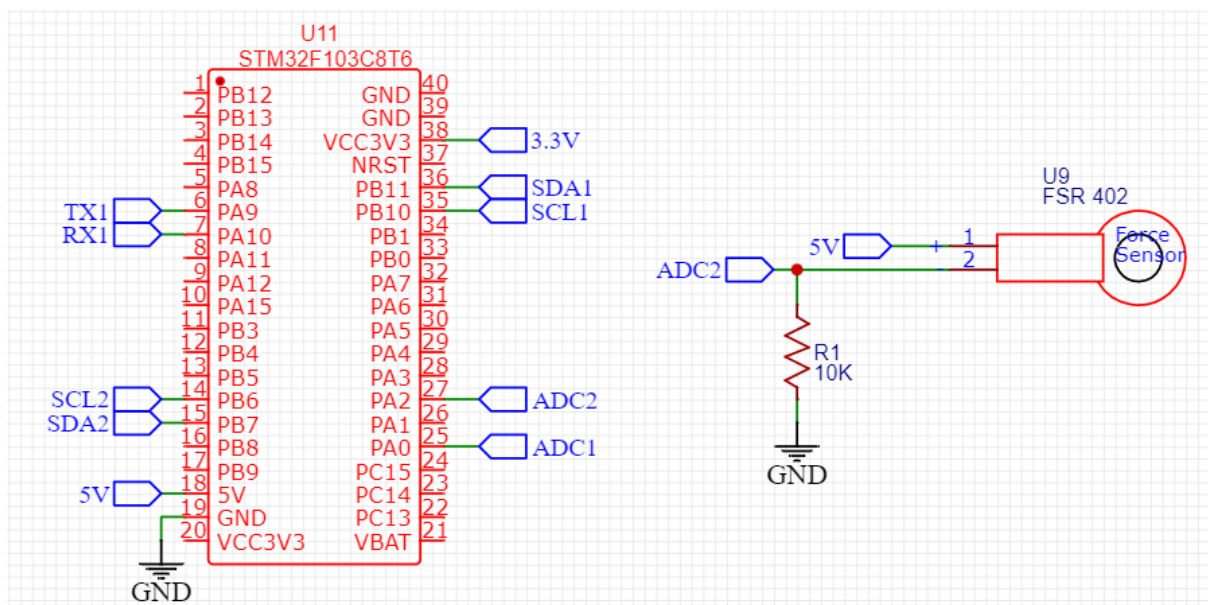
Trong đề tài, cảm biến FSR402 không được sử dụng để đo giá trị lực tuyệt đối mà chỉ nhằm xác định trạng thái có hoặc không có lực tác động. Giá trị ADC thu được từ cảm biến được dùng như một đặc trưng đầu vào cho mô hình trí tuệ nhân tạo, đồng thời hỗ trợ nhận biết các thao tác kích hoạt hoặc thực hiện lệnh điều khiển trong trò chơi.

Nguyên lý hoạt động của cảm biến có thể được mô tả như sau: khi người dùng chưa tác động lực, điện trở của FSR402 ở mức cao nên điện áp đầu ra của mạch phân áp có giá trị thấp. Khi lực nhấn tăng lên, điện trở của cảm biến giảm, làm điện áp đầu ra thay đổi theo. STM32 thực hiện lấy mẫu điện áp thông qua bộ chuyển đổi ADC và chuyển thành dữ liệu số để phục vụ quá trình xử lý. Với bộ ADC 12 bit tích hợp trên STM32, giá trị số thu được nằm trong khoảng từ 0 đến 4095, tạo điều kiện thuận lợi cho việc xử lý và phân loại dữ liệu trong hệ thống.

Bảng 3.3: Kết nối cảm biến lực FSR402 với STM32F1.

Chân/Điểm nối	Kết nối đến	Chức năng	Ghi chú
Một chân FSR402	5V	Cấp nguồn cho cảm biến	Dùng nguồn 3.3V
Chân còn lại FSR402	PA2 STM32	Đưa tín hiệu lực vào ADC	PA2 là ADC1_IN2
Điện trở 10kΩ	PA2 xuống GND	Tạo mạch phân áp	Giúp tạo điện áp thay đổi theo lực
GND	GND hệ thống	Mass chung	Nối chung với STM32 và ESP32

Tín hiệu FSR được đọc bằng ADC, sau khi đọc giá trị này được lọc để giảm dao động và làm ổn định dữ liệu lực. Khi người dùng không nắm hoặc không tác động lực, giá trị đọc từ FSR thường ở mức thấp. Khi người dùng bóp hoặc nhấn, giá trị FSR tăng lên rõ rệt. Sự thay đổi này được sử dụng để phân biệt trạng thái điều khiển liên quan đến lực.



Hình 3.3: Sơ đồ kết nối chân giữa FSR và stm32

3.3.3 Thiết kế khối xử lý trung tâm STM32

STM32 được bố trí ở trung tâm mạch để thuận tiện kết nối với các khối cảm biến và khối truyền dữ liệu ESP32. Các chân giao tiếp được lựa chọn dựa trên ngoại vi có sẵn của vi điều khiển, bao gồm I2C để đọc MPU6050, ADC để đọc FSR402 và UART để truyền dữ liệu sang ESP32.

Hai cảm biến MPU6050 được kết nối vào hai bus I2C riêng biệt của STM32. Cảm

biến thứ nhất gắn ở phần cánh tay được kết nối với I2C1, cảm biến thứ hai gắn ở phần cẳng tay được kết nối với I2C2. Cách bố trí này giúp tránh hiện tượng trùng địa chỉ I2C giữa hai cảm biến MPU6050, đồng thời giúp chương trình dễ phân biệt dữ liệu của từng vị trí đặt cảm biến.

Bảng 3.4: Tổng hợp các chân sử dụng trên STM32F103C8T6.

Chân STM32F103C8T6	Kết nối đến	Chức năng	Ghi chú
PB6	SCL của MPU6050 thứ nhất	Xung clock I2C1	Đọc cảm biến cánh tay
PB7	SDA của MPU6050 thứ nhất	Dữ liệu I2C1	Đọc cảm biến cánh tay
PB10	SCL của MPU6050 thứ hai	Xung clock I2C2	Đọc cảm biến cẳng tay
PB11	SDA của MPU6050 thứ hai	Dữ liệu I2C2	Đọc cảm biến cẳng tay
PA2	Điểm tín hiệu mạch FSR402	Đọc ADC cảm biến lực	ADC1_IN2
PA9	RX của ESP32	Truyền dữ liệu UART từ STM32	USART1_TX
PA10	TX của ESP32	Nhận dữ liệu UART từ ESP32	USART1_RX, dùng khi cần truyền hai chiều
5V	VCC các cảm biến	Cấp nguồn	Dùng chung mức 5V
GND	GND toàn hệ thống	Mass chung	Bắt buộc nối chung với ESP32 và cảm biến

Dữ liệu cảm biến sau khi đọc được xử lý sơ bộ trước khi truyền đi. Đối với MPU6050, STM32 lọc dữ liệu gia tốc và vận tốc góc, sau đó tính toán góc Roll và Pitch. Đối với FSR402, STM32 lọc giá trị ADC để làm ổn định dữ liệu lực. Sau khi xử lý, các giá trị đặc trưng được đóng gói thành một chuỗi dữ liệu và gửi sang ESP32 thông qua UART.

3.3.4 Thiết kế khối truyền dữ liệu ESP32

ESP32 được sử dụng làm khối truyền dữ liệu và kết nối IoT của hệ thống. ESP32 có tích hợp Wi-Fi, phù hợp với các ứng dụng cần gửi dữ liệu cảm biến lên nền tảng lưu trữ hoặc hệ thống giám sát trực tuyến. Trong đề tài, ESP32 không đảm nhiệm nhiệm vụ đọc toàn bộ cảm biến mà chủ yếu nhận dữ liệu từ STM32, kiểm tra dữ liệu và gửi lên nền

tảng IoT.

Bảng 3.5: Kết nối chân của ESP32 trong hệ thống.

Chân ESP32	Kết nối đến	Chức năng	Ghi chú
GPIO16	PA9 của STM32	Nhận dữ liệu UART từ STM32	RXD2 của ESP32
GPIO17	PA10 của STM32	Truyền dữ liệu UART về STM32	TXD2 của ESP32, dùng khi cần truyền hai chiều
5V	Nguồn cấp phù hợp	Cấp nguồn cho ESP32	Tùy loại board ESP32 sử dụng
GND	GND STM32 và cảm biến	Mass chung	Bắt buộc nối chung GND toàn hệ thống
Wi-Fi	Router hoặc điểm phát Wi-Fi	Kết nối mạng không dây	Dùng để gửi dữ liệu lên nền tảng IoT

Khi STM32 gửi một gói dữ liệu, ESP32 đọc chuỗi dữ liệu đến ký tự kết thúc dòng, sau đó kiểm tra phần đầu gói tin và số lượng trường dữ liệu. Nếu gói dữ liệu hợp lệ, ESP32 sẽ chuyển dữ liệu lên nền tảng lưu trữ. Nếu gói dữ liệu sai định dạng hoặc thiếu giá trị, ESP32 bỏ qua gói tin để tránh làm sai dữ liệu đầu vào của mô hình nhận dạng.

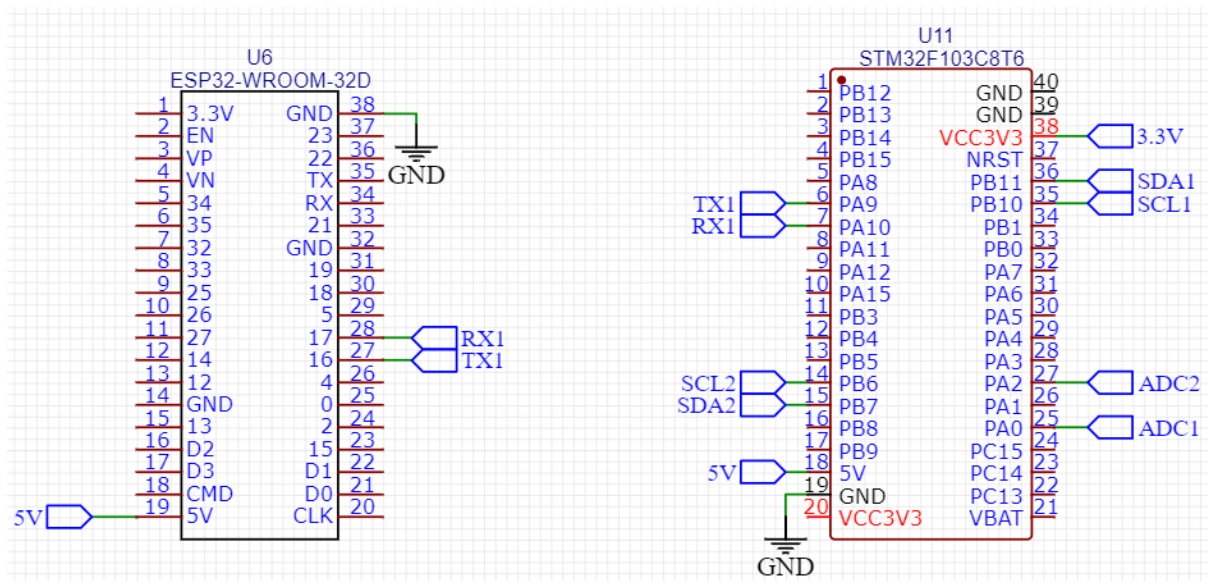
3.3.5 Thiết kế giao tiếp giữa STM32 và ESP32

Giao tiếp giữa STM32 và ESP32 được thực hiện bằng chuẩn UART. Đây là chuẩn truyền thông nối tiếp phổ biến, dễ lập trình và phù hợp để truyền dữ liệu dạng chuỗi giữa hai vi điều khiển. UART sử dụng hai đường tín hiệu chính là TX và RX. Chân TX của thiết bị truyền được nối với chân RX của thiết bị nhận và ngược lại.

Trong hệ thống, STM32 đóng vai trò gửi dữ liệu cảm biến đã xử lý, còn ESP32 đóng vai trò nhận dữ liệu và gửi lên nền tảng IoT. Hai vi điều khiển cần được cấu hình cùng tốc độ baud rate, số bit dữ liệu, bit stop và parity để đảm bảo quá trình truyền nhận chính xác.

Bảng 3.6: Kết nối UART giữa STM32F103C8T6 và ESP32.

STM32F103C8T6	ESP32	Chức năng	Ghi chú
PA9 – USART1_TX	GPIO16 – RXD2	STM32 truyền dữ liệu sang ESP32	Kết nối TX sang RX
PA10 – USART1_RX	GPIO17 – TXD2	ESP32 truyền dữ liệu về STM32	Dùng khi cần truyền hai chiều
GND	GND	Mass chung cho UART	Bắt buộc nối chung GND



Hình 3.4: sơ đồ kết nối uart giữa stm và esp

3.3.6 Thiết kế khối nguồn

Khối nguồn có nhiệm vụ cung cấp điện áp ổn định cho toàn bộ hệ thống phần cứng, bao gồm STM32, ESP32, các cảm biến MPU6050 và cảm biến FSR402. Việc thiết kế nguồn ổn định là yếu tố quan trọng vì các cảm biến analog và vi điều khiển dễ bị ảnh hưởng nếu điện áp cấp không ổn định. Nguồn nhiễu hoặc sụt áp có thể làm dữ liệu cảm biến dao động, gây sai lệch trong quá trình nhận dạng.

Trong hệ thống, các module chính hoạt động ở mức điện áp 3.3V. STM32F103C8T6, ESP32 và MPU6050 đều sử dụng mức logic 3.3V. Đối với FSR402, mạch phân áp cũng sử dụng nguồn 3.3V để điện áp đưa vào ADC không vượt quá ngưỡng cho phép của STM32.

Bảng 3.7: Yêu cầu điện áp và dòng cấp cho các khối phần cứng.

Khối sử dụng nguồn	Điện áp cấp	Dòng dự kiến	Chức năng	Ghi chú
STM32F103C8T6	5V	Khoảng 100mA	Khối xử lý trung tâm	Cấp vào chân nguồn của board STM32
ESP32	5V	Khoảng 300mA–500mA	Khối truyền dữ liệu Wi-Fi	Cần nguồn ổn định khi ESP32 truyền Wi-Fi
MPU6050 thứ nhất	3.3V	Khoảng 10mA	Cảm biến chuyển động cánh tay	Nối chung GND với STM32
MPU6050 thứ hai	3.3V	Khoảng 10mA	Cảm biến chuyển động cẳng tay	Nối chung GND với STM32
FSR402 và mạch phân áp	5V	Nhỏ hơn 5mA	Cảm biến lực	Tín hiệu đưa vào ADC phải giới hạn dưới 3.3V
Toàn hệ thống	GND chung	–	Mass tham chiếu	Bắt buộc nối chung GND các khối

Từ bảng yêu cầu nguồn, có thể thấy khối tiêu thụ dòng lớn nhất là ESP32, đặc biệt khi module kết nối Wi-Fi và truyền dữ liệu. Vì vậy, nguồn 5V cấp cho STM32, ESP32 và mạch cảm biến FSR402 cần có khả năng cung cấp dòng ổn định. Các cảm biến MPU6050 tiêu thụ dòng nhỏ hơn nhiều và được cấp bằng mức 3.3V để phù hợp với mức điện áp hoạt động của cảm biến.

Dựa trên yêu cầu đó, khối nguồn được thiết kế từ bộ pin Li-ion gồm 3 cell mắc nối tiếp. Mỗi cell Li-ion có điện áp danh định khoảng 3.7V, do đó khi mắc nối tiếp 3 cell sẽ tạo ra điện áp danh định khoảng:

$$V_{pin} = 3 \times 3.7V = 11.1V \quad (3.1)$$

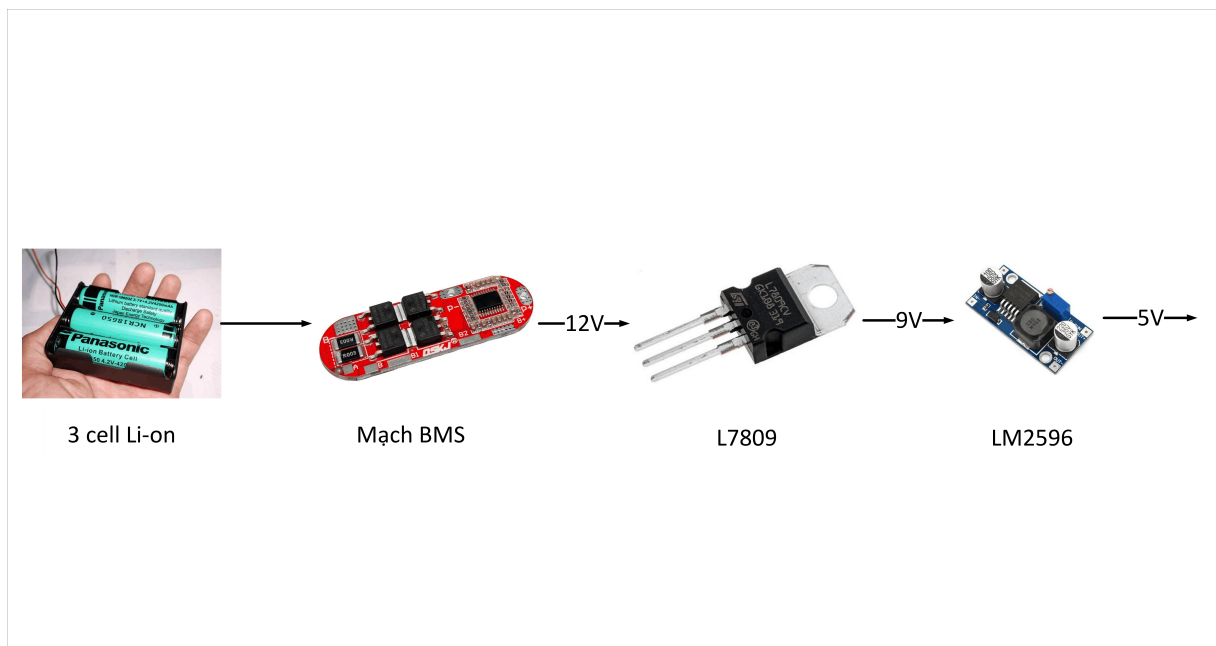
Khi sạc đầy, điện áp của bộ pin có thể đạt khoảng 12.4V. Vì vậy, trong mạch nguồn, bộ pin 3 cell được sử dụng làm nguồn đầu vào xấp xỉ 12V. Bộ pin được đưa qua mạch BMS để bảo vệ trong quá trình sử dụng, hạn chế các trường hợp quá sạc, quá xả và quá dòng.

Để bảo vệ bộ pin trong quá trình sử dụng, các cell pin được kết nối qua mạch BMS. Mạch BMS có nhiệm vụ bảo vệ pin khi sạc và xả, hạn chế các trường hợp quá áp, quá dòng, ngắn mạch hoặc xả pin quá mức. Các điểm nối B0, B1, B2 và B3 trên mạch BMS

được nối lần lượt với các điểm giữa của bộ pin mắc nối tiếp. Sau mạch BMS, hệ thống lấy ra đường nguồn 12V và GND để cấp cho các tầng ổn áp phía sau.

Từ nguồn 12V, hệ thống sử dụng IC L7809 để hạ và ổn định điện áp xuống mức 9V. Mức 9V này tiếp tục được đưa vào module hạ áp LM2596 để tạo ra nguồn 5V. Nguồn 5V sau LM2596 được dùng để cấp cho STM32F103C8T6, ESP32 và mạch phân áp của cảm biến FSR402. Việc sử dụng module LM2596 giúp nguồn 5V có khả năng cấp dòng tốt hơn, phù hợp với ESP32 vì module này tiêu thụ dòng cao khi kết nối Wi-Fi.

Quá trình tạo nguồn chính của hệ thống được mô tả như sau:



Hình 3.5: Sơ đồ tạo nguồn chính của hệ thống

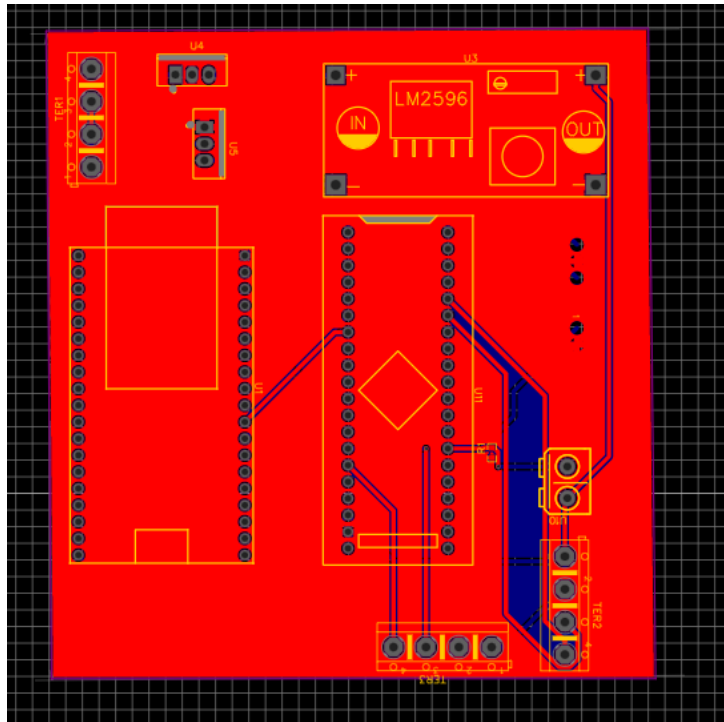
Bảng sau trình bày các kết nối chính trong khối nguồn:

Bảng 3.8: Bảng kết nối các khối trong mạch nguồn.

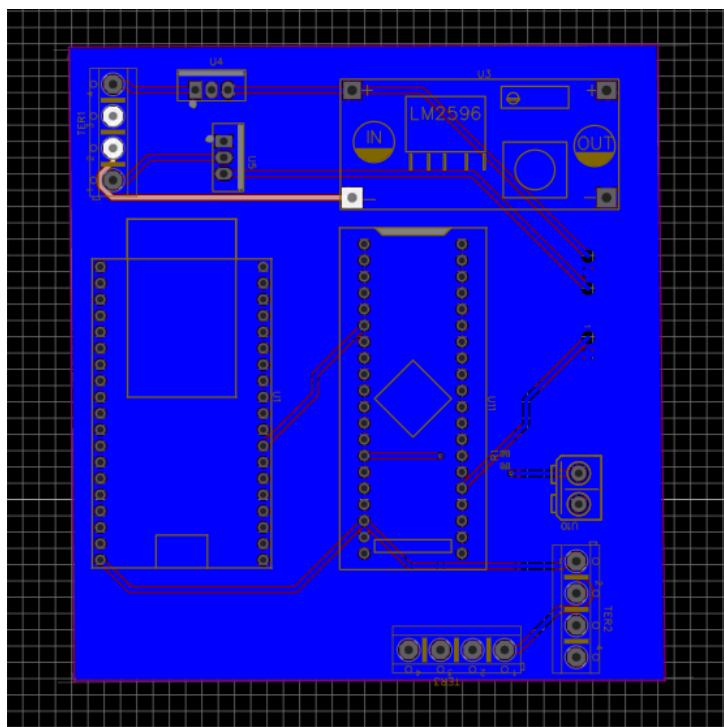
Khối nguồn	Chân/Điểm nối	Kết nối đến	Chức năng
Pin Li-ion 3 cell	Cực dương, cực âm và điểm giữa các cell	Các chân B0, B1, B2, B3 của BMS	Tạo nguồn đầu vào khoảng 12V
Mạch BMS	12V, GND	Đầu vào mạch ổn áp	Cấp nguồn chính cho hệ thống
IC L7809	IN	Đường nguồn 12V sau BMS	Nhận điện áp đầu vào để hạ xuống 9V
IC L7809	GND	GND hệ thống	Tạo mốc tham chiếu điện áp
IC L7809	OUT	Đầu vào module LM2596	Xuất điện áp ổn áp 9V
Module LM2596	IN+	Ngõ ra 9V từ L7809	Nhận nguồn đầu vào để hạ áp
Module LM2596	IN-	GND hệ thống	Mass đầu vào của module hạ áp
Module LM2596	OUT+	Đường nguồn 5V	Cấp nguồn cho STM32, ESP32 và FSR402
Module LM2596	OUT-	GND hệ thống	Mass đầu ra của nguồn 5V

Trong quá trình thiết kế, cần chú ý tất cả các khối phải dùng chung GND. GND chung giúp tín hiệu UART giữa STM32 và ESP32 hoạt động chính xác, đồng thời giúp tín hiệu ADC từ FSR402 có cùng mốc tham chiếu với STM32. Ngoài ra, do ESP32 tiêu thụ dòng khá lớn khi truyền Wi-Fi, nguồn 5V từ LM2596 cần được điều chỉnh ổn định trước khi cấp vào mạch. Khi đấu nối thực tế, nên kiểm tra điện áp đầu ra của LM2596 bằng đồng hồ đo trước khi cấp cho STM32 và ESP32 để tránh cấp sai điện áp gây hư hỏng linh kiện.

3.3.7 Sơ đồ nguyên lý phần cứng hoàn chỉnh



Hình 3.7: Thiết kế PCB mặt trước



Hình 3.8: Thiết kế PCB mặt sau

3.4 Thiết kế phần mềm hệ thống

Phần mềm hệ thống được thiết kế nhằm đảm bảo quá trình thu nhận dữ liệu cảm biến, xử lý tín hiệu, truyền dữ liệu, nhận dạng chuyển động và điều khiển đối tượng trong trò

chơi được thực hiện liên tục. Hệ thống phần mềm gồm các khối chính: chương trình xử lý cảm biến trên STM32, chương trình truyền dữ liệu trên ESP32, chương trình nhận dạng chuyển động bằng mô hình AI và chương trình điều khiển đối tượng trong Unity.

Trong hệ thống, STM32 đảm nhiệm việc đọc dữ liệu từ hai cảm biến MPU6050 và cảm biến lực FSR402. ESP32 nhận dữ liệu từ STM32 thông qua UART và gửi luồng dữ liệu UDP đến IP của máy tính. Chương trình Python đóng vai trò là một socket server đọc luồng dữ liệu UDP này. Unity đọc kết quả điều khiển và thực hiện thay đổi trạng thái của đối tượng trong trò chơi.

3.4.1 Thiết kế xử lý tín hiệu cảm biến MPU6050

Hai cảm biến MPU6050 được sử dụng để thu nhận chuyển động của tay người dùng. Một cảm biến được gắn tại phần cánh tay, cảm biến còn lại được gắn tại phần cẳng tay. Mỗi cảm biến cung cấp dữ liệu gia tốc ba trục và vận tốc góc ba trục. Từ các dữ liệu này, chương trình tính toán góc Roll, Pitch và các góc tương đối để mô tả tư thế của tay.

Trên STM32, hai cảm biến MPU6050 được đọc thông qua hai bus I2C riêng biệt. Khi hệ thống khởi động, chương trình tiến hành kiểm tra kết nối cảm biến bằng thanh ghi nhận dạng, sau đó cấu hình các thông số hoạt động cho MPU6050 như tốc độ lấy mẫu, thang đo gia tốc, thang đo con quay hồi chuyển và bộ lọc thông thấp số.

Các thông số cấu hình chính của MPU6050 được trình bày trong bảng sau:

Bảng 3.9: Các thông số xử lý tín hiệu MPU6050.

Thông số	Giá trị thiết kế	Ý nghĩa	Ghi chú
Tốc độ cập nhật MPU	100 ms	Cập nhật dữ liệu chuyển động	Tương đương 100 Hz
Thang đo gia tốc	$\pm 2g$	Đọc gia tốc theo ba trục	Phù hợp chuyển động tay
Thang đo gyro	$\pm 250^\circ/s$	Đọc vận tốc góc	Phù hợp chuyển động tay thông thường
Bộ lọc DLPF	Bật	Giảm nhiễu tần số cao từ cảm biến	Cấu hình trong MPU6050
Bộ lọc IIR	Áp dụng cho gia tốc và gyro	Làm mượt dữ liệu sau khi đọc	Thực hiện trên STM32
Bộ lọc bổ sung	Kết hợp gia tốc và gyro	Tính góc Roll, Pitch ổn định hơn	Giảm trôi góc theo thời gian

Sau khi đọc dữ liệu thô từ MPU6050, chương trình chuyển đổi giá trị raw sang đơn vị thực. Dữ liệu gia tốc được quy đổi sang đơn vị g, còn dữ liệu vận tốc góc được quy đổi sang đơn vị độ trên giây. Tiếp theo, chương trình áp dụng bộ lọc IIR để làm mượt tín hiệu:

$$y[n] = y[n-1] + \alpha(x[n] - y[n-1]) \quad (3.2)$$

Trong đó, $x[n]$ là giá trị mới đọc được, $y[n]$ là giá trị sau lọc và α là hệ số lọc. Hệ số α càng nhỏ thì tín hiệu càng mượt nhưng tốc độ đáp ứng chậm hơn.

Góc Roll và Pitch được tính dựa trên sự kết hợp giữa dữ liệu gia tốc và dữ liệu gyro. Gia tốc giúp xác định góc nghiêng dựa trên trọng lực, trong khi gyro giúp theo dõi sự thay đổi góc nhanh hơn. Bộ lọc bổ sung được sử dụng để kết hợp ưu điểm của hai nguồn dữ liệu này.

Khi hệ thống khởi động, người dùng giữ tay ở tư thế gốc để hệ thống lấy mốc ban đầu. Giá trị Roll và Pitch tại tư thế gốc được lưu lại để tính góc tương đối trong quá trình hoạt động:

$$Roll_{rel} = Roll - Roll_0 \quad (3.3)$$

$$Pitch_{rel} = Pitch - Pitch_0 \quad (3.4)$$

Đối với hệ thống dùng hai MPU6050, chương trình còn tính thêm góc tương đối giữa cẳng tay và cánh tay:

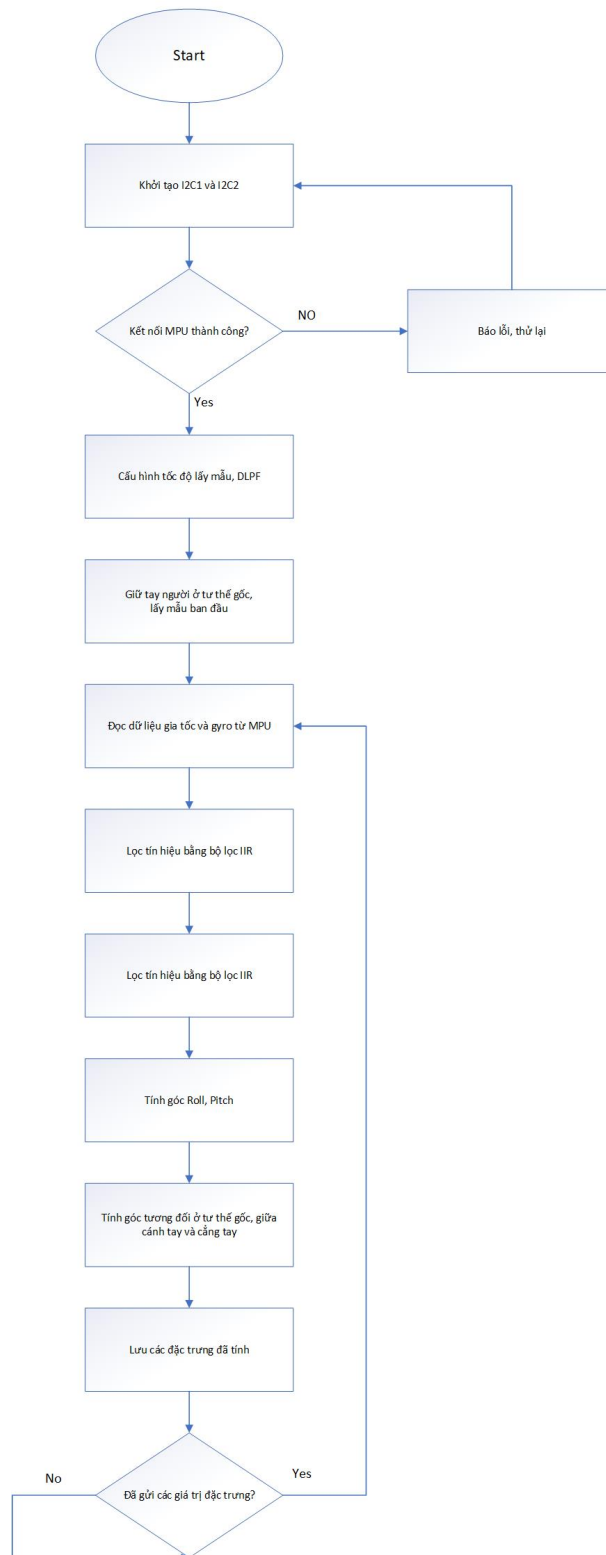
$$RelRoll = ForeRoll_{rel} - UpperRoll_{rel} \quad (3.5)$$

$$RelPitch = ForePitch_{rel} - UpperPitch_{rel} \quad (3.6)$$

Các giá trị này giúp mô hình AI nhận biết sự thay đổi tư thế của tay khi người dùng đưa tay lên, đưa tay xuống, nghiêng trái hoặc nghiêng phải.

Lưu đồ xử lý tín hiệu MPU6050 được đề xuất gồm các bước sau:

Lưu đồ xử lí tín hiệu của MPU



Hình 3.9: Lưu đồ xử lí tín hiệu MPU

3.4.2 Thiết kế xử lý tín hiệu cảm biến FSR402

Cảm biến FSR402 được sử dụng để nhận biết lực tác động của tay người dùng. Trong hệ thống, cảm biến này được dùng để xác định thao tác nhấn hoặc bóp tay, từ đó tạo thêm lệnh điều khiển đặc biệt trong trò chơi.

Tín hiệu từ FSR402 được đưa vào STM32 thông qua mạch phân áp và chân ADC. Khi người dùng không tác động lực, điện trở của FSR402 lớn nên điện áp tại điểm đo có giá trị thấp. Khi người dùng nhấn hoặc bóp tay, điện trở của cảm biến giảm xuống làm điện áp tại chân ADC thay đổi. STM32 đọc giá trị ADC này dưới dạng số trong khoảng từ 0 đến 4095.

Dữ liệu FSR thường có dao động nhỏ do lực tay không hoàn toàn ổn định, vì vậy chương trình sử dụng bộ lọc IIR để làm mượt giá trị đọc được. Công thức lọc có dạng:

$$FSR_f[n] = FSR_f[n-1] + \alpha(FSR_{raw}[n] - FSR_f[n-1]) \quad (3.7)$$

Trong đó, FSR_{raw} là giá trị ADC đọc trực tiếp, còn FSR_f là giá trị sau lọc. Giá trị sau lọc được sử dụng làm đặc trưng lực đưa vào mô hình AI và hỗ trợ xác định trạng thái có lực tác động.

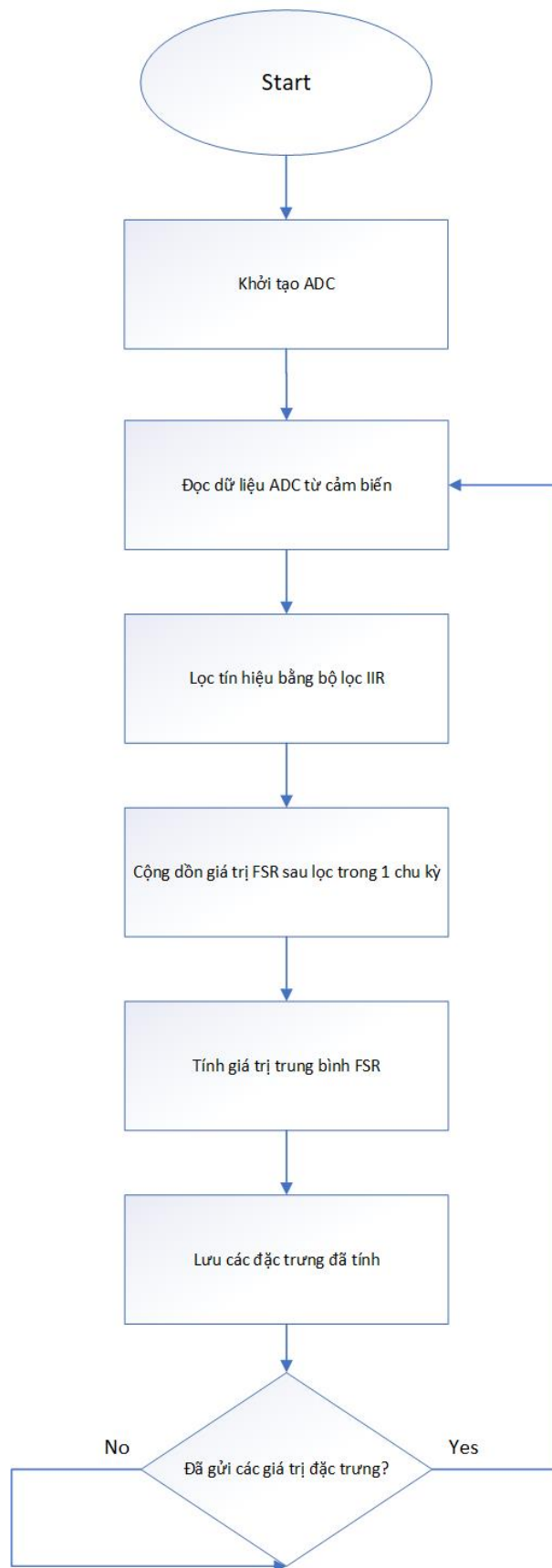
Các thông số xử lý FSR402 được trình bày trong bảng sau:

Bảng 3.10: Các thông số xử lý tín hiệu FSR402.

Thông số	Giá trị thiết kế	Ý nghĩa	Ghi chú
Chân đọc tín hiệu	PA2	Đọc điện áp từ mạch phân áp	ADC1_IN2
Độ phân giải ADC	12 bit	Giá trị đọc từ 0 đến 4095	Phụ thuộc cấu hình STM32
Chu kỳ đọc ADC	5 ms	Theo dõi lực tác động liên tục	Tương đương 200 Hz
Bộ lọc	IIR	Giảm dao động tín hiệu lực	Giữ tín hiệu ổn định hơn
Đặc trưng đầu ra	fsr_filtered	Giá trị lực sau lọc	Đưa vào mô hình AI

Lưu đồ xử lý tín hiệu FSR402 được đề xuất gồm:

Lưu đồ xử lí tín hiệu của FSR



Hình 3.10: Sơ đồ xử lí tín hiệu FSR

3.4.3 Thiết kế giao thức truyền dữ liệu UART

Sau khi STM32 đọc và xử lý dữ liệu từ các cảm biến, dữ liệu được đóng gói và truyền sang ESP32 thông qua giao tiếp UART. Giao tiếp UART được lựa chọn vì đơn giản, dễ kiểm tra bằng Serial Monitor và phù hợp để truyền dữ liệu dạng chuỗi giữa hai vi điều khiển.

Trong thiết kế này, STM32 gửi dữ liệu theo từng gói. Mỗi gói bắt đầu bằng chuỗi nhận dạng DATA, sau đó là các giá trị đặc trưng được phân tách bằng dấu phẩy. Cuối mỗi gói dữ liệu có ký tự xuống dòng để ESP32 xác định điểm kết thúc gói tin.

Cấu trúc gói dữ liệu được thiết kế như sau:

```
upper_roll_rel,upper_pitch_rel,  
fore_roll_rel,fore_pitch_rel,  
rel_roll,rel_pitch,  
upper_gx,upper_gy,upper_gz,  
fore_gx,fore_gy,fore_gz,  
fsr_filtered,packet_id, stm_send_ms
```

Trong đó, các giá trị góc và vận tốc góc có thể được nhân với 100 trước khi gửi để chuyển thành số nguyên. Khi ESP32 nhận dữ liệu, các giá trị này được chia lại cho 100 để khôi phục giá trị thực. Cách làm này giúp chuỗi dữ liệu ngắn hơn và hạn chế sai số khi truyền.

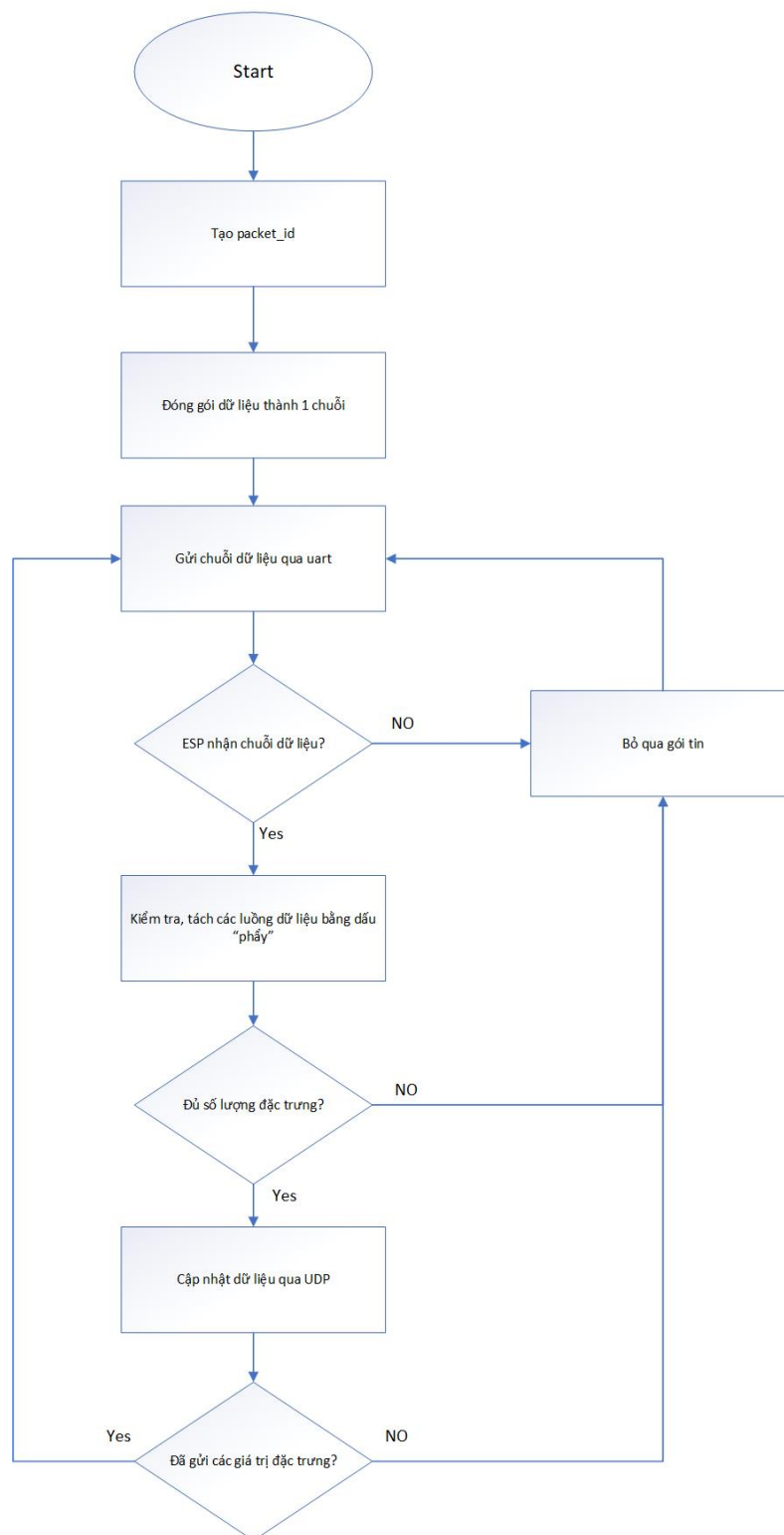
Bảng sau trình bày ý nghĩa các trường dữ liệu trong gói UART:

Bảng 3.11: Cấu trúc gói dữ liệu UART từ STM32 sang ESP32.

Trường dữ liệu	Nguồn dữ liệu	Ý nghĩa	Ghi chú
DATA	STM32	Ký hiệu bắt đầu gói tin	Dùng để kiểm tra gói hợp lệ
upper_roll_rel	MPU cánh tay	Góc Roll tương đối của cánh tay	Đơn vị độ
upper_pitch_rel	MPU cánh tay	Góc Pitch tương đối của cánh tay	Đơn vị độ
fore_roll_rel	MPU cẳng tay	Góc Roll tương đối của cẳng tay	Đơn vị độ
fore_pitch_rel	MPU cẳng tay	Góc Pitch tương đối của cẳng tay	Đơn vị độ
rel_roll	Hai MPU6050	Chênh lệch Roll giữa hai cảm biến	Mô tả quan hệ cánh tay và cẳng tay
rel_pitch	Hai MPU6050	Chênh lệch Pitch giữa hai cảm biến	Mô tả quan hệ cánh tay và cẳng tay
upper_gx, upper_gy, upper_gz	MPU cánh tay	Vận tốc góc của cánh tay	Đơn vị độ/giây
fore_gx, fore_gy, fore_gz	MPU cẳng tay	Vận tốc góc của cẳng tay	Đơn vị độ/giây
fsr_filtered	FSR402	Giá trị lực sau lọc	Giá trị ADC sau xử lý
packet_id	STM32	Số thứ tự gói tin	Kiểm tra dữ liệu mới
stm_send_ms	STM32	Thời điểm tạo gói dữ liệu	Hỗ trợ đánh giá thời gian

Lưu đồ truyền dữ liệu UART được đề xuất gồm các bước sau:

Lưu đồ truyền dữ liệu uart giữa
stm32 và esp32



Hình 3.11: Lưu đồ truyền, nhận dữ liệu uart

3.4.4 Thiết kế chương trình ESP32 nhận và truyền dữ liệu bằng UDP

ESP32 đóng vai trò trung gian giữa STM32 và chương trình xử lý trên máy tính. Chương trình trên ESP32 thực hiện hai nhiệm vụ chính: nhận dữ liệu từ STM32 thông qua UART và truyền dữ liệu hợp lệ đến máy tính bằng Wi-Fi thông qua giao thức UDP.

Khi khởi động, ESP32 tiến hành kết nối Wi-Fi và mở cổng UART2 để nhận dữ liệu từ STM32. Trong quá trình hoạt động, ESP32 liên tục kiểm tra dữ liệu truyền đến từ UART. Khi nhận được một dòng dữ liệu đầy đủ, chương trình kiểm tra định dạng gói tin, tách các trường dữ liệu và loại bỏ những gói sai định dạng.

Các giá trị góc và vận tốc góc được STM32 gửi ở dạng số nguyên đã nhân 100, do đó ESP32 có thể giữ nguyên dạng gói dữ liệu hoặc chuyển đổi lại về giá trị thực trước khi gửi đi. Giá trị FSR được giữ ở dạng số nguyên vì đây là giá trị ADC sau lọc. Sau khi kiểm tra dữ liệu thành công, ESP32 đóng gói dữ liệu và gửi đến địa chỉ IP của máy tính thông qua UDP.

Bảng sau trình bày nhiệm vụ chính của ESP32 trong hệ thống:

Bảng 3.12: Các tác vụ chính của chương trình ESP32 khi truyền dữ liệu bằng UDP.

Tác vụ	Dữ liệu vào	Dữ liệu ra	Ghi chú
Kết nối Wi-Fi	Tên mạng và mật khẩu	Trạng thái kết nối mạng	Cần cùng mạng với máy tính
Nhận UART	Chuỗi dữ liệu từ STM32	Dòng dữ liệu hoàn chỉnh	Kết thúc bằng ký tự xuống dòng
Tách dữ liệu	Chuỗi dữ liệu	Các giá trị đặc trưng	Kiểm tra số lượng trường dữ liệu
Đóng gói UDP	Dữ liệu cảm biến hợp lệ	Gói dữ liệu UDP	Gồm đặc trưng, FSR và mã gói
Gửi UDP	Gói dữ liệu UDP	Dữ liệu đến chương trình Python	Truyền đến IP và port đã cấu hình

3.4.5 Thiết kế luồng truyền dữ liệu UDP và xử lý nhận dạng

Trong hệ thống, UDP được sử dụng để truyền dữ liệu giữa ESP32, chương trình nhận dạng AI trên máy tính và phần mềm Unity. Cách thiết kế này giúp dữ liệu được truyền trực tiếp giữa các khối xử lý, không cần thông qua nền tảng lưu trữ trung gian. Nhờ đó,

hệ thống giảm được độ trễ và phù hợp hơn với yêu cầu điều khiển đối tượng trò chơi theo thời gian thực.

Luồng dữ liệu của hệ thống được tổ chức theo hướng truyền liên tục. STM32 đọc dữ liệu từ các cảm biến MPU và FSR, xử lý sơ bộ và gửi sang ESP32 bằng UART. ESP32 nhận dữ liệu, kiểm tra định dạng và truyền gói dữ liệu đến chương trình Python trên máy tính bằng UDP. Chương trình Python tiếp nhận dữ liệu cảm biến, đưa vào mô hình trí tuệ nhân tạo để phân loại chuyển động tay, sau đó gửi kết quả nhận dạng sang Unity cũng bằng UDP.

Bảng sau trình bày luồng truyền dữ liệu chính trong hệ thống:

Bảng 3.13: Luồng truyền dữ liệu UDP trong hệ thống.

Luồng dữ liệu	Phương thức truyền	Nội dung dữ liệu	Mục đích
STM32 đến ESP32	UART	Dữ liệu MPU, FSR và mã gói	Chuyển dữ liệu từ phần cứng cảm biến sang khối giao tiếp
ESP32 đến Python	UDP qua Wi-Fi	Gói dữ liệu cảm biến đã xử lý	Cung cấp dữ liệu đầu vào cho mô hình AI
Python đến Unity	UDP nội bộ trên máy tính	Nhãn chuyển động, hướng điều khiển và trạng thái thao tác đặc biệt	Điều khiển đối tượng trong trò chơi
Python ghi log	Lưu file trên máy tính	Kết quả nhận dạng và thông tin thời gian xử lý	Phục vụ đánh giá độ chính xác và độ trễ

Với thiết kế này, ESP32 không thực hiện lưu trữ dữ liệu mà chỉ đóng vai trò truyền dữ liệu không dây từ phần cứng đến máy tính. Chương trình Python là nơi tiếp nhận dữ liệu, xử lý bằng mô hình AI và tạo kết quả điều khiển. Unity nhận lệnh điều khiển từ Python để cập nhật trạng thái đối tượng trong trò chơi.

Do UDP không có cơ chế xác nhận gói tin như TCP, dữ liệu truyền trong hệ thống cần được đóng gói ngắn gọn và gửi theo chu kỳ phù hợp. Mỗi gói dữ liệu có thể kèm theo mã gói để chương trình nhận kiểm tra thứ tự dữ liệu và phát hiện trường hợp mất gói. Cách truyền này giúp hệ thống đạt tốc độ phản hồi nhanh, phù hợp với bài toán điều khiển trò chơi theo thời gian thực.

3.4.6 Thiết kế dữ liệu đầu vào và chương trình nhận dạng chuyển động cho mô hình AI

Chương trình nhận dạng chuyển động được xây dựng bằng Python. Chương trình này có nhiệm vụ nhận dữ liệu cảm biến từ ESP32 thông qua giao thức UDP, tiền xử lý dữ liệu, đưa dữ liệu vào mô hình AI để dự đoán chuyển động tay và gửi kết quả điều khiển sang Unity bằng UDP.

Dữ liệu đầu vào của mô hình AI gồm 13 đặc trưng được lấy từ hai cảm biến MPU6050 và cảm biến FSR402. Các đặc trưng này mô tả tư thế của cánh tay, tư thế của cẳng tay, sự chênh lệch giữa hai cảm biến và lực tác động của tay. Dựa trên các đặc trưng đó, mô hình AI phân loại chuyển động tay thành các lệnh điều khiển tương ứng như sang trái, sang phải, đi lên, đi xuống hoặc thực hiện thao tác đặc biệt trong trò chơi.

Bảng sau trình bày các đặc trưng đầu vào của mô hình AI:

Bảng 3.14: Các đặc trưng đầu vào của mô hình AI.

Nhóm đặc trưng	Tên đặc trưng	Ý nghĩa	Nguồn
Góc cánh tay	upper_roll_rel, upper_pitch_rel	Mô tả tư thế phần cánh tay	MPU cánh tay
Góc cẳng tay	fore_roll_rel, fore_pitch_rel	Mô tả tư thế phần cẳng tay	MPU cẳng tay
Góc tương đối	rel_roll, rel_pitch	Mô tả quan hệ giữa cánh tay và cẳng tay	Hai MPU6050
Gyro cánh tay	upper_gx, upper_gy, upper_gz	Mô tả vận tốc thay đổi của cánh tay	MPU cánh tay
Gyro cẳng tay	fore_gx, fore_gy, fore_gz	Mô tả vận tốc thay đổi của cẳng tay	MPU cẳng tay
Lực tác động	fsr_filtered	Nhận biết thao tác nhấn hoặc bóp tay	FSR402

Khi chương trình Python bắt đầu chạy, hệ thống yêu cầu người dùng giữ tay ở tư thế gốc trong một khoảng thời gian ngắn để thu thập dữ liệu nền. Các mẫu dữ liệu này được nhận từ ESP32 thông qua giao thức UDP và được tính giá trị trung bình để tạo baseline. Sau đó, mỗi mẫu dữ liệu mới sẽ được hiệu chỉnh dựa trên baseline trước khi đưa vào mô hình AI. Cách làm này giúp giảm sai lệch do vị trí đeo cảm biến, tư thế ban đầu của người dùng và sự thay đổi nhỏ trong quá trình lắp đặt hệ thống.

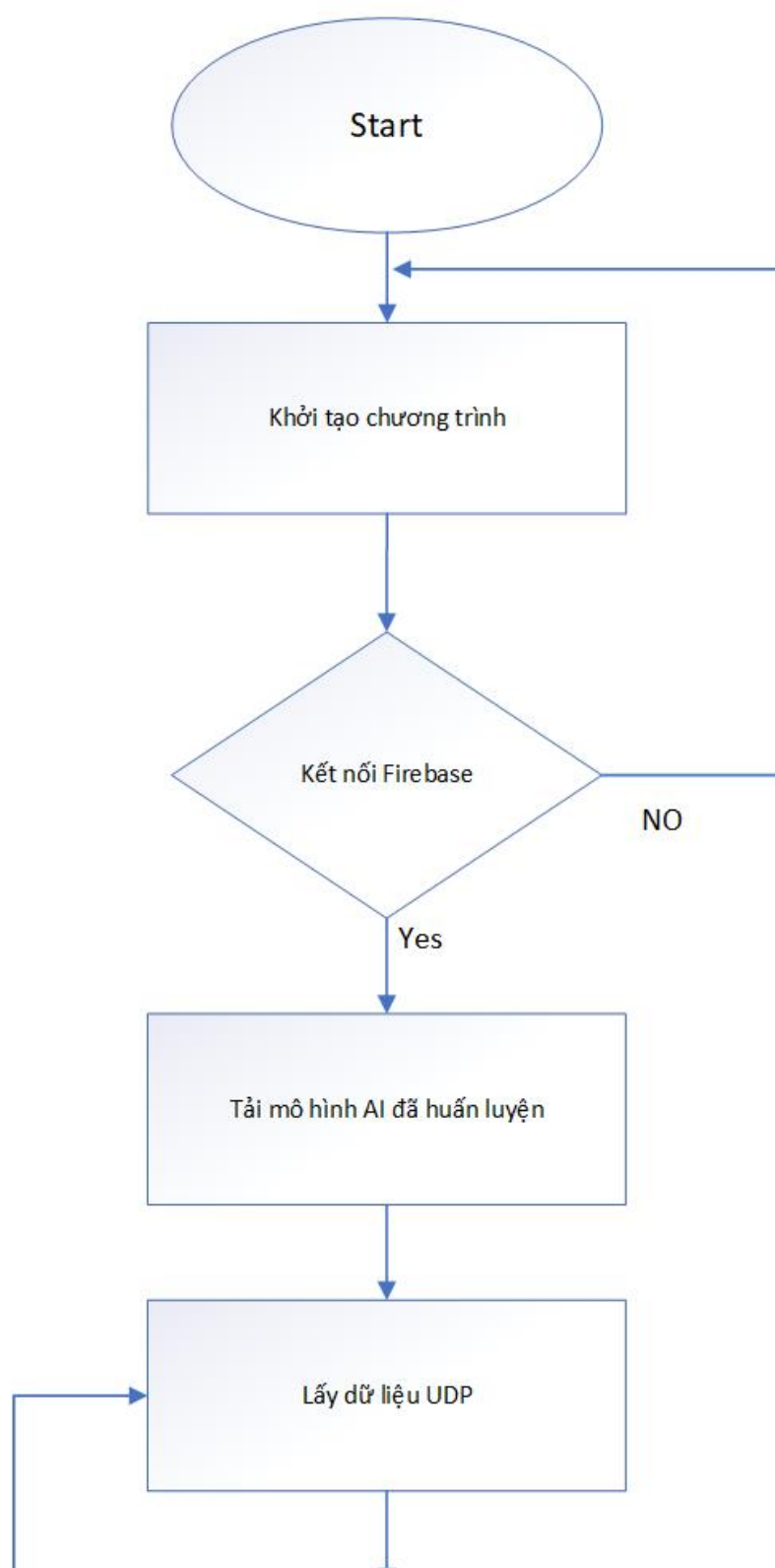
Mô hình AI sau khi dự đoán sẽ trả về nhãn chuyển động. Các nhãn đầu ra được thiết kế gồm nhóm không bắn và nhóm có bắn. Nhóm không bắn gồm: IDLE, LEFT, RIGHT, UP, DOWN. Nhóm có bắn gồm: IDLE_SHOOT, LEFT_SHOOT, RIGHT_SHOOT, UP_SHOOT, DOWN_SHOOT. Từ nhãn này, chương trình tách ra hai thông tin chính là hướng di chuyển và trạng thái bắn.

Bảng sau trình bày dữ liệu đầu ra của chương trình nhận dạng:
Bảng 3.15: Dữ liệu đầu ra của chương trình nhận dạng chuyển động.

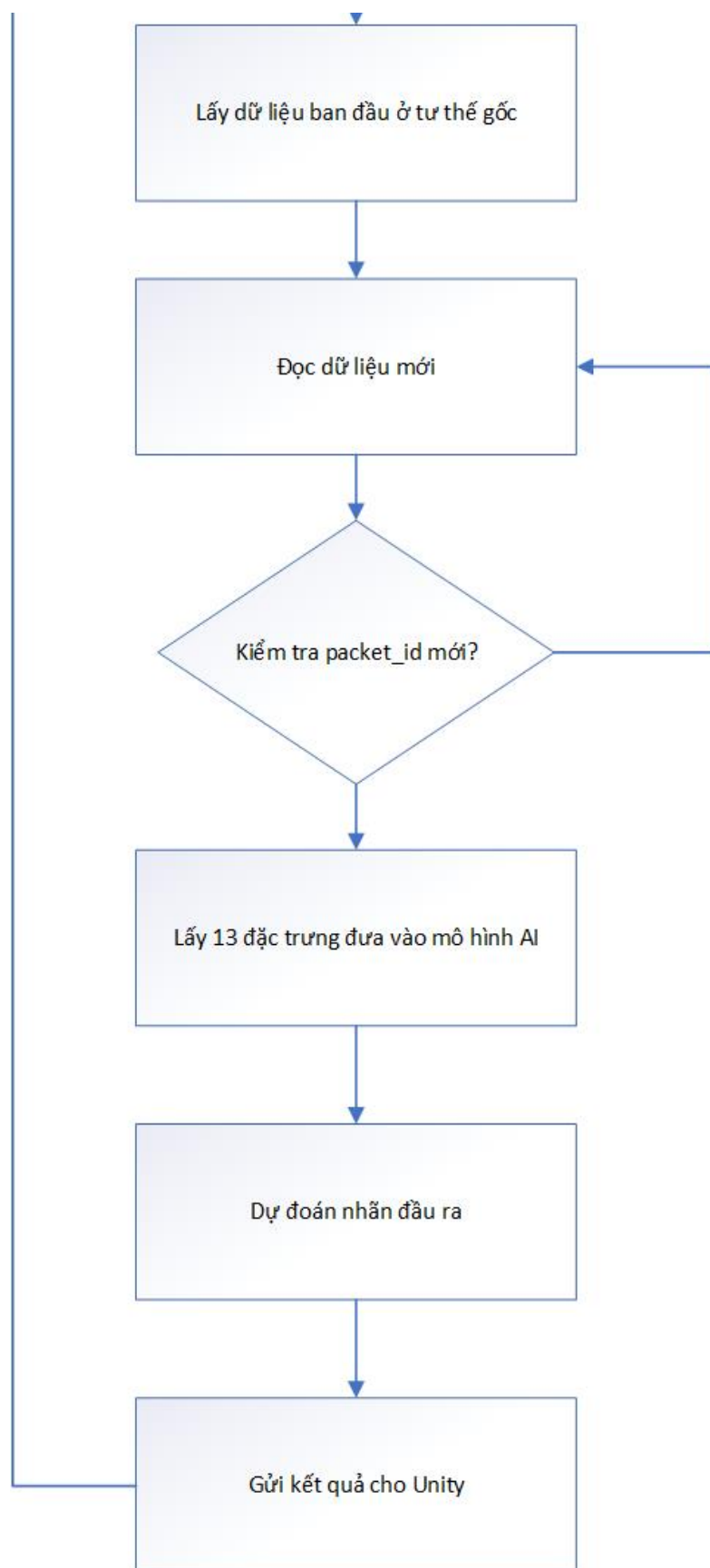
Đầu ra	Kiểu dữ liệu	Ý nghĩa	Ví dụ
label	Chuỗi	Nhãn đầy đủ do mô hình AI dự đoán	RIGHT_SHOOT
move	Chuỗi	Hướng di chuyển của đối tượng	RIGHT
shoot	Số nguyên	Trạng thái bắn hoặc kích hoạt lệnh	0 hoặc 1

Lưu đồ chương trình nhận dạng AI được đề xuất gồm các bước sau:

Lưu đồ nhận dạng bằng AI



Hình 3.12: Lưu đồ chương trình nhận dạng bằng AI - phần 1



Hình 3.13: Lưu đồ chương trình nhận dạng bằng AI - phần 2

3.4.7 Thiết kế chương trình điều khiển đối tượng trong Unity

Trong hệ thống, Unity đóng vai trò là môi trường mô phỏng thời gian thực, tiếp nhận kết quả nhận dạng cử chỉ tay và trực quan hóa chúng thông qua chuyển động của các đối tượng ảo. Quá trình giao tiếp được thực hiện thông qua giao thức mạng, trong đó Unity liên tục truy xuất dữ liệu JSON từ Firebase Realtime Database và tiến hành ánh xạ (mapping) thành các tập lệnh điều khiển tương ứng.

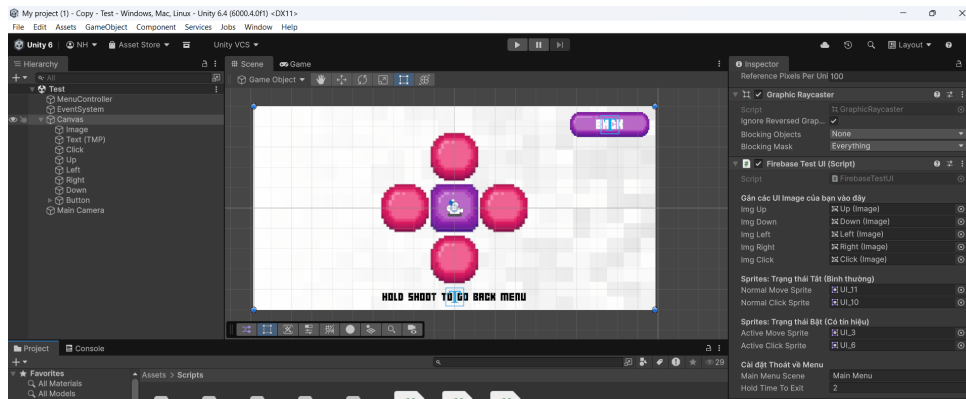
Dữ liệu đầu vào từ Firebase bao gồm hai trường thông tin cốt lõi: hướng di chuyển (move) và trạng thái kích hoạt (shoot).

- **Hướng di chuyển (move):** Quyết định vector vận tốc, dùng để thay đổi tọa độ của nhân vật (ví dụ: phi thuyền, giỏ hứng) hoặc điều hướng con trỏ ảo trên giao diện Menu người dùng.
- **Trạng thái kích hoạt (shoot):** Kích hoạt các sự kiện logic (event trigger) mang tính thời điểm như thao tác chọn nút (Click), bắn đạn, hoặc tạo lực đẩy vũ cánh.

Bảng 3.16 trình bày chi tiết quy tắc ánh xạ giữa chuỗi dữ liệu nhận dạng và hành động thực thi trong Unity:

Bảng 3.16: Ánh xạ kết quả nhận dạng sang hành động trong Unity.

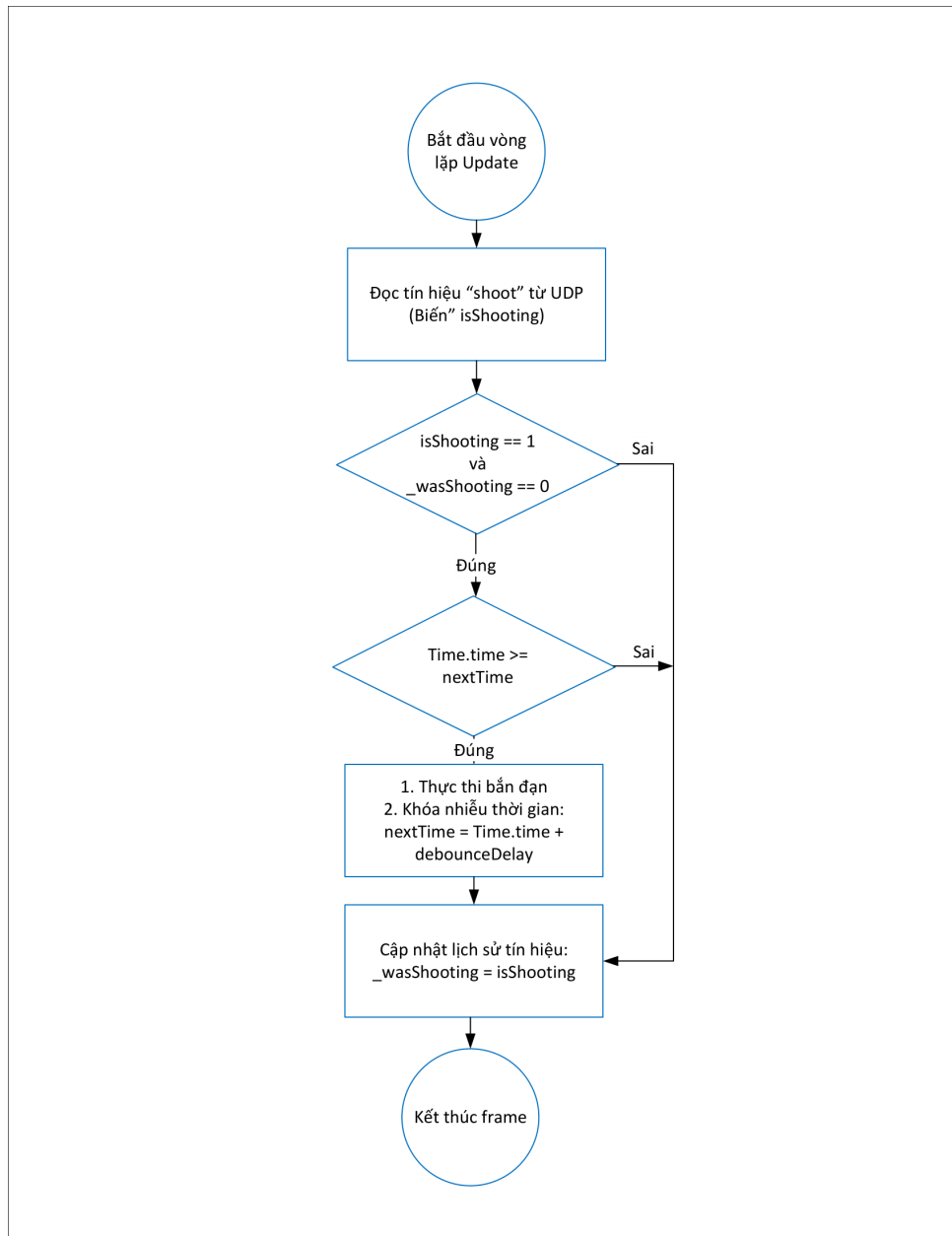
Giá trị nhận dạng	Lệnh điều khiển	Hành động trong Unity	Ghi chú
IDLE	Giữ nguyên	Đối tượng không đổi hướng	Trạng thái nghỉ
LEFT	Sang trái	Đối tượng di chuyển sang trái	Điều khiển theo trục ngang
RIGHT	Sang phải	Đối tượng di chuyển sang phải	Điều khiển theo trục ngang
UP	Đi lên	Đối tượng di chuyển lên	Điều khiển theo trục dọc
DOWN	Đi xuống	Đối tượng di chuyển xuống	Điều khiển theo trục dọc
shoot = 1	Bắn hoặc kích hoạt	Tạo đạn hoặc thực hiện thao tác đặc biệt	Dựa trên dữ liệu cảm biến áp trở và nhãn AI



Hình 3.14: Cấu hình các tham số điều khiển và ánh xạ tín hiệu trong giao diện Unity Inspector

Nhằm đảm bảo tính ổn định và mượt mà cho hệ thống mô phỏng, chương trình điều khiển trong Unity được thiết kế tích hợp các cơ chế xử lý tín hiệu chuyên sâu:

- **Cơ chế chống dội tín hiệu (Debounce):** Đối với các lệnh mang tính kích hoạt (như `shoot = 1`), hệ thống sử dụng các cờ logic (boolean flags) để lưu trữ trạng thái của khung hình trước đó. Lệnh chỉ được thực thi duy nhất một lần tại thời điểm chuyển giao trạng thái từ 0 lên 1, giúp ngăn chặn hiện tượng spam lệnh liên tục (chatter) khi người dùng giữ nguyên tư thế nắm tay.
- **Nội suy chuyển động và Giới hạn biên:** Để khắc phục độ trễ (latency) của quá trình truyền nhận dữ liệu mạng, vận tốc di chuyển của đối tượng được đồng bộ hóa với thời gian thực của từng khung hình (`Time.deltaTime`). Đồng thời, các thuật toán giới hạn không gian (`Mathf.Clamp`) được áp dụng nghiêm ngặt để đảm bảo đối tượng luôn nằm trong phạm vi hiển thị an toàn của camera, tránh các lỗi logic out-of-bounds trong quá trình vận hành.



Hình 3.15: Lưu đồ thuật toán cơ chế chống dội tín hiệu (Debounce) trong vòng lặp cập nhật trạng thái

Chương 4. KẾT QUẢ THỰC NGHIỆM

Chương này trình bày kết quả thực nghiệm của hệ thống nhận dạng chuyển động tay theo thời gian thực trong điều khiển đối tượng trò chơi. Nội dung chương bao gồm môi trường thực nghiệm, tiêu chí đánh giá, kết quả thi công mô hình phần cứng, kết quả truyền và xử lý dữ liệu, đánh giá mô hình AI, đồng thời nêu ra ưu điểm, hạn chế và hướng phát triển của hệ thống.

4.1 Môi trường thực nghiệm và các tiêu chí đánh giá

4.1.1 Môi trường và thiết bị thử nghiệm

Người dùng đeo các cảm biến MPU6050 và FSR402 trên tay, sau đó thực hiện các thao tác chuyển động tương ứng với các nhấn điều khiển. Dữ liệu cảm biến được STM32 thu thập, xử lý sơ bộ và gửi sang ESP32. ESP32 tiếp tục gửi dữ liệu qua UDP để chương trình nhận dạng chuyển động đọc về, xử lý bằng mô hình AI và cập nhật kết quả điều khiển cho Unity.

Các thiết bị sử dụng trong quá trình thực nghiệm gồm:

- Vi điều khiển STM32F103C8T6 dùng để đọc cảm biến và xử lý sơ bộ dữ liệu.
- ESP32 dùng để nhận dữ liệu từ STM32 và gửi dữ liệu thông qua UDP.
- Hai cảm biến MPU6050 dùng để thu nhận chuyển động của cánh tay và cẳng tay.
- Cảm biến lực FSR402 dùng để nhận biết thao tác nhấn hoặc bóp tay.
- Máy tính cá nhân dùng để chạy chương trình Python nhận dạng chuyển động và phần mềm Unity.
- Mô hình trò chơi trong Unity dùng để kiểm tra kết quả điều khiển đối tượng.

Môi trường thử nghiệm được bố trí sao cho người dùng có thể thực hiện chuyển động tay tự nhiên. Trước khi bắt đầu nhận dạng, người dùng giữ tay ở tư thế gốc trong một khoảng thời gian ngắn để hệ thống lấy dữ liệu nền. Sau đó, người dùng lần lượt thực hiện các thao tác điều khiển như giữ yên, đưa tay lên, đưa tay xuống, nghiêng sang trái, nghiêng sang phải và nhấn cảm biến lực để kích hoạt thao tác bắn.

Bảng sau trình bày các điều kiện thực nghiệm chính của hệ thống:

Bảng 4.1: Môi trường và thiết bị sử dụng trong quá trình thực nghiệm.

Thành phần	Thiết bị/Phần mềm	Vai trò	Ghi chú
Khối xử lý cảm biến	STM32F103C8T6	Đọc MPU6050, FSR402 và xử lý sơ bộ	Gửi dữ liệu qua UART
Khối giao tiếp	ESP32	Nhận UART và gửi dữ liệu	Kết nối Wi-Fi
Cảm biến chuyển động	2 MPU6050	Thu nhận góc và vận tốc góc	Gắn tại cánh tay và cẳng tay
Cảm biến lực	FSR402	Nhận biết thao tác nhấn/bóp	Dùng cho trạng thái bắn
Chương trình AI	Python	Nhận dạng chuyển động tay	Đọc và gửi dữ liệu
Mô phỏng	Unity	Điều khiển đối tượng trò chơi	Đọc kết quả điều khiển

4.1.2 Các tiêu chí đánh giá

Để đánh giá khả năng hoạt động của hệ thống, các tiêu chí được lựa chọn dựa trên mục tiêu của đề tài là nhận dạng chuyển động tay theo thời gian thực và điều khiển đối tượng trò chơi. Các tiêu chí đánh giá gồm độ ổn định của phần cứng, khả năng truyền dữ liệu, độ trễ toàn hệ thống, độ chính xác của mô hình AI và mức độ phản hồi của đối tượng trong Unity.

Các tiêu chí đánh giá chính được trình bày trong bảng sau:

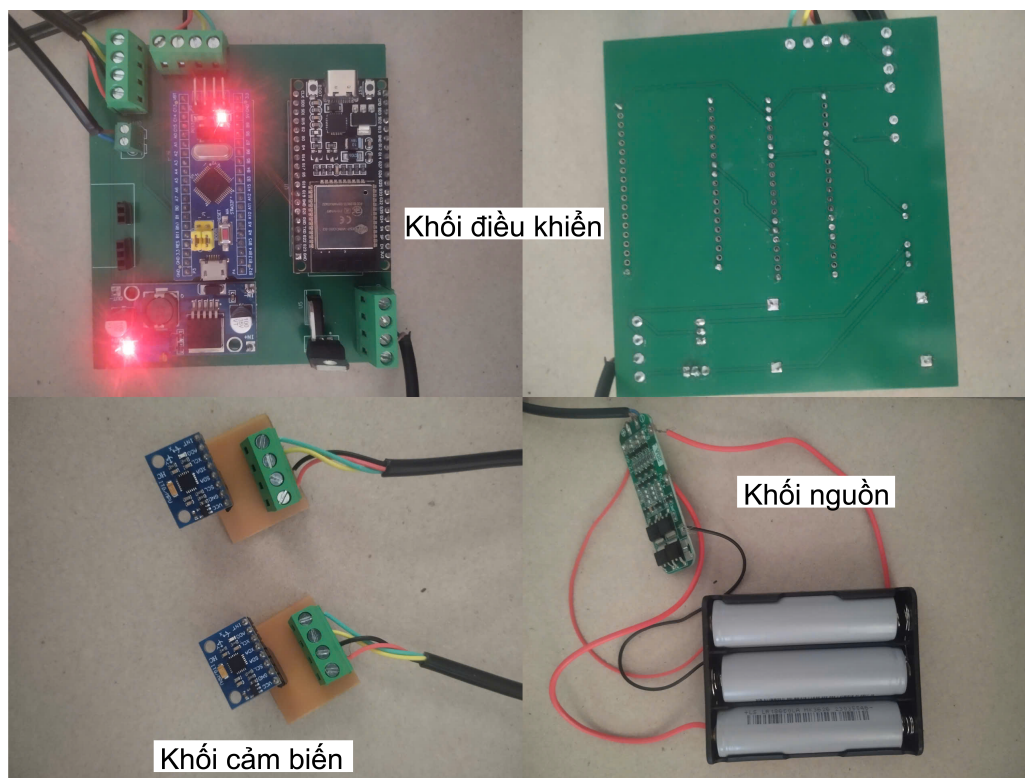
Bảng 4.2: Các tiêu chí đánh giá hệ thống.

Tiêu chí	Nội dung đánh giá	Cách đánh giá	Mục tiêu
Độ ổn định phần cứng	Khả năng hoạt động của cảm biến, STM32 và ESP32	Quan sát dữ liệu và quá trình chạy thử	Không mất kết nối, không reset bất thường
Độ ổn định dữ liệu	Mức dao động của dữ liệu MPU6050 và FSR402	Theo dõi giá trị cảm biến sau lọc	Dữ liệu thay đổi đúng theo thao tác tay
Khả năng truyền dữ liệu	Luồng truyền STM32, ESP32, UDP, Python và Unity	Kiểm tra dữ liệu cập nhật liên tục	Không sai định dạng, hạn chế mất gói
Độ trễ hệ thống	Thời gian từ lúc có dữ liệu cảm biến đến lúc Unity nhận lệnh	Đo từng giai đoạn xử lý	Phản hồi đủ nhanh cho điều khiển trò chơi
Độ chính xác AI	Khả năng phân loại đúng các chuyển động tay	Accuracy, report và confusion matrix	Phân biệt tốt các nhãn điều khiển
Khả năng điều khiển Unity	Mức độ phản hồi của đối tượng trò chơi	Quan sát trực tiếp trong mô phỏng	Di chuyển đúng hướng và bắn đúng thao tác

Trong các tiêu chí trên, độ trễ và độ chính xác nhận dạng là hai tiêu chí quan trọng nhất. Nếu hệ thống có độ chính xác cao nhưng độ trễ lớn, đối tượng trong trò chơi sẽ phản hồi chậm so với thao tác của người dùng. Ngược lại, nếu hệ thống phản hồi nhanh nhưng nhận dạng sai nhiều, đối tượng sẽ di chuyển không đúng ý muốn. Vì vậy, quá trình thực nghiệm cần đánh giá đồng thời cả tốc độ xử lý và độ chính xác nhận dạng.

4.2 Kết quả thi công mô hình phần cứng

Đầu tiên về mô hình phần cứng, mô hình sau khi lắp ráp có khả năng đọc dữ liệu từ hai cảm biến MPU6050 thông qua giao tiếp I2C, đọc dữ liệu FSR402 thông qua ADC và truyền chuỗi dữ liệu sang ESP32 thông qua UART. Các khối phần cứng được cấp nguồn chung và nối chung GND để đảm bảo tín hiệu truyền thông và tín hiệu cảm biến có cùng mốc tham chiếu.



Hình 4.1: Mô hình phần cứng sau khi thi công.

Bảng sau trình bày kết quả kiểm tra các khối phần cứng sau khi thi công:

Bảng 4.3: Kết quả kiểm tra các khối phần cứng.

Khối kiểm tra	Nội dung kiểm tra	Kết quả đạt được	Nhận xét
Khối nguồn	Kiểm tra điện áp cấp cho STM32, ESP32 và cảm biến	Các mức nguồn hoạt động ổn định	Đáp ứng yêu cầu cấp nguồn
MPU6050 thứ nhất	Đọc dữ liệu gia tốc và gyro qua I2C1	Có dữ liệu thay đổi khi chuyển động tay	Hoạt động đúng
MPU6050 thứ hai	Đọc dữ liệu gia tốc và gyro qua I2C2	Có dữ liệu thay đổi khi chuyển động tay	Hoạt động đúng
FSR402	Đọc giá trị ADC khi nhấn và thả tay	Giá trị tăng khi có lực tác động	Phù hợp nhận biết thao tác bấm
STM32	Đọc cảm biến, lọc dữ liệu và gửi UART	Gửi được chuỗi dữ liệu đúng định dạng	Hoạt động ổn định
ESP32	Nhận UART và gửi UDP	Dữ liệu được gửi qua UDP	Hoạt động đúng chức năng

Kết quả kiểm tra cho thấy các khối phần cứng cơ bản đáp ứng được yêu cầu thiết kế. Khi người dùng thay đổi tư thế tay, dữ liệu từ hai cảm biến MPU6050 thay đổi tương ứng. Khi người dùng nhấn hoặc bóp cảm biến FSR402, giá trị lực tăng rõ rệt so với trạng

thái không nhân. Điều này cho thấy phần cứng có khả năng thu nhận các thông tin cần thiết để phục vụ quá trình nhận dạng chuyển động.

Ngoài ra, việc thi công PCB giúp giảm số lượng dây nối ngoài, hạn chế tình trạng lỏng dây trong quá trình thử nghiệm và làm cho hệ thống gọn hơn so với khi lắp ráp trên breadboard. Các khối chức năng được bố trí theo sơ đồ nguyên lý đã thiết kế, giúp quá trình kiểm tra và sửa lỗi thuận tiện hơn.

4.3 Kết quả việc truyền, xử lý dữ liệu

4.3.1 Kết quả truyền dữ liệu từ STM32 cho đến UNITY

Dữ liệu sau khi được STM32 đọc và xử lý sẽ được đóng gói thành chuỗi UART. Chuỗi dữ liệu bắt đầu bằng ký hiệu DATA, theo sau là các đặc trưng từ hai cảm biến MPU6050, giá trị FSR402 sau lọc, mã gói dữ liệu và thông tin thời gian. Việc sử dụng định dạng chuỗi giúp ESP32 dễ dàng tách dữ liệu bằng dấu phẩy và kiểm tra tính hợp lệ của gói tin.

Cấu trúc truyền dữ liệu tổng quát của hệ thống được mô tả như sau:



Hình 4.2: Cấu trúc truyền dữ liệu của hệ thống.

Trong đó, STM32 là khối tạo dữ liệu cảm biến, ESP32 là khối truyền dữ liệu, UDP là khối truyền dữ liệu, Python AI là khối nhận dạng chuyển động và Unity là khối sử dụng kết quả điều khiển.

Bảng sau trình bày các giai đoạn truyền và xử lý dữ liệu trong hệ thống:

Bảng 4.4: Các giai đoạn truyền và xử lý dữ liệu trong hệ thống.

Giai đoạn	Dữ liệu vào	Dữ liệu ra	Chức năng
STM32 xử lý cảm biến	Dữ liệu MPU6050 và FSR402	Chuỗi DATA, . . .	Lọc, tính đặc trưng và đóng gói dữ liệu
ESP32 nhận UART	Chuỗi từ STM32	Các trường dữ liệu đã tách	Kiểm tra định dạng và chuyển đổi dữ liệu
ESP32 truyền UDP	Dữ liệu đã tách	Dữ liệu cảm biến	Cập nhật dữ liệu thời gian thực
Python AI xử lý	Gói dữ liệu mạng UDP	Nhãn chuyển động	Dự đoán bằng mô hình AI
Python ghi kết quả	Nhãn chuyển động	label, move, shoot	Cập nhật kết quả điều khiển
Unity điều khiển	move, shoot	Chuyển động đối tượng trò chơi	Hiển thị kết quả điều khiển

4.3.2 Kết quả truyền dữ liệu bằng UDP

Trong hệ thống thực nghiệm, giao thức UDP được sử dụng để truyền dữ liệu giữa ESP32, chương trình Python và Unity. Sau khi STM32 đọc và xử lý sơ bộ dữ liệu từ các cảm biến MPU và FSR, dữ liệu được gửi sang ESP32 thông qua UART. ESP32 tiếp tục truyền gói dữ liệu đến chương trình Python trên máy tính bằng Wi-Fi thông qua giao thức UDP.

Chương trình Python có nhiệm vụ nhận dữ liệu cảm biến, kiểm tra định dạng gói tin, đưa dữ liệu vào mô hình AI để nhận dạng chuyển động tay và gửi kết quả điều khiển sang Unity bằng UDP. Unity nhận kết quả này để cập nhật trạng thái điều khiển của đối tượng trong trò chơi.

Kết quả thực nghiệm cho thấy dữ liệu cảm biến có thể được truyền liên tục từ phần cứng đến chương trình nhận dạng và môi trường trò chơi. So với phương án sử dụng nền tảng trung gian, truyền dữ liệu bằng UDP giúp giảm độ trễ do dữ liệu được gửi trực tiếp trong mạng nội bộ. Nhờ đó, hệ thống có khả năng phản hồi nhanh hơn và phù hợp hơn với yêu cầu điều khiển đối tượng trò chơi theo thời gian thực.

4.3.3 Đánh giá thời gian xử lý và độ trễ hệ thống

Để đánh giá khả năng đáp ứng thời gian thực, hệ thống được đo thời gian ở các giai đoạn chính gồm: thời gian STM32 gửi dữ liệu sang ESP32, thời gian ESP32 truyền dữ liệu UDP đến chương trình Python, thời gian Python xử lý và chạy mô hình AI, thời gian Python gửi kết quả điều khiển sang Unity và thời gian Unity cập nhật trạng thái đối tượng.

Các thông số thời gian được ghi nhận trong quá trình chạy hệ thống gồm:

- Thời gian STM32 gửi dữ liệu cảm biến sang ESP32 thông qua UART.
- Thời gian ESP32 nhận dữ liệu UART và truyền gói UDP đến máy tính.
- Thời gian Python nhận dữ liệu, xử lý đặc trưng và chạy mô hình nhận dạng.
- Thời gian Python gửi kết quả điều khiển sang Unity bằng UDP.
- Thời gian Unity nhận lệnh và cập nhật chuyển động của đối tượng trong trò chơi.

Bảng sau trình bày kết quả đo thời gian xử lý trong hệ thống:

Bảng 4.5: Kết quả đo thời gian xử lý và truyền dữ liệu trong hệ thống UDP.

Giai đoạn đo	Giá trị trung bình	Giá trị lớn nhất	Nhận xét
STM32 gửi dữ liệu qua UART	7-9 ms	12 ms	Thời gian nhỏ, ổn định
ESP32 nhận UART và gửi UDP	2-4 ms	10-15 ms	Phụ thuộc chất lượng Wi-Fi nội bộ
Python nhận dữ liệu và chạy AI	0.5-2 ms	4 ms	Tăng khi thay đổi chuyển động nhanh hoặc dữ liệu dao động
Python gửi kết quả sang Unity bằng UDP	<0.5 ms	1 ms	Truyền nội bộ, độ trễ thấp
Unity nhận lệnh và cập nhật đối tượng	16-33 ms	50 ms	Phụ thuộc tốc độ khung hình và chu kỳ cập nhật
Tổng độ trễ hệ thống	30-46 ms	70-80 ms	Đáp ứng được yêu cầu điều khiển gần thời gian thực

Qua quá trình thử nghiệm, có thể nhận thấy thời gian truyền dữ liệu qua UART và UDP nhỏ hơn đáng kể so với thời gian xử lý trong chương trình Python. Nguyên nhân là do UART và UDP chỉ thực hiện truyền dữ liệu cục bộ trong hệ thống, trong khi chương

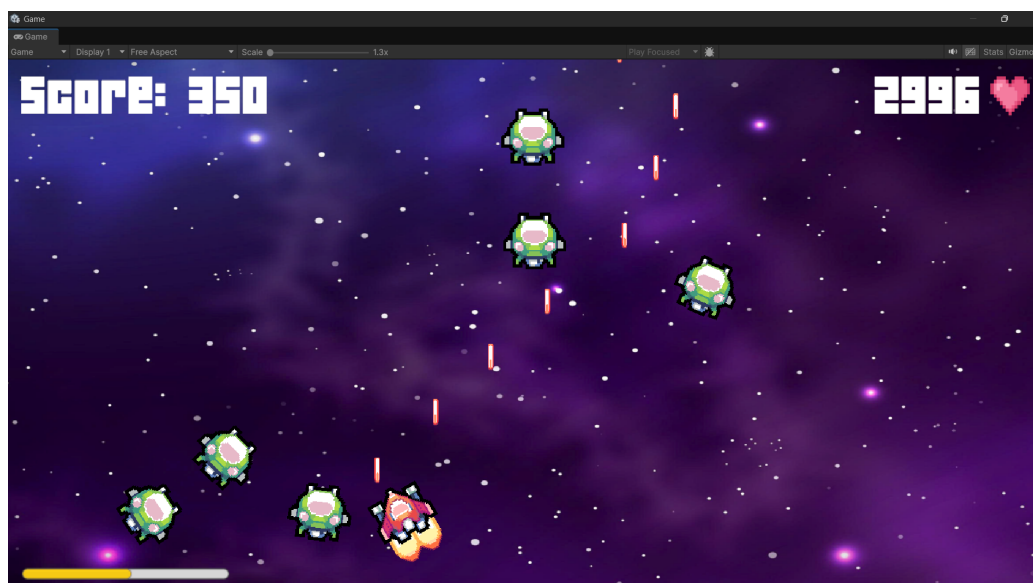
trình Python cần thực hiện nhận dữ liệu, tiền xử lý, đưa dữ liệu vào mô hình AI và tạo lệnh điều khiển tương ứng.

So với phương án sử dụng Firebase, truyền dữ liệu bằng UDP giúp giảm độ trễ vì dữ liệu không phải đi qua nền tảng lưu trữ trung gian. ESP32 gửi trực tiếp dữ liệu đến máy tính, sau đó chương trình Python gửi kết quả nhận dạng trực tiếp sang Unity. Cách truyền này phù hợp hơn với bài toán điều khiển đối tượng trò chơi theo thời gian thực.

4.3.4 Kết quả điều khiển đối tượng trong Unity

Sau khi chương trình Python nhận dạng chuyển động tay, kết quả được tách thành hai thông tin chính là move và shoot. Unity đọc hai giá trị này qua gói tin UDP và điều khiển đối tượng trong trò chơi theo hướng tương ứng.

Các trạng thái điều khiển được kiểm tra gồm: giữ nguyên, di chuyển sang trái, di chuyển sang phải, di chuyển lên, di chuyển xuống và bắn. Khi người dùng thực hiện chuyển động tay tương ứng, đối tượng trong Unity thay đổi trạng thái theo kết quả nhận dạng của mô hình AI.



Hình 4.3: Kết quả điều khiển đối tượng trong Unity.

Bảng sau trình bày kết quả kiểm tra các lệnh điều khiển trong Unity:

Kết quả thực nghiệm cho thấy hệ thống có thể điều khiển đối tượng trong Unity thông qua chuyển động tay. Khi dữ liệu cảm biến ổn định và mô hình AI nhận dạng đúng, đối tượng phản hồi đúng với thao tác của người dùng. Trong một số trường hợp, nếu người dùng thực hiện chuyển động quá nhanh hoặc đặt tay không đúng tư thế gốc, kết quả nhận

Bảng 4.6: Kết quả kiểm tra điều khiển đối tượng trong Unity.

Thao tác tay	Kết quả AI	Hành động trong Unity	Đánh giá
Giữ tay ở tư thế gốc	IDLE	Đối tượng giữ trạng thái	Đạt
Nghiêng tay sang trái	LEFT	Đối tượng di chuyển trái	Đạt
Nghiêng tay sang phải	RIGHT	Đối tượng di chuyển phải	Đạt
Đưa tay lên	UP	Đối tượng di chuyển lên	Đạt
Đưa tay xuống	DOWN	Đối tượng di chuyển xuống	Đạt
Nhấn hoặc bóp FSR402	SHOOT	Đối tượng thực hiện bắn	Đạt

dạng có thể dao động trong thời gian ngắn. Do đó, việc hiệu chuẩn ban đầu và giữ cố định cảm biến trên tay là yếu tố quan trọng để hệ thống hoạt động ổn định.

4.3.5 Đánh giá quá trình huấn luyện và kiểm thử mô hình AI

Trong đề tài, mô hình AI được sử dụng để nhận dạng hướng chuyển động tay của người dùng dựa trên dữ liệu thu được từ hai cảm biến MPU6050 và cảm biến lực FSR402. Tuy nhiên, trong quá trình thiết kế và tối ưu hệ thống, các nhãn có kết hợp thao tác bắn không được đưa trực tiếp vào mô hình phân loại. Thay vào đó, mô hình AI chỉ thực hiện nhận dạng hướng chuyển động tay, còn trạng thái bắn được xác định riêng dựa trên giá trị cảm biến FSR402.

Dữ liệu đầu vào của mô hình gồm các đặc trưng được trích từ hai cảm biến MPU6050 và cảm biến FSR402. Trong đó, các đặc trưng từ MPU6050 được sử dụng để mô tả tư thế và chuyển động của tay, còn giá trị FSR402 được dùng để hỗ trợ xác định thao tác nhấn hoặc bóp tay. Các nhãn đầu ra của mô hình AI gồm 5 trạng thái chính: IDLE, LEFT, RIGHT, UP và DOWN. Sau khi mô hình AI dự đoán hướng chuyển động, chương trình Python tiếp tục kiểm tra giá trị FSR để tạo trạng thái shoot.

Việc tách riêng hướng chuyển động và trạng thái bắn giúp giảm số lượng lớp cần phân loại của mô hình AI. Thay vì phải huấn luyện mô hình với 10 nhãn gồm cả các trạng thái có bắn, hệ thống chỉ cần huấn luyện mô hình với 5 nhãn chuyển động. Điều này giúp quá trình huấn luyện đơn giản hơn, giảm khả năng nhầm lẫn giữa các lớp và làm cho kết quả

điều khiển ổn định hơn trong quá trình chạy thực tế.

Đánh giá trong quá trình huấn luyện mô hình

Dữ liệu huấn luyện được thu thập khi người dùng thực hiện các thao tác tay tương ứng với 5 trạng thái điều khiển chính. Mỗi mẫu dữ liệu gồm các đặc trưng cảm biến và một nhãn chuyển động tương ứng. Sau khi thu thập, dữ liệu được lưu thành file CSV để phục vụ quá trình huấn luyện, kiểm tra và so sánh các mô hình AI.

Các nhãn được sử dụng trong quá trình huấn luyện gồm:

- IDLE: trạng thái giữ tay ở tư thế gốc hoặc không điều khiển.
- LEFT: trạng thái điều khiển đối tượng sang trái.
- RIGHT: trạng thái điều khiển đối tượng sang phải.
- UP: trạng thái điều khiển đối tượng đi lên.
- DOWN: trạng thái điều khiển đối tượng đi xuống.

Trong quá trình huấn luyện, tập dữ liệu được chia thành hai phần gồm tập train và tập test. Tập train được sử dụng để mô hình học mối quan hệ giữa dữ liệu cảm biến và nhãn chuyển động. Tập test được sử dụng để đánh giá khả năng dự đoán của mô hình trên dữ liệu chưa được dùng trực tiếp trong quá trình huấn luyện. Các mô hình được thử nghiệm gồm Random Forest, KNN và Gaussian Naive Bayes.

Hình sau trình bày dữ liệu huấn luyện được lưu dưới dạng file CSV. File dữ liệu gồm các cột đặc trưng cảm biến và cột nhãn tương ứng với từng thao tác tay.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	upper_roll	upper_pitch	fore_roll	fore_pitch	rel_roll	rel_pitch	upper_gx	upper_gy	upper_gz	fore_gx	fore_gy	fore_gz	action_label	
2	0.746	-0.961	-0.108	-0.996	-0.847	-0.031	0.581	-0.343	-0.625	0.855	-3.233	-0.321	IDLE	
3	0.826	-1.211	-0.388	-1.556	-1.207	-0.351	1.371	-4.423	-1.655	-0.435	-2.723	-1.971	IDLE	
4	0.686	-0.481	0.942	-9.316	0.253	-8.851	-0.409	3.457	-1.815	3.865	-40.503	-1.071	IDLE	
5	0.676	-0.371	1.322	-11.186	0.643	-10.831	1.291	-1.913	-1.475	-3.855	-7.683	1.619	IDLE	
6	1.656	-1.821	-0.388	-3.846	-2.037	-2.041	-0.469	3.627	1.215	0.215	9.997	0.149	IDLE	
7	1.846	-0.681	-0.358	-4.986	-2.207	-4.311	-0.439	4.087	-0.375	0.595	-3.053	0.099	IDLE	
8	1.596	-0.571	-0.168	-4.696	-1.757	-4.131	-0.009	0.837	-0.135	0.075	3.197	-0.561	IDLE	
9	1.606	-1.151	-0.158	-4.646	-1.767	-3.501	0.441	0.247	-0.485	-0.115	-2.783	-0.121	IDLE	
10	1.656	-1.311	-0.298	-4.156	-1.957	-2.851	-0.229	0.677	-0.925	-0.205	3.457	0.109	IDLE	
11	1.516	-0.921	-0.338	-3.726	-1.857	-2.821	0.221	-1.013	-1.315	0.055	-1.523	-0.291	IDLE	
12	1.526	-0.811	-0.238	-3.926	-1.767	-3.131	0.121	0.307	-1.155	0.495	-3.903	-0.261	IDLE	
13	1.406	-0.781	-0.298	-3.546	-1.707	-2.771	-0.329	1.687	-0.425	0.185	2.077	0.169	IDLE	
14	1.366	-0.631	-0.288	-3.406	-1.657	-2.781	-0.309	1.327	-0.695	-0.025	3.667	-0.071	IDLE	
15	1.336	-0.681	-0.358	-3.786	-1.697	-3.121	0.471	-1.053	-0.745	-0.235	-1.223	0.219	IDLE	
16	1.376	-0.801	-0.438	-4.146	-1.817	-3.361	0.281	0.737	-0.285	0.205	-0.023	0.189	IDLE	
17	1.396	-0.911	-0.398	-4.076	-1.797	-3.181	0.351	0.607	0.255	-0.065	0.747	-0.191	IDLE	
18	1.556	-1.001	-0.388	-4.276	-1.937	-3.291	0.031	1.897	0.085	0.165	-0.863	0.609	IDLE	
19	1.296	-0.401	-0.238	-4.336	-1.527	-3.951	-0.359	1.307	0.025	-0.005	-3.683	1.709	IDLE	
20	1.246	-0.261	-0.158	-3.136	-1.397	-2.881	-0.449	2.767	-2.195	0.965	8.807	-1.581	IDLE	
21	0.496	0.899	0.182	-0.246	-0.317	-1.151	-1.359	0.457	0.095	-1.345	-2.963	-0.261	IDLE	
22	0.526	0.579	0.042	-2.566	-0.477	-3.141	0.301	-0.993	0.425	-0.145	-6.723	0.559	IDLE	
23	0.446	0.849	0.072	-0.656	-0.367	-1.511	-0.949	1.517	-1.545	0.645	4.617	-0.311	IDLE	
24	0.126	1.379	0.102	-0.206	-0.017	-1.591	0.121	-1.263	1.355	-0.675	-6.113	1.059	IDLE	
25	0.186	1.239	0.162	-0.156	-0.027	-1.391	-0.779	2.607	-2.555	0.675	9.897	-1.621	IDLE	
26	-0.104	1.629	0.132	-1.596	0.233	-3.221	1.361	-4.963	2.185	-0.325	-15.153	3.379	IDLE	
27	0.186	1.049	0.302	0.894	0.123	-0.161	-1.269	1.347	-3.225	0.055	11.847	-2.231	IDLE	
28	0.046	1.699	0.292	0.864	0.243	-0.841	-0.719	-0.263	2.825	-1.565	-13.553	4.229	IDLE	
29	-0.234	1.579	0.152	1.544	0.383	-0.041	-0.939	1.517	-2.465	0.135	6.057	-2.081	IDLE	
30	-0.354	1.819	0.172	1.234	0.533	-0.581	0.441	1.287	-3.255	0.405	8.127	-2.231	IDLE	

Hình 4.4: Dữ liệu huấn luyện mô hình AI dưới dạng file CSV.

Bảng sau trình bày cấu trúc dữ liệu sử dụng trong quá trình huấn luyện mô hình AI:

Bảng 4.7: Cấu trúc dữ liệu sử dụng trong quá trình huấn luyện mô hình AI.

Thành phần dữ liệu	Nội dung	Vai trò	Ghi chú
Đặc trưng MPU6050	Góc, góc tương đối và vận tốc góc	Nhận dạng hướng chuyển động tay	Lấy từ hai cảm biến MPU6050
Đặc trưng FSR402	Giá trị lực sau lọc	Xác định trạng thái nhấn hoặc bóp	Dùng riêng cho trạng thái bắn
Nhãn dữ liệu	5 trạng thái chuyển động	Là đầu ra cần dự đoán của mô hình AI	Không gồm các nhãn SHOOT
Tập train	Phần dữ liệu dùng để huấn luyện	Giúp mô hình học quy luật dữ liệu	Chiếm phần lớn tập dữ liệu
Tập test	Phần dữ liệu dùng để kiểm tra	Đánh giá khả năng dự đoán	Không dùng trực tiếp khi huấn luyện
Mô hình AI	Random Forest, KNN, Gaussian Naive Bayes	Phân loại hướng chuyển động tay	So sánh để chọn mô hình phù hợp

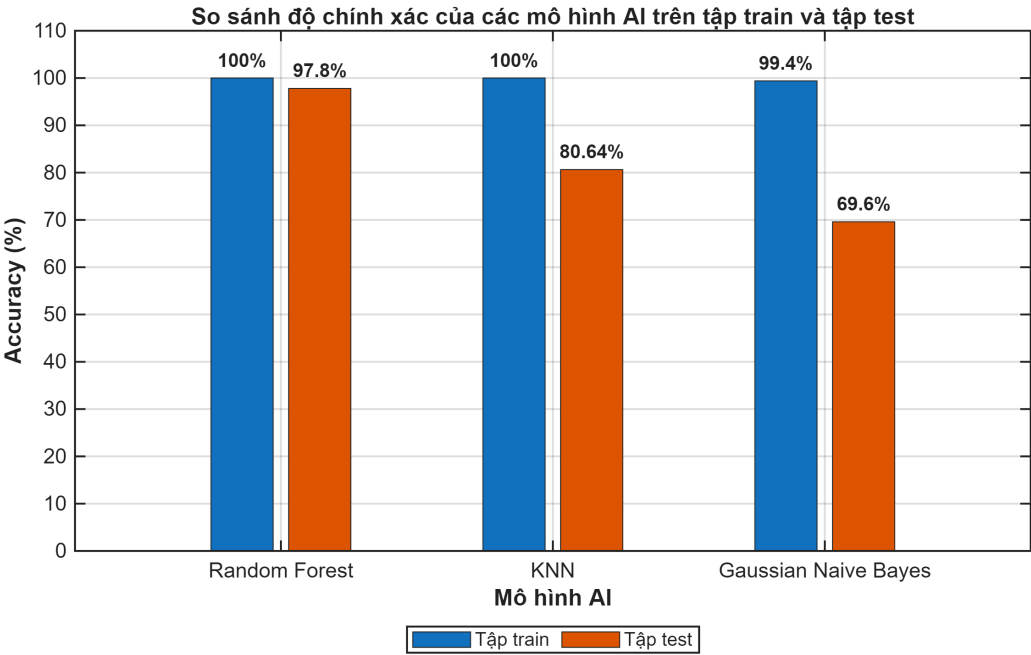
Sau khi huấn luyện, các mô hình được so sánh dựa trên độ chính xác và khả năng phân loại từng lớp. Các chỉ số được sử dụng gồm Accuracy, Precision, Recall, F1-score

và ma trận nhầm lẫn. Trong đó, Accuracy cho biết tỷ lệ dự đoán đúng tổng thể, Precision và Recall cho biết khả năng nhận dạng đúng từng lớp, còn ma trận nhầm lẫn giúp quan sát các lớp dễ bị nhầm lẫn với nhau.

Bảng 4.8: Kết quả so sánh độ chính xác của các mô hình AI trên tập train và tập test.

Mô hình	Accuracy trên tập train	Accuracy trên tập test	Nhận xét
Random Forest	100%	97.8%	Độ chính xác cao, phân loại ổn định
KNN	100%	80.64%	Dễ triển khai nhưng phụ thuộc khoảng cách dữ liệu
Gaussian Naive Bayes	99.4%	69.6%	Tốc độ nhanh nhưng độ chính xác có thể thấp hơn

Hình sau trình bày biểu đồ so sánh độ chính xác của các mô hình AI trên tập train và tập test. Biểu đồ này giúp quan sát trực quan sự chênh lệch giữa khả năng học trên dữ liệu huấn luyện và khả năng dự đoán trên dữ liệu kiểm tra.

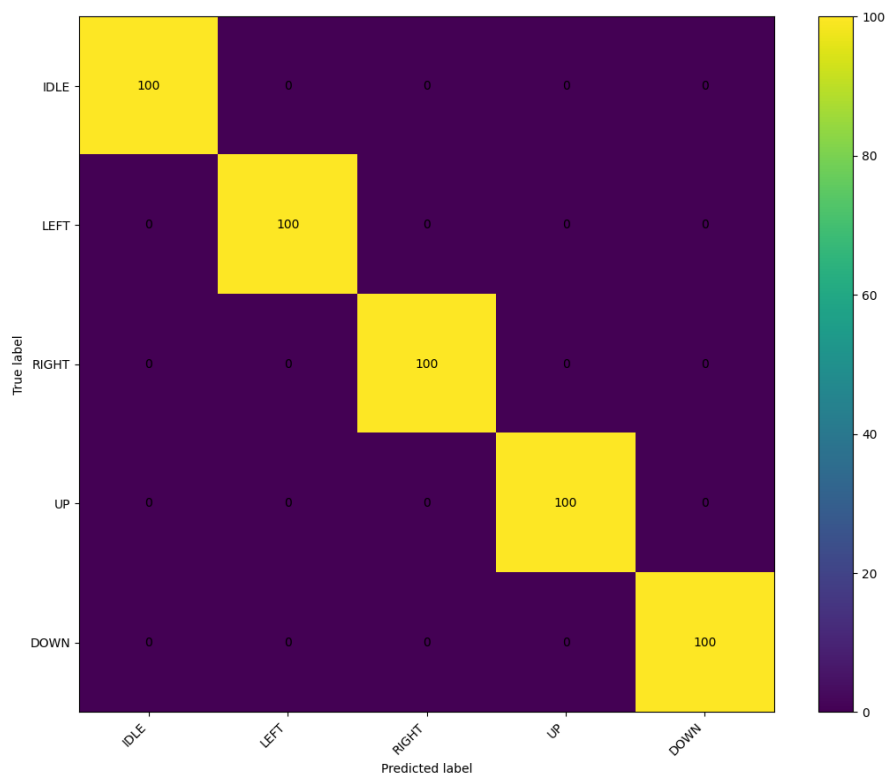


Hình 4.5: Biểu đồ so sánh Accuracy của các mô hình AI trên tập train và tập test.

Trong các mô hình được thử nghiệm, Random Forest được lựa chọn làm mô hình chính cho hệ thống. Nguyên nhân là mô hình này có khả năng xử lý tốt dữ liệu có nhiều đặc trưng, ít nhạy với nhiễu và cho kết quả phân loại ổn định. Ngoài ra, Random Forest phù hợp với dữ liệu cảm biến vì mô hình có thể học được mối quan hệ phi tuyến giữa

các đặc trưng góc, vận tốc góc và hướng chuyển động tay.

Ma trận nhầm lẫn của mô hình Random Forest được sử dụng để đánh giá chi tiết khả năng phân loại từng nhãn trên tập test.



Hình 4.6: Ma trận nhầm lẫn của mô hình Random Forest trên tập test.

Từ các dữ liệu đặc trưng, chương trình đọc dữ liệu từ file CSV, chia dữ liệu thành tập train và tập test, huấn luyện các mô hình, sau đó xuất ra kết quả đánh giá dựa trên các đặc trưng đó mà độ quan trọng đặc trưng khác.

Dựa trên kết quả đánh giá mức độ quan trọng của các đặc trưng, có thể thấy các đặc trưng liên quan đến góc Pitch và Roll có ảnh hưởng lớn nhất đến quá trình nhận dạng chuyển động. Trong đó, `upper_pitch_rel` và `fore_pitch_rel` có giá trị quan trọng cao nhất, cho thấy chuyển động lên xuống của cánh tay và cẳng tay đóng vai trò chính trong phân loại. Các đặc trưng `rel_roll`, `fore_roll_rel` và `upper_roll_rel` cũng có ảnh hưởng đáng kể, hỗ trợ nhận dạng các chuyển động nghiêng trái, nghiêng phải. Ngược lại, các đặc trưng vận tốc góc như `upper_gx`, `upper_gz` có mức ảnh hưởng thấp hơn. Điều này cho thấy mô hình chủ yếu dựa vào thông tin góc tương đối của tay để phân loại hướng chuyển động, còn các đặc trưng gyro chỉ đóng vai trò bổ trợ.

	A	B	C	D
1	feature	importance		
2	upper_pitch_rel	0.20613206823509192		
3	fore_pitch_rel	0.18840799212200443		
4	rel_roll	0.16749946460630608		
5	fore_roll_rel	0.1543780653226876		
6	upper_roll_rel	0.08622642792934311		
7	rel_pitch	0.048399645230736206		
8	fore_gz	0.045740589521139144		
9	upper_gy	0.03961125374799036		
10	fore_gx	0.02409412541128661		
11	fore_gy	0.023533744819731316		
12	upper_gx	0.013958641981915156		
13	upper_gz	0.0020179810717680467		
14				

Hình 4.7: Độ quan trọng của các đặc trưng

Hình sau trình bày kết quả đánh giá chi tiết từng lớp của mô hình Random Forest:

class	precision	recall	f1_score	support
IDLE	1	1	1	100
LEFT	1	1	1	100
RIGHT	1	1	1	100
UP	1	1	1	100
DOWN	1	1	1	100

Hình 4.8: Kết quả đánh giá chi tiết từng lớp của mô hình Random Forest.

Kết quả huấn luyện cho thấy mô hình AI có thể học được mối quan hệ giữa dữ liệu cảm biến và các trạng thái chuyển động tay. Các đặc trưng góc từ hai cảm biến MPU6050 đóng vai trò quan trọng trong việc phân biệt các hướng di chuyển. Việc tách riêng trạng

thái bắn bằng FSR402 giúp mô hình giảm số lớp phân loại và tập trung vào nhiệm vụ nhận dạng hướng chuyển động.

Đánh giá trong quá trình kiểm thử thực tế

Sau khi huấn luyện, mô hình Random Forest được lưu thành file và được chương trình Python sử dụng trong quá trình chạy thực tế. Ở giai đoạn kiểm thử, dữ liệu không còn được đọc trực tiếp từ file CSV mà được nhận từ ESP32 thông qua giao thức UDP. ESP32 nhận dữ liệu cảm biến từ STM32 qua UART, sau đó gửi gói dữ liệu đến chương trình Python trên máy tính.

Trong quá trình chạy thực tế, chương trình Python thực hiện hai nhiệm vụ chính. Nhiệm vụ thứ nhất là đưa các đặc trưng chuyển động vào mô hình AI để nhận dạng một trong năm nhãn IDLE, LEFT, RIGHT, UP hoặc DOWN. Nhiệm vụ thứ hai là kiểm tra giá trị `fsr_filtered`. Nếu giá trị FSR vượt qua ngưỡng đặt trước, chương trình đặt `shoot = 1`; ngược lại, chương trình đặt `shoot = 0`. Sau đó, kết quả `move` và `shoot` được gửi sang Unity bằng giao thức UDP để điều khiển đối tượng trong trò chơi.

Quy trình kiểm thử thực tế được thực hiện như sau: người dùng đeo cảm biến lên tay, giữ tay ở tư thế gốc để lấy baseline, sau đó thực hiện từng thao tác điều khiển. Chương trình Python nhận các gói dữ liệu UDP, kiểm tra mã gói `packet_id` để tránh xử lý lặp dữ liệu cũ. Với mỗi gói dữ liệu mới, chương trình chọn 13 đặc trưng đầu vào, tiền xử lý dữ liệu, dự đoán hướng chuyển động và xác định trạng thái bắn dựa trên FSR402.

Hình sau trình bày dữ liệu cảm biến được ESP32 gửi đến chương trình Python thông qua UDP trong quá trình kiểm thử thực tế.

```

10:44:51.279 ->
10:44:51.281 -> ===== UART DA NHAN =====
10:44:51.281 -> Lan nhan UART #1219
10:44:51.281 -> Thoi gian nhan 1 goi UART: 2 ms
10:44:51.281 -> Raw line: DATA,-72,-72,469,-44,542,28,-69,-95,152,-113,53,-101,2001,4140,430700
10:44:51.333 -> ----- Gia tri goc nhan tu STM32 -----
10:44:51.333 -> packet_id      = 4140
10:44:51.333 -> stm_send_ms     = 430700
10:44:51.333 -> ----- Goc tuong doi, da chia /100 -----
10:44:51.333 -> upper_roll_rel  = -0.72
10:44:51.333 -> upper_pitch_rel = -0.72
10:44:51.333 -> fore_roll_rel   = 4.69
10:44:51.333 -> fore_pitch_rel  = -0.44
10:44:51.333 -> rel_roll        = 5.42
10:44:51.333 -> rel_pitch       = 0.28
10:44:51.333 -> ----- Gyro, da chia /100 -----
10:44:51.333 -> upper_gx = -0.69
10:44:51.333 -> upper_gy = -0.95
10:44:51.333 -> upper_gz = 1.52
10:44:51.333 -> fore_gx  = -1.13
10:44:51.333 -> fore_gy  = 0.53
10:44:51.333 -> fore_gz  = -1.01
10:44:51.333 -> ----- FSR -----
10:44:51.333 -> fsr_filtered = 2001
10:44:51.333 -> =====
10:44:51.335 -> Gui UDP #1218 den 192.168.1.16:4210 | ok=1
10:44:51.335 ->

```

Hình 4.9: Dữ liệu cảm biến gửi qua UDP trong ESP32.

Dữ liệu sẽ được đưa vào mô hình AI để dự đoán chuyển động:

```

Predict #7: UP | move=UP | shoot=0 | fsr=0.0 | packet_id=28 | python_model_ms=0.496 ms | python_upload_firebase_ms=264.141 ms
Predict #9: UP | move=UP | shoot=0 | fsr=0.0 | packet_id=30 | python_model_ms=0.492 ms | python_upload_firebase_ms=269.848 ms
Predict #11: UP | move=UP | shoot=0 | fsr=0.0 | packet_id=32 | python_model_ms=0.647 ms | python_upload_firebase_ms=267.198 ms
Predict #13: UP | move=UP | shoot=0 | fsr=0.0 | packet_id=34 | python_model_ms=0.512 ms | python_upload_firebase_ms=261.981 ms
Predict #15: RIGHT | move=RIGHT | shoot=0 | fsr=0.0 | packet_id=36 | python_model_ms=0.403 ms | python_upload_firebase_ms=269.854 ms
Predict #17: RIGHT | move=RIGHT | shoot=0 | fsr=0.0 | packet_id=39 | python_model_ms=0.278 ms | python_upload_firebase_ms=272.122 ms
Predict #19: RIGHT | move=RIGHT | shoot=0 | fsr=0.0 | packet_id=41 | python_model_ms=0.466 ms | python_upload_firebase_ms=269.261 ms
Predict #21: RIGHT | move=RIGHT | shoot=0 | fsr=0.0 | packet_id=44 | python_model_ms=0.456 ms | python_upload_firebase_ms=263.019 ms
Predict #23: RIGHT | move=RIGHT | shoot=0 | fsr=0.0 | packet_id=46 | python_model_ms=0.219 ms | python_upload_firebase_ms=265.538 ms

```

Hình 4.10: Chương trình Python dự đoán hướng chuyển động trong quá trình kiểm thử thực tế.

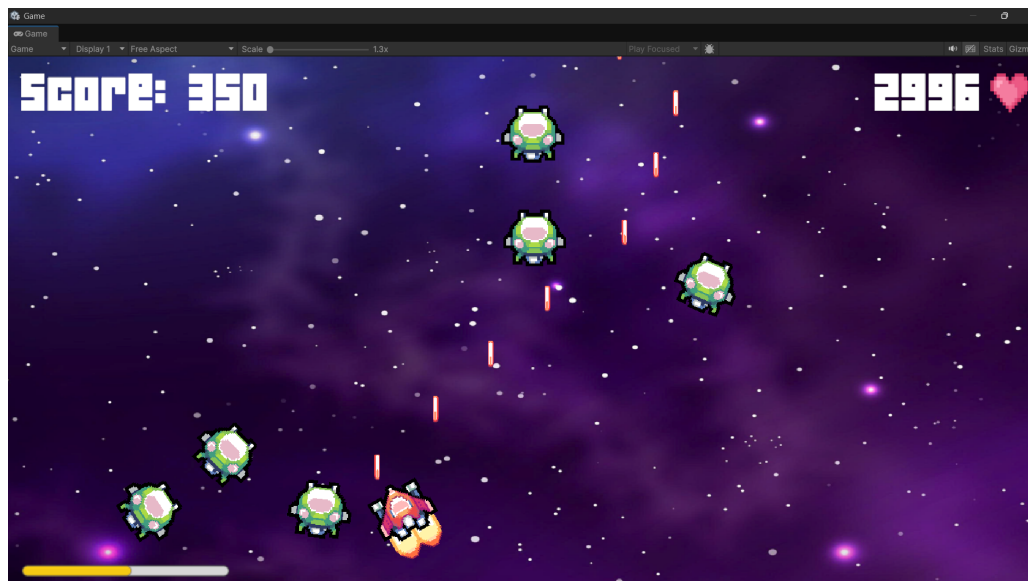
Sau khi mô hình AI dự đoán xong, kết quả điều khiển được ghi ra các kết quả dưới dạng các trường label, move và shoot. Trong đó, label và move thể hiện hướng chuyển động do mô hình AI dự đoán, còn shoot thể hiện trạng thái bắn được xác định từ cảm biến FSR402.

Kết quả kiểm thử các thao tác điều khiển trong Unity được trình bày trong bảng sau:

Bảng 4.9: Kết quả kiểm thử thực tế khi tách riêng hướng chuyển động và trạng thái FSR.

Thao tác kiểm thử	Nhãn AI dự đoán	Giá trị FSR	Hành động trong Unity
Giữ tay ở tư thế gốc	IDLE	Không bóp	Đối tượng giữ trạng thái
Nghiêng tay sang trái	LEFT	Không bóp	Đối tượng di chuyển sang trái
Nghiêng tay sang phải	RIGHT	Không bóp	Đối tượng di chuyển sang phải
Đưa tay lên	UP	Không bóp	Đối tượng di chuyển lên
Đưa tay xuống	DOWN	Không bóp	Đối tượng di chuyển xuống
Giữ tay và nhấn FSR402	IDLE	Có bóp	Đối tượng thực hiện bắn
Nghiêng trái và nhấn FSR402	LEFT	Có bóp	Đối tượng sang trái và bắn
Nghiêng phải và nhấn FSR402	RIGHT	Có bóp	Đối tượng sang phải và bắn
Đưa tay lên và nhấn FSR402	UP	Có bóp	Đối tượng đi lên và bắn
Đưa tay xuống và nhấn FSR402	DOWN	Có bóp	Đối tượng đi xuống và bắn

Hình sau trình bày kết quả điều khiển đối tượng trong Unity khi hệ thống nhận dạng được hướng chuyển động và trạng thái bắn của người dùng.



Hình 4.11: Kết quả điều khiển đối tượng trong Unity khi kiểm thử thực tế.

Kết quả kiểm thử thực tế cho thấy hệ thống có thể kết hợp mô hình AI và cảm biến FSR402 để điều khiển đối tượng trong Unity. Mô hình AI đảm nhiệm việc xác định hướng chuyển động, còn cảm biến FSR402 đảm nhiệm việc xác định thao tác bắn. Khi cảm biến được gán cố định và người dùng thực hiện thao tác rõ ràng, hệ thống có khả năng nhận dạng đúng các hướng di chuyển và kích hoạt bắn theo lực tác động. Tuy nhiên, trong một số trường hợp người dùng chuyển động quá nhanh, cảm biến bị lệch hoặc tư thế gốc không ổn định, kết quả dự đoán có thể dao động trong thời gian ngắn. Vì vậy, việc hiệu chuẩn ban đầu, cố định cảm biến và lọc kết quả đầu ra là các yếu tố quan trọng giúp hệ thống hoạt động ổn định hơn.

Như vậy, quá trình đánh giá mô hình AI không chỉ dừng lại ở kết quả huấn luyện trên tập dữ liệu có sẵn mà còn được kiểm chứng trong điều kiện chạy thực tế. Kết quả train cho thấy mô hình có khả năng học và phân loại 5 hướng chuyển động tay, trong khi kết quả test thực tế cho thấy hệ thống có thể kết hợp kết quả AI với giá trị FSR402 để tạo thành hai lệnh điều khiển chính là move và shoot. ““

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Ưu điểm hạn chế và hướng phát triển

5.1.1 Ưu điểm

Hệ thống sau khi thi công và thử nghiệm đã đạt được một số kết quả nhất định, đáp ứng được mục tiêu cơ bản của đề tài là nhận dạng chuyển động tay theo thời gian thực và ứng dụng kết quả nhận dạng để điều khiển đối tượng trong môi trường trò chơi. Hệ thống đã kết hợp được các khối phần cứng cảm biến, vi điều khiển, truyền thông không dây, chương trình nhận dạng bằng trí tuệ nhân tạo và môi trường mô phỏng Unity thành một mô hình thực nghiệm hoàn chỉnh.

Các ưu điểm chính của hệ thống gồm:

- Hệ thống có thể thu nhận dữ liệu chuyển động tay bằng hai cảm biến MPU được bố trí tại cánh tay và cẳng tay người dùng.
- Hệ thống có thể thu nhận dữ liệu lực tác động bằng cảm biến FSR, từ đó bổ sung thêm thao tác đặc biệt trong quá trình điều khiển trò chơi.
- STM32 đảm nhiệm việc đọc dữ liệu cảm biến, lọc nhiễu, tính toán góc nghiêng và xử lý sơ bộ trước khi truyền đi, giúp dữ liệu đầu vào của mô hình AI ổn định hơn.
- Dữ liệu cảm biến không được truyền hoàn toàn ở dạng thô mà được xử lý thành các đặc trưng cần thiết, giúp giảm kích thước gói dữ liệu và thuận tiện cho quá trình nhận dạng.
- ESP32 hỗ trợ kết nối Wi-Fi và truyền dữ liệu đến máy tính thông qua giao thức UDP, giúp hệ thống giảm độ trễ so với phương án truyền qua nền tảng trung gian.
- Chương trình Python có thể nhận dữ liệu cảm biến từ ESP32, tiền xử lý dữ liệu, đưa vào mô hình AI và tạo ra kết quả nhận dạng chuyển động tay.
- Mô hình AI có thể phân loại các trạng thái chuyển động tay cơ bản như IDLE, LEFT, RIGHT, UP và DOWN.

- Giá trị FSR402 được sử dụng để xác định trạng thái thao tác đặc biệt, giúp hệ thống có thêm chức năng điều khiển ngoài các hướng di chuyển cơ bản.
- Unity có thể nhận kết quả điều khiển từ chương trình Python thông qua UDP và mô phỏng chuyển động của đối tượng trò chơi theo thao tác tay người dùng.
- Hệ thống được thiết kế theo từng khối riêng biệt nên thuận tiện cho việc kiểm tra, sửa lỗi, thay đổi chương trình và mở rộng chức năng trong tương lai.

Ưu điểm lớn nhất của hệ thống là kết hợp được phần cứng nhúng, xử lý dữ liệu cảm biến, truyền dữ liệu không dây bằng UDP, mô hình trí tuệ nhân tạo và môi trường trò chơi Unity vào cùng một mô hình thực nghiệm. Hệ thống không chỉ dừng lại ở việc đọc dữ liệu cảm biến mà còn thực hiện xử lý, truyền nhận, nhận dạng và ứng dụng kết quả nhận dạng vào điều khiển đối tượng trong môi trường mô phỏng trực quan.

Việc sử dụng giao thức UDP giúp luồng dữ liệu giữa ESP32, chương trình Python và Unity trở nên trực tiếp hơn. Dữ liệu từ ESP32 được gửi đến máy tính trong cùng mạng nội bộ, sau đó chương trình Python gửi kết quả nhận dạng sang Unity để điều khiển đối tượng. Cách truyền này phù hợp với yêu cầu điều khiển gần thời gian thực vì giảm được sự phụ thuộc vào máy chủ trung gian và rút ngắn quá trình truyền dữ liệu.

Ngoài ra, mô hình Random Forest được sử dụng trong chương trình Python có ưu điểm là phù hợp với dữ liệu cảm biến dạng đặc trưng, dễ triển khai và có thời gian dự đoán tương đối nhanh sau khi mô hình đã được huấn luyện. Điều này giúp hệ thống có thể nhận dạng các thao tác tay cơ bản và tạo lệnh điều khiển tương ứng trong quá trình chạy thực tế.

5.1.2 Hạn chế

Bên cạnh các kết quả đạt được, hệ thống vẫn còn tồn tại một số hạn chế cần được cải thiện trong các phiên bản tiếp theo. Các hạn chế này chủ yếu liên quan đến độ ổn định khi đeo cảm biến, độ chính xác nhận dạng trong các trạng thái chuyển tiếp, chất lượng dữ liệu huấn luyện và độ trễ tổng thể của hệ thống.

Một số hạn chế chính của hệ thống gồm:

- Độ chính xác nhận dạng phụ thuộc nhiều vào cách gắn cảm biến trên tay người dùng.
- Nếu cảm biến MPU6050 bị lệch trong quá trình sử dụng, dữ liệu gốc có thể thay đổi

so với lúc thu thập dữ liệu huấn luyện, làm giảm độ chính xác của mô hình.

- Tư thế gốc khi hiệu chuẩn có ảnh hưởng trực tiếp đến dữ liệu đầu vào, do đó người dùng cần giữ tay đúng tư thế ban đầu trước khi vận hành hệ thống.
- Một số chuyển động có biên độ gần nhau có thể gây nhầm lẫn nếu người dùng thực hiện thao tác chưa dứt khoát.
- Dữ liệu huấn luyện được thu trong điều kiện thực nghiệm giới hạn nên khả năng tổng quát với nhiều người dùng khác nhau chưa cao.
- Giao thức UDP có độ trễ thấp nhưng không có cơ chế xác nhận gói tin, do đó hệ thống có thể gặp hiện tượng mất gói nếu kết nối Wi-Fi không ổn định.
- Thiết kế cơ khí đeo cảm biến vẫn còn ở mức mô hình thử nghiệm, chưa thật sự nhỏ gọn và ổn định như một thiết bị hoàn chỉnh.

Trong thực tế, hệ thống nhận dạng chuyển động tay chịu ảnh hưởng bởi nhiều yếu tố như vị trí đeo cảm biến, tốc độ chuyển động, lực tác động lên FSR, chất lượng kết nối Wi-Fi và cách người dùng thực hiện thao tác. Vì vậy, để hệ thống hoạt động ổn định, người dùng cần đeo cảm biến đúng vị trí, thực hiện bước hiệu chuẩn trước khi sử dụng và thao tác tương đối rõ ràng trong quá trình điều khiển.

Một hạn chế đáng chú ý là dữ liệu cảm biến có thể dao động trong thời điểm chuyển trạng thái. Ví dụ, khi người dùng chuyển từ trạng thái đứng yên sang đi xuống hoặc từ hướng trái sang hướng phải, dữ liệu có thể xuất hiện một số mẫu trung gian. Các mẫu này có thể làm mô hình dự đoán nhầm trong thời gian ngắn. Đây là vấn đề thường gặp trong các hệ thống nhận dạng chuyển động theo thời gian thực, đặc biệt khi hệ thống phải xử lý từng gói dữ liệu liên tục.

Ngoài ra, mặc dù UDP giúp giảm độ trễ so với các phương án truyền dữ liệu qua nền tảng trung gian, giao thức này không đảm bảo tất cả các gói tin đều được nhận đầy đủ. Vì vậy, chương trình cần sử dụng `packet_id` để kiểm tra thứ tự gói dữ liệu, tránh xử lý trùng lặp và hỗ trợ phát hiện trường hợp mất gói. Nếu mạng Wi-Fi không ổn định hoặc máy tính xử lý chậm, dữ liệu truyền đến chương trình Python hoặc Unity có thể không đều, ảnh hưởng đến độ mượt của đối tượng trong trò chơi.

Về mô hình AI, kết quả nhận dạng phụ thuộc lớn vào chất lượng bộ dữ liệu huấn luyện. Nếu dữ liệu chưa đủ đa dạng về người dùng, cách đeo cảm biến, tốc độ chuyển

động và mức độ tác động lực, mô hình có thể hoạt động chưa ổn định khi điều kiện sử dụng thay đổi. Do đó, việc mở rộng và chuẩn hóa bộ dữ liệu là yếu tố quan trọng để cải thiện chất lượng nhận dạng của hệ thống.

5.1.3 Hướng phát triển

Để nâng cao hiệu quả hoạt động của hệ thống, một số hướng phát triển có thể thực hiện trong tương lai gồm:

- Tối ưu cơ khí đeo cảm biến để cảm biến được cố định chắc chắn hơn trên tay người dùng, hạn chế sai lệch dữ liệu trong quá trình vận hành.
- Thiết kế mạch in và vỏ bảo vệ hoàn chỉnh để hệ thống gọn hơn, bền hơn và thuận tiện hơn khi sử dụng.
- Mở rộng tập dữ liệu huấn luyện với nhiều người dùng, nhiều tốc độ chuyển động và nhiều điều kiện thử nghiệm khác nhau.
- Thu thập thêm dữ liệu ở các trạng thái chuyển tiếp giữa các thao tác để mô hình nhận dạng ổn định hơn khi người dùng thay đổi chuyển động nhanh.
- Tối ưu mô hình AI để giảm thời gian dự đoán, tăng độ chính xác và hạn chế nhầm lẫn giữa các chuyển động có đặc trưng gần nhau.
- Bổ sung cơ chế lọc kết quả đầu ra, chẳng hạn như bỏ phiếu theo nhiều mẫu liên tiếp hoặc làm mượt nhãn dự đoán, nhằm giảm hiện tượng nhảy nhót.
- Tối ưu chương trình Python theo hướng ưu tiên xử lý gói dữ liệu mới nhất, hạn chế hàng đợi dữ liệu quá dài để giảm độ trễ.
- Cải thiện luồng truyền UDP bằng cách sử dụng mã gói, thời gian gửi và cơ chế bỏ qua gói cũ để đảm bảo Unity luôn nhận được trạng thái điều khiển mới nhất.
- Tối ưu chương trình Unity để đối tượng di chuyển mượt hơn, phản hồi nhanh hơn và phù hợp hơn với tốc độ gửi lệnh từ chương trình Python.
- Nghiên cứu tích hợp mô hình AI trực tiếp vào thiết bị nhúng hoặc tối ưu hệ thống theo hướng giảm phụ thuộc vào máy tính trung gian.
- Mở rộng số lượng thao tác điều khiển, chẳng hạn như xoay, tăng tốc, giảm tốc hoặc các thao tác đặc biệt khác trong trò chơi.

- Thử nghiệm thêm các mô hình học máy khác để so sánh với Random Forest và lựa chọn phương án phù hợp hơn cho hệ thống thời gian thực.

Trong các hướng phát triển trên, việc tối ưu độ trễ và mở rộng dữ liệu huấn luyện là hai nội dung quan trọng nhất. Nếu giảm được thời gian truyền nhận và xử lý dữ liệu, hệ thống sẽ phản hồi nhanh hơn, giúp trải nghiệm điều khiển trong trò chơi trở nên tự nhiên hơn. Đồng thời, nếu bộ dữ liệu huấn luyện được mở rộng với nhiều người dùng và nhiều điều kiện sử dụng, mô hình AI sẽ có khả năng nhận dạng ổn định hơn, hạn chế phụ thuộc vào một cách đeo cảm biến hoặc một kiểu thao tác nhất định.

Một hướng phát triển quan trọng khác là cải thiện cơ chế xử lý kết quả đầu ra của mô hình. Trong quá trình vận hành thực tế, mô hình có thể dự đoán sai trong một vài mẫu khi người dùng đang chuyển động. Nếu bổ sung bộ lọc nhiễu hoặc cơ chế xác nhận chuyển trạng thái, hệ thống có thể giảm hiện tượng nhảy lệnh và giúp đối tượng trong Unity di chuyển ổn định hơn.

Bên cạnh đó, hệ thống có thể được phát triển theo hướng thiết bị đeo hoàn chỉnh. Các dây nối giữa cảm biến, STM32 và ESP32 có thể được rút gọn; mạch điện có thể được thiết kế lại trên PCB; nguồn cấp có thể được tích hợp bằng pin sạc; toàn bộ thiết bị có thể được đặt trong vỏ bảo vệ nhỏ gọn. Điều này giúp hệ thống dễ sử dụng hơn và phù hợp hơn với các ứng dụng tương tác người – máy trong thực tế.

Tóm lại, hệ thống đã đáp ứng được mục tiêu cơ bản của đề tài là nhận dạng chuyển động tay theo thời gian thực và điều khiển đối tượng trò chơi. Kết quả thực nghiệm cho thấy sự kết hợp giữa cảm biến MPU6050, FSR402, STM32, ESP32, giao thức UDP, mô hình AI và Unity có thể tạo thành một hệ thống điều khiển chuyển động tay hoàn chỉnh. Tuy vẫn còn một số hạn chế về độ trễ, dữ liệu huấn luyện, độ ổn định khi đeo cảm biến và khả năng tổng quát với nhiều người dùng, hệ thống có khả năng phát triển tiếp để ứng dụng trong các bài toán điều khiển tương tác người – máy.

Tài liệu tham khảo

- [1] A. S. Kundu, O. Mazumder, P. K. Lenka, and S. Bhaumik, “Hand Gesture Recognition Based Omnidirectional Wheelchair Control Using IMU and EMG Sensors,” *Journal of Intelligent & Robotic Systems*, vol. 91, pp. 529–541, 2018. [Online]. Available: <https://link.springer.com/article/10.1007/s10846-017-0725-0>
- [2] S. Jiang, B. Lv, X. Sheng, C. Zhang, H. Wang, and P. B. Shull, “Development of a Real-Time Hand Gesture Recognition Wristband Based on sEMG and IMU Sensing,” Conference paper, 2016, accessed: 2026-06-13. [Online]. Available: https://www.wearablesystems.org/s/Jiangshuo2016_conf.pdf
- [3] M. Georgi, C. Amma, and T. Schultz, “Recognizing Hand and Finger Gestures with IMU Based Motion and EMG Based Muscle Activity Sensing,” in *Proceedings of the International Conference on Bio-inspired Systems and Signal Processing*, 2015, pp. 99–108. [Online]. Available: https://www.csl.uni-bremen.de/cms/images/documents/publications/GeorgiAmmaSchultz_RecognizingHandAndFingerGesturesWithIMUBasedMotionAndEMGBasedMuscleActivitySensing.pdf
- [4] K. Suri and R. Gupta, “Classification of Hand Gestures from Wearable IMUs Using Deep Neural Network,” arXiv preprint, 2020, accessed: 2026-06-13. [Online]. Available: <https://arxiv.org/abs/2005.00410>
- [5] M. Kralik and M. Suppa, “WaveGlove: Transformer-Based Hand Gesture Recognition Using Multiple Inertial Sensors,” arXiv preprint, 2021, accessed: 2026-06-12. [Online]. Available: <https://arxiv.org/abs/2105.01753>
- [6] X. Xu, J. Gong, C. Brum, L. Liang, B. Suh, S. K. Gupta, Y. Agarwal, L. Lindsey, R. Kang, B. Shahsavari, T. Nguyen, H. Nieto, S. E. Hudson, C. Maalouf, J. S. Mousavi, and G. Laput, “Enabling Hand Gesture Customization on Wrist-Worn Devices,” in *Proceedings of the CHI Conference on Human Factors in Computing Systems*, 2022, pp. 1–19. [Online]. Available: <https://3dvar.com/Xu2022Enabling.pdf>
- [7] W. Jiang, X. Ye, R. Chen, F. Su, M. Lin, Y. Ma, Y. Zhu, and S. Huang,

“Wearable On-Device Deep Learning System for Hand Gesture Recognition Based on FPGA Accelerator,” *Mathematical Biosciences and Engineering*, vol. 18, no. 1, pp. 132–153, 2021. [Online]. Available: <https://www.aimspress.com/article/id/5fb9b07ba35de3b25382f81>

- [8] C. K. Mummadi, F. P. P. Leo, K. D. Verma, S. Kasireddy, P. M. Scholl, and K. Van Laerhoven, “Real-Time Embedded Recognition of Sign Language Alphabet Fingerspelling in an IMU-Based Glove,” in *Proceedings of the International Workshop on Sensor-Based Activity Recognition and Interaction*, 2017, pp. 1–6. [Online]. Available: https://ubi29.informatik.uni-siegen.de/usi/pdf/ubi_iwoar2017b.pdf